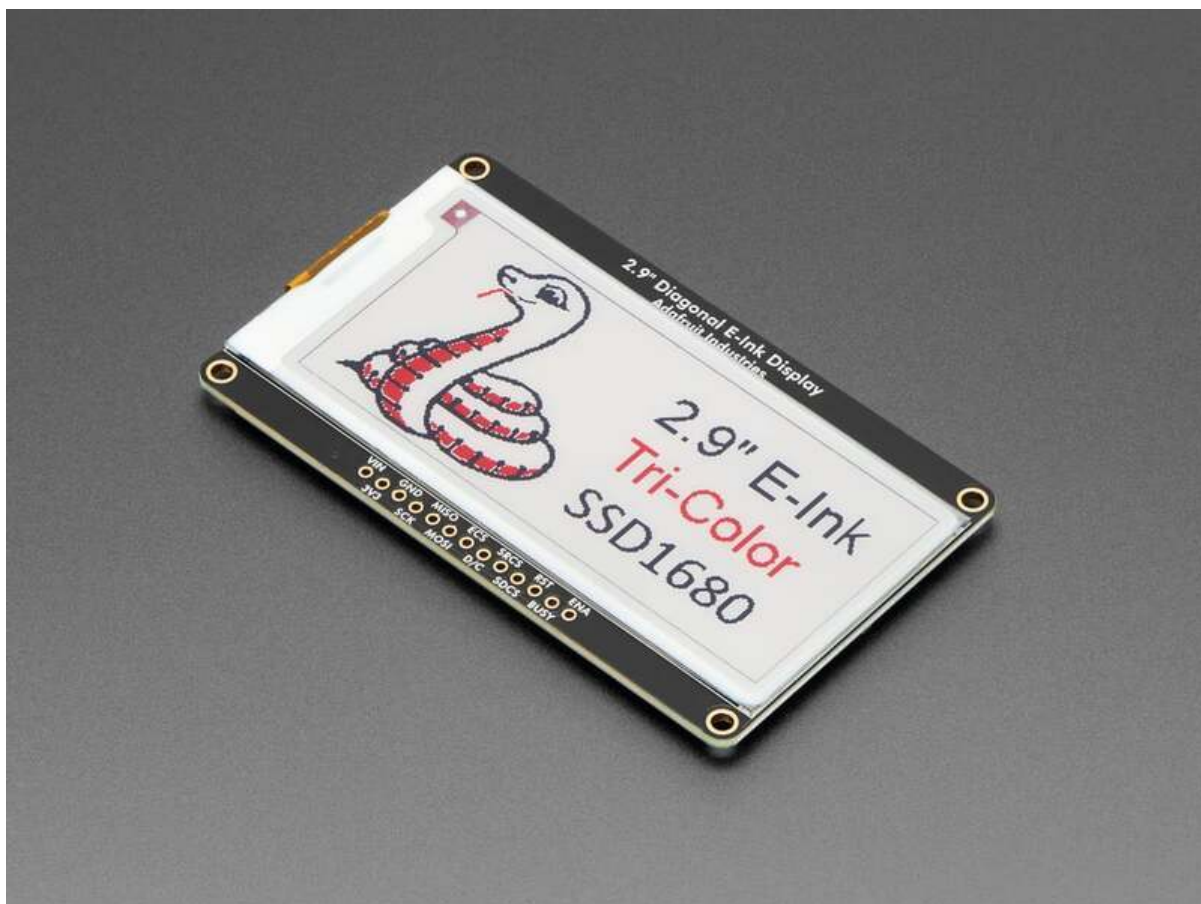




Adafruit 2.9" eInk Display Breakouts and FeatherWings

Created by Melissa LeBlanc-Williams



<https://learn.adafruit.com/adafruit-2-9-eink-display-breakouts-and-featherwings>

Last updated on 2026-07-06 05:53:06 PM UTC

Table of Contents

Overview	5
• 2.9" Tri-Color eInk Display and FeatherWing Revision • We have multiple 2.9" EPD displays: • 2.9" Tri-Color eInk Display Revision History • 2.9" Tri-Color eInk FeatherWing Revision History	
Pinouts	9
• 2.9" Tri-Color eInk Display and FeatherWing Revision • eInk Breakout Friend • Power Pins • Data Control Pins • FeatherWing Connections • 2.9" Tri-Color eInk FeatherWing Revision History • FeatherWing Buttons	
EYESPI	15
• The EYESPI Connector and Cables • Wiring Your EYESPI Display • EYESPI Pins	
Plugging in an EYESPI Cable	19
Assembly	21
• Assembly • Add the E-Ink Display • And Solder!	
Wiring	24
• EYESPI Connector • Breakout Wiring • FeatherWing Connection • Python Wiring	
Usage and Expectations	27
Arduino Setup	28
Arduino Usage	29
• 2.9" Monochrome 296x128 Pixel Display • 2.9" Tri-Color 296x128 Pixel Display • 2.9" Tri-Color eInk Display and FeatherWing Revision • 2.9" Grayscale 296x128 Pixel Display • Configure Pins • Configure Display Type & Size • Upload Sketch	
Arduino Bitmaps	35
• Tri-Color and Mono Display Demo • GrayScale Display Demo	
CircuitPython Monochrome Usage	40
• CircuitPython eInk displayio Library Installation	

- [Image File](#)
- [Monochrome Display Usage](#)

CircuitPython Grayscale Usage 44

- [CircuitPython eInk displayio Library Installation](#)
- [SSD1680 Updated Version](#)
- [Image File](#)
- [Grayscale Display Usage](#)

CircuitPython Tri-Color Usage 49

- [2.9" Tri-Color eInk Display and FeatherWing Revision](#)
- [CircuitPython eInk displayio Library Installation](#)
- [Image File](#)
- [Adafruit_CircuitPython_UC8151D](#)
- [UC8151D Display Usage](#)
- [Adafruit_CircuitPython_IL0373](#)

Python Setup 59

- [Setup Virtual Environment](#)
- [Install Adafruit_Blinka](#)
- [Python Installation of EPD Library](#)
- [Download font5x8.bin](#)
- [DejaVu TTF Font](#)
- [Pillow Library](#)
- [Chip Enable Lines](#)

Python Usage 62

- [2.9" Tri-Color eInk Display and FeatherWing Revision](#)
- [Monochrome Example](#)
- [Tri-Color Example](#)
- [Full Example Code](#)
- [Bitmap Example](#)
- [Image Drawing with Pillow](#)
- [Drawing Shapes and Text with Pillow](#)

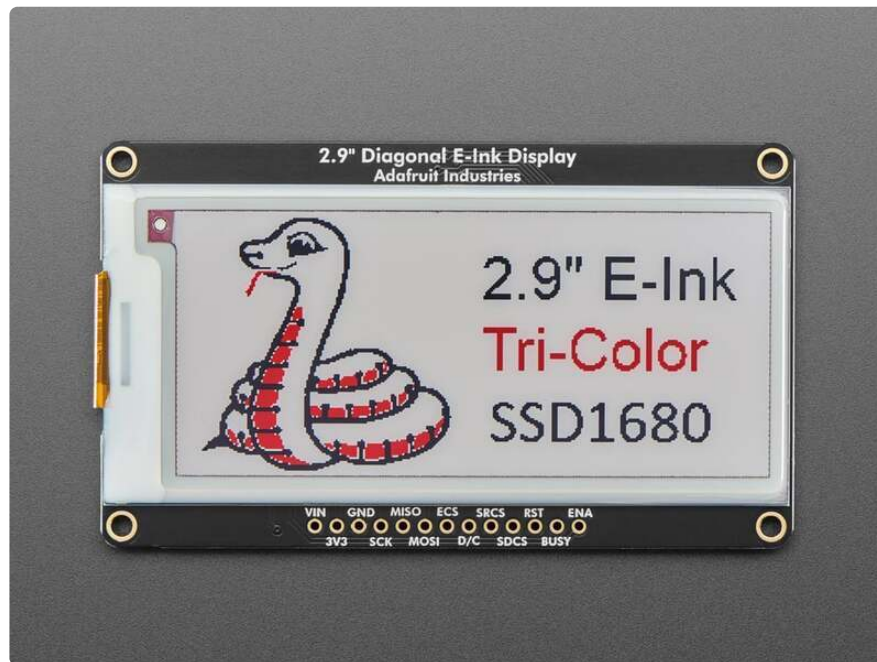
WipperSnapper Wiring & Test 84

- [What is WipperSnapper](#)
- [Wiring](#)
- [Usage](#)

Downloads 93

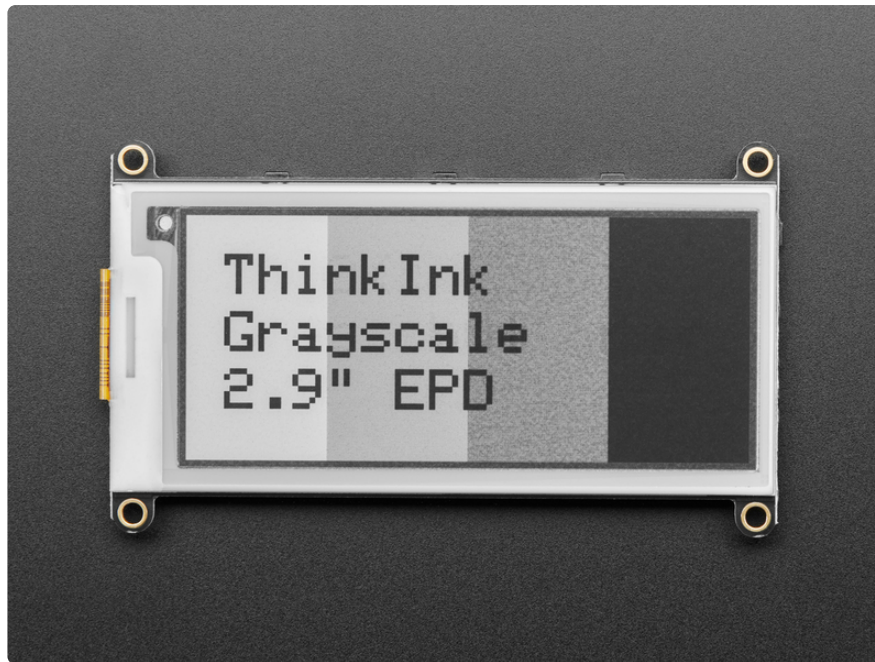
- [2.9" Tri-Color eInk Display Revision History](#)
- [2.9" Tri-Color eInk FeatherWing Revision History](#)
- [Files](#)
- [Schematic & Fabrication Prints](#)
- [2.9" FeatherWing](#)
- [eInk Friends](#)

Overview



 Text 2.9" Tri-Color elnk Display and FeatherWing Revision As of emphasized June 20, 2025 - The tri-color display and FeatherWing have with a been updated (old ones discontinued) to now use a spiffy yellow modern SSD1680 chip - we have support in CircuitPython/ exclamation:Python/Arduino for it!

Easy e-paper finally comes to microcontrollers with these breakouts, shields and friends that are designed to make it a breeze to add a monochrome, tri-color, or grayscale elnk display. Chances are you've seen one of those new-fangled 'e-readers' like the Kindle or Nook. They have gigantic electronic paper 'static' displays - that means the image stays on the display even when power is completely disconnected. The image is also high contrast and very daylight readable. It really does look just like printed paper!



Adafruit has liked these displays for a long time, but they were never designed for makers to use. Finally, we decided to make our own!



We have multiple 2.9" EPD displays:
Breakouts and Flexibles

- We have a newer [296x128 Tri-Color display \(http://adafru.it/1028\)](http://adafru.it/1028) with the UC8151D chipset, which has black and red ink pixels and a white-ish background.

- The older tri-color breakout, which had the same Product ID, had the IL0373 chipset. This display is no longer offered.
- The [2.9" monochrome flexible display \(http://adafru.it/4262\)](http://adafru.it/4262) has a resolution of 296x128 and is flexible. For this display, you will probably want to pick up an [e-Ink Breakout Friend \(http://adafru.it/4224\)](http://adafru.it/4224) or [e-Ink Feather Friend \(http://adafru.it/4446\)](http://adafru.it/4446). There is an [extension cable \(http://adafru.it/4230\)](http://adafru.it/4230) available for this type of connection also.

FeatherWings

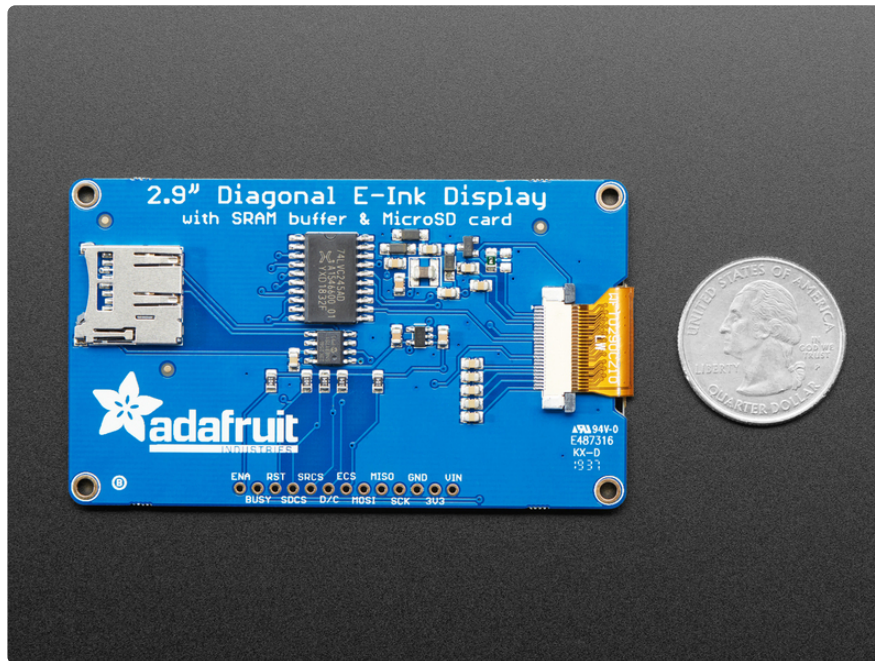
- The newer [296x128 Tri-Color FeatherWing \(http://adafru.it/4778\)](http://adafru.it/4778) with the SSD1675 chipset.
- The older tri-color FeatherWing with the IL0373 has the same display as the older breakout. This display is also no longer offered.
- The grayscale FeatherWing features 4 levels of grayscale. We have a [296x128 Grayscale FeatherWing \(http://adafru.it/4777\)](http://adafru.it/4777).



Using our Arduino library, you can create a 'frame buffer' with what pixels you want to have activated and then write that out to the display. Most simple breakouts leave it at that. But if you do the math, $296 \times 128 \text{ pixels} \times 2\text{-bits-per-pixel} = 9.5 \text{ KBytes}$. Which won't fit into many microcontroller memories. Heck, even if you do have 32KB of RAM, why waste 9KB?

So we did you a favor and tossed a small SRAM chip on the back. This chip shares the SPI port the elnk display uses, so you only need one extra pin. And, no more

frame-buffering! You can use the SRAM to set up whatever you want to display, then shuffle data from SRAM to elnk when you're ready. [The library we wrote does all the work for you \(https://adafru.it/BRK\)](https://adafru.it/BRK), you can just interface with it as if it were an [Adafruit_GFX compatible display \(https://adafru.it/BRK\)](https://adafru.it/BRK).



For ultra-low power usages, the onboard 3.3V regulator has the Enable pin brought out so you can shut down the power to the SRAM, MicroSD and display.

We even added on a MicroSD socket to the breakouts and FeatherWings so you can store images, text files, whatever you like to display. Everything is 3 or 5V logic safe so you can use it with any and all common Maker microcontrollers.

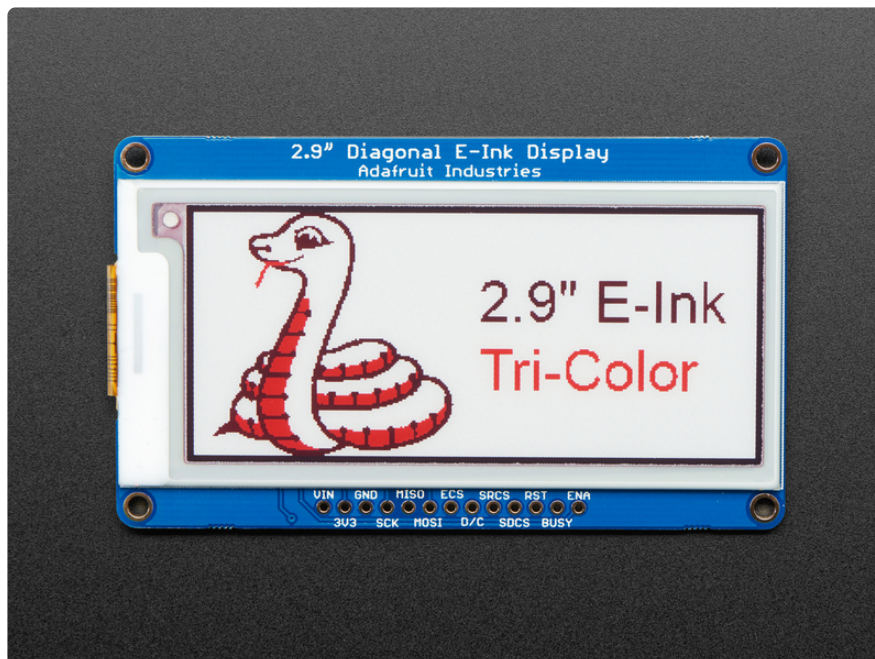
Text emphasized with a blue exclamation:

2.9" Tri-Color elnk Display Revision History As of June 20, 2025 - The display has been updated (old ones discontinued) to use a spiffy modern SSD1680 chip - we have support in CircuitPython/Python/Arduino for it! As of May 22th, 2024 – We've updated this PCB with a new EYESPI connector along with the Adafruit Penguin to make a lovely and legible silkscreen. As of November 16, 2022 - this revision has been updated to have a 2.9" Tri-Color elnk display only with a UC8151D chipset.

2.9" Tri-Color eInk FeatherWing Revision History As of June 24th, 2025 – The display has been updated (old one discontinued) it now uses a spiffy modern SSD1680 chip - we emphasized have support in CircuitPython/Python/Arduino for it! As of Sep 8, 2023 – we've updated this PCB with Adafruit Pinguin to exclamation:make a lovely and legible silkscreen - you may get the new PCB or the older version with vector fonts - both are identical other than the fancy silkscreen.

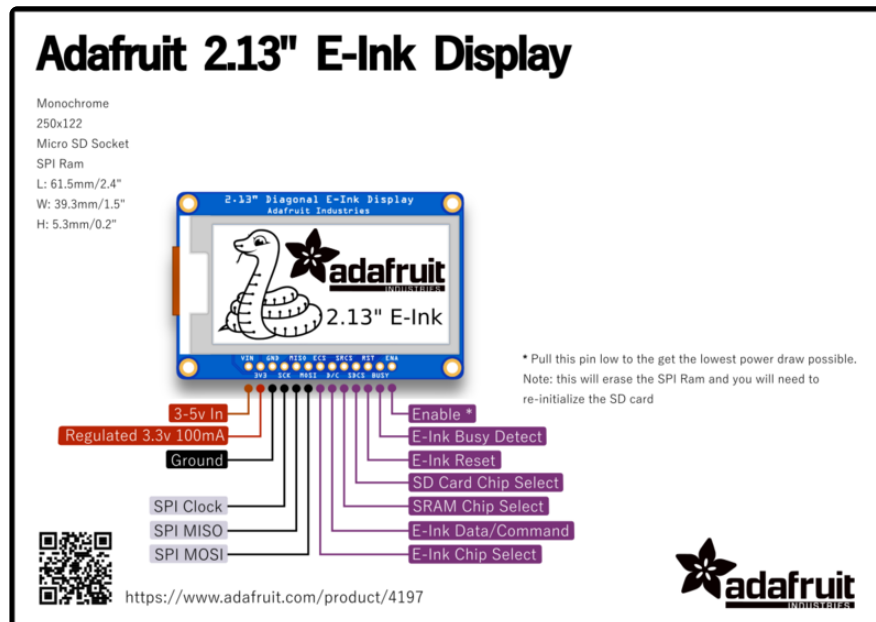
Pinouts

2.9" Tri-Color eInk Display and FeatherWing Revision As of June 20, 2025 - The tri-color display and FeatherWing have been updated (old ones discontinued) to now use a spiffy modern SSD1680 chip - we have support in CircuitPython/Python/Arduino for it!



This e-Paper display uses SPI to receive image data. Since the display is SPI, it was easy to add two more SPI devices to share the bus - an SPI SRAM chip and SPI-driven

SD card holder. There's quite a few pins and a variety of possible combinations for control depending on your needs

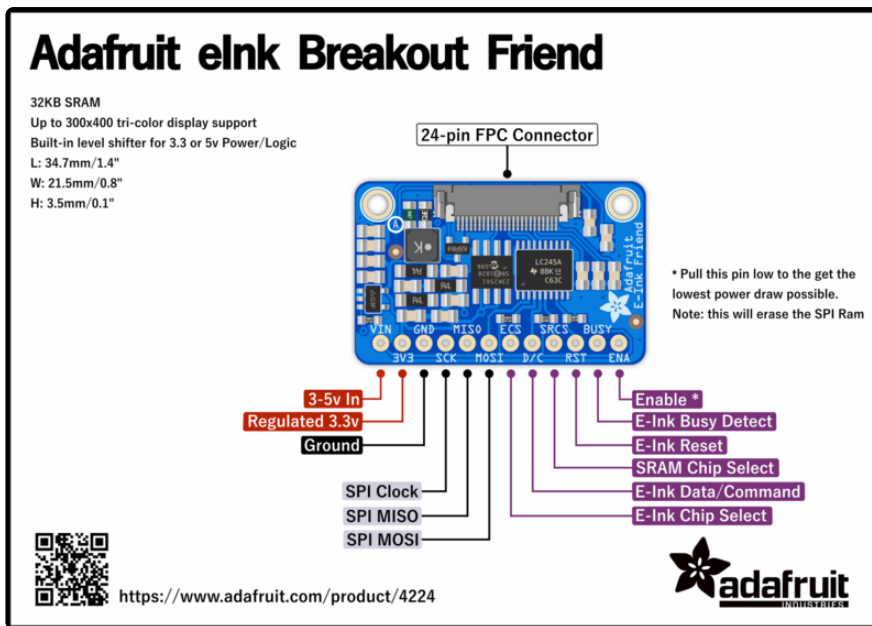


Text emphasized with a blue exclamation:

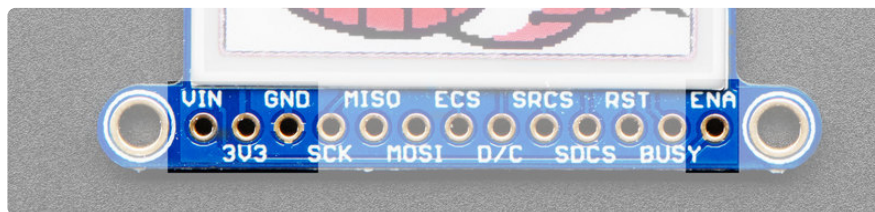
The pinouts are identical for the 2.13" and 2.9" E-Ink display!

eInk Breakout Friend

Connect a bare eInk display to this breakout to use it!

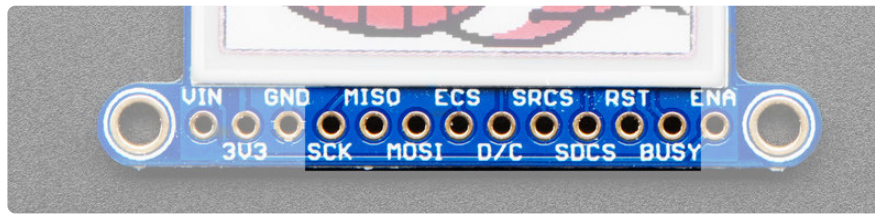


Power Pins



- **3-5V / Vin** - this is the power pin, connect to 3-5VDC - it has reverse polarity protection but try to wire it right!
- **3.3V** out - this is the 3.3V output from the onboard regulator, you can 'borrow' about 100mA if you need to power some other 3.3V logic devices
- **GND** - this is the power and signal ground pin
- **ENABLE** - This pin is all the way on the right. It is connected to the enable pin on the onboard regulator that powers everything. If you want to really have the lowest possible power draw, pull this pin low! Note that if you do so you will cut power to the eInk display but also the SPI RAM (thus erasing it) and the SD card (which means you'll have to re-initialize it when you re-power)

Data Control Pins



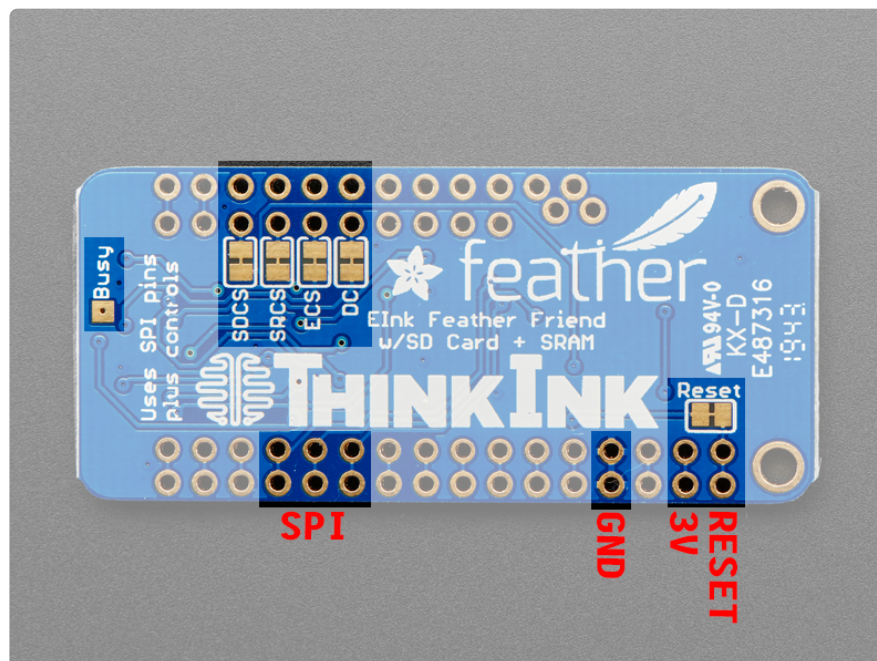
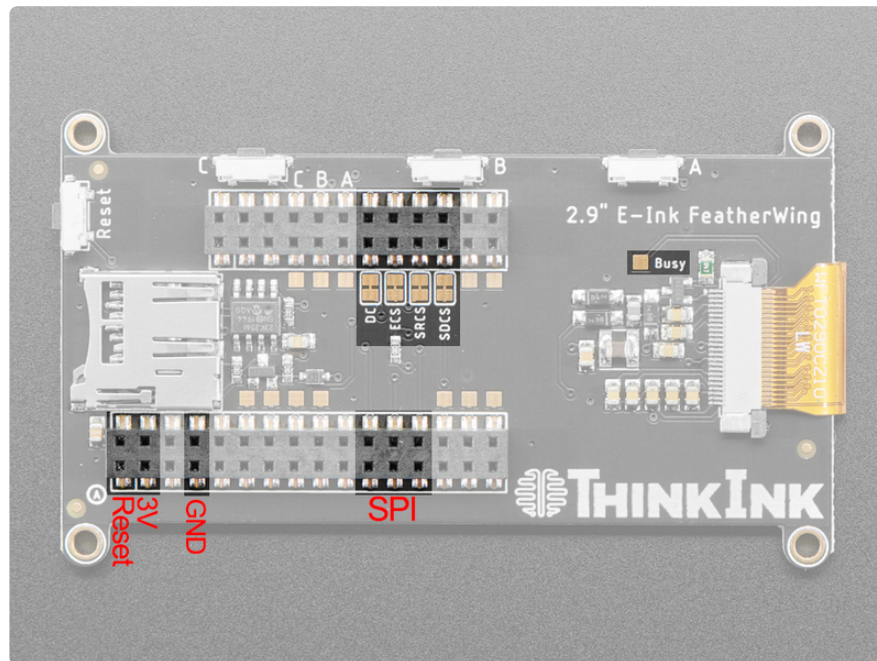
- **SCK** - this is the SPI clock input pin, required for e-Ink, SRAM and SD card
- **MISO** - this is the SPI Microcontroller In Serial Out pin, its used for the SD card and SRAM. It isn't used for the e-Ink display which is write-only, however you'll likely be using the SRAM to buffer the display so connect this one too!
- **MOSI** - this is the SPI Microcontroller Out Serial In pin, it is used to send data from the microcontroller to the SD card, SRAM and e-Ink display
- **ECS** - this is the **E**-Ink **C**hip **S**elect, required for controlling the display
- **D/C** - this is the e-Ink **D**ata/**C**ommand pin, required for controlling the display
- **SRCS** - this is the **S**RAM **C**hip **S**elect, required for communicating with the onboard RAM chip.
- **SDCS** - this is the **S**D card **C**hip **S**elect, required for communicating with the onboard SD card holder. You can leave this disconnected if you aren't going to access SD cards
- **RST** - this is the E-Ink **R**e**S**e**T** pin, you may be able to share this with your microcontroller reset pin but if you can, connect it to a digital pin.
- **BUSY** - this is the e-Ink busy detect pin, and is optional if you don't want to connect the pin (in which case the code will just wait an approximate number of seconds)

FeatherWing Connections

2.9" Tri-Color eInk FeatherWing Revision History As of June 24th, 2025 – The display has been updated (old one discontinued) it now uses a spiffy modern SSD1680 chip - we emphasized have support in CircuitPython/Python/Arduino for it! As of Sep 8, 2023 – we've updated this PCB with Adafruit Pinguin to exclamation:make a lovely and legible silkscreen - you may get the new



PCB or the older version with vector fonts - both are identical other than the fancy silkscreen.



The FeatherWing eInk Display and eInk Feather Friend are a little more compact but have just about the same pins as the breakout

- **SPI MOSI/MISO/SCK** are on the FeatherWing SPI connection pads

SD CS, SRAM CS, EINK CS and DC are in order after the two I2C pins. The numbers of the pins these correspond to will differ from board to board. However, on 32u4/328p/M0/M4/nRF52840 and many other boards you will see the following connections

- **SD CS** to Pin **D5**
- **SRAM CS** to Pin **D6**
- **EINK CS** to Pin **D9**
- **EINK DC** to Pin **D10**

If you do not plan to use the SD card, you can cut the trace to SD CS. Likewise for SRAM CS.

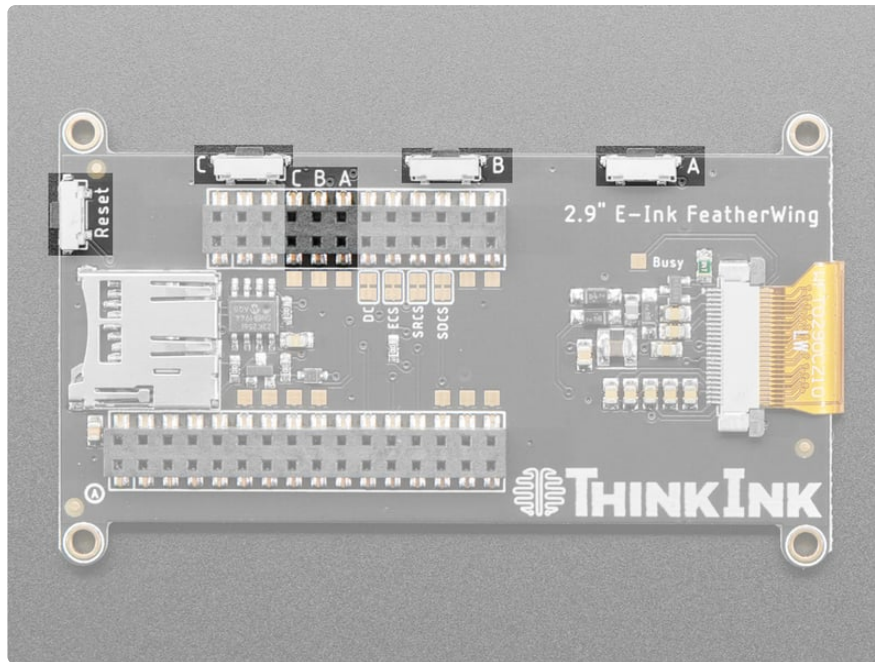
The **Reset** pin for the E-Ink display is connected to an auto-reset circuit and also to the Feather Reset pin, so it will reset when you press the reset button.

The **Busy** pin is available on a breakout pad, you can solder it to a wire and connect to a pin if you need it - we figure most people will just use a fixed delay.

FeatherWing Buttons

The 2.9" eInk FeatherWings also feature a few buttons. The exact pin the buttons are connected to can vary by the specific feather.

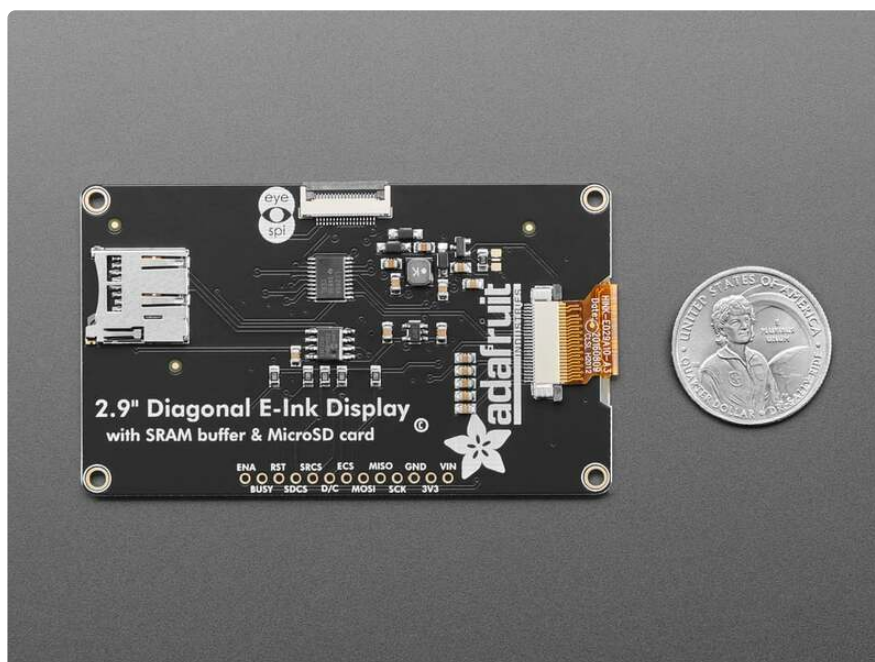
- **A** - this is button A and is connected to Pin **D11** on many Feathers, but may vary.
- **B** - this is button B and is connected to Pin **D12** on many Feathers, but may vary.
- **C** - this is button C and is connected to Pin **D13** on many Feathers, but may vary.
- **Reset** - this button will reset the attached Feather.



EYESPI



Text emphasized with a green exclamation: Currently, only the 2.9" Tri-Color eInk display has been updated to have an EYESPI connector.

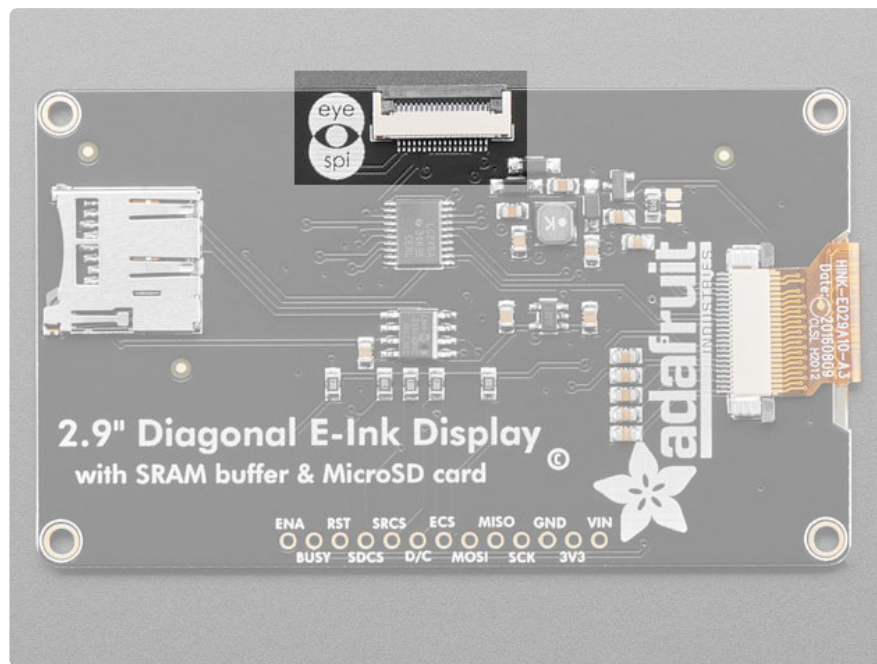


This display now comes with an **EYESPI connector**. This connector allows you to connect your display without soldering. There are [EYESPI cables](https://adafru.it/18eT) (<https://adafru.it/18eT>) available in multiple lengths, which means you can find one to fit any project. This is especially useful if your project requires the display to be freestanding, and not tied directly into a breadboard. Inspired by the popularity of STEMMA QT, it provides plug-n-play for displays!

The EYESPI Connector and Cables

The EYESPI connector is an 18 pin 0.5mm pitch FPC connector with a flip-top tab for locking in the associated flex cable. It is designed to allow you to connect a display, without needing to solder headers or wires to the display.

The EYESPI connector location on this display is indicated below.



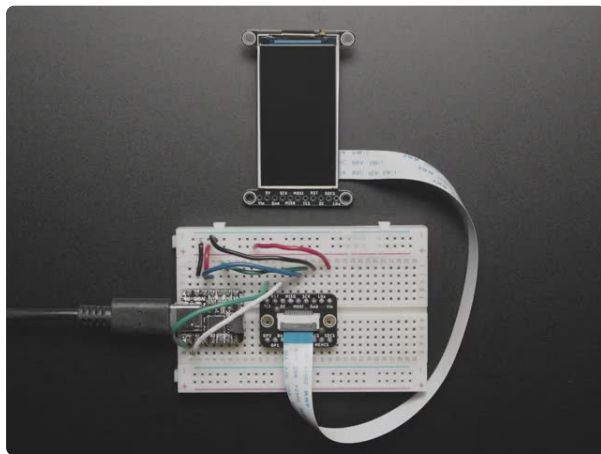
The EYESPI cables are 18 pin 0.5mm pitch flex cables. They are ~9.6mm wide, and designed to fit perfectly into the EYESPI connector. Adafruit currently offers EYESPI cables in three different lengths: [50mm](http://adafru.it/5462) (<http://adafru.it/5462>), [100mm](http://adafru.it/5239) (<http://adafru.it/5239>), and [200mm](http://adafru.it/5240) (<http://adafru.it/5240>).



Text emphasizedThe EYESPI connector is designed to work with 18-pin with a yellow exclamation: 0.5mm pitch flex cables. Other flex cables, such as Raspberry Pi camera flex cables, will not work!

Wiring Your EYESPI Display

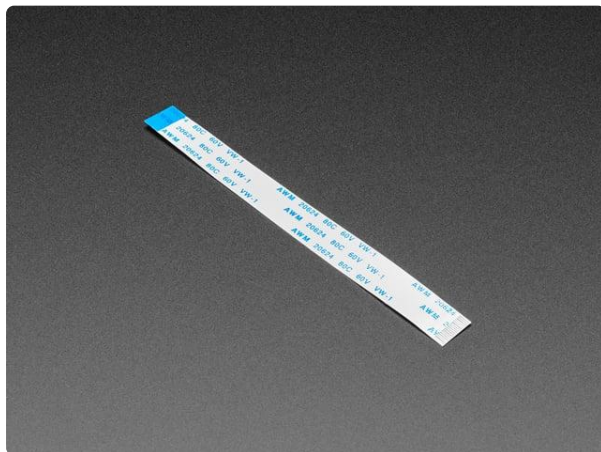
Wiring your EYESPI display to a microcontroller via the EYESPI connector requires the EYESPI breakout board and an EYESPI cable.



Adafruit EYESPI Breakout Board - 18 Pin FPC Connector

Our most recent display breakouts have come with a new feature: an 18-pin "EYESPI" standard FPC...

<https://www.adafruit.com/product/5613>



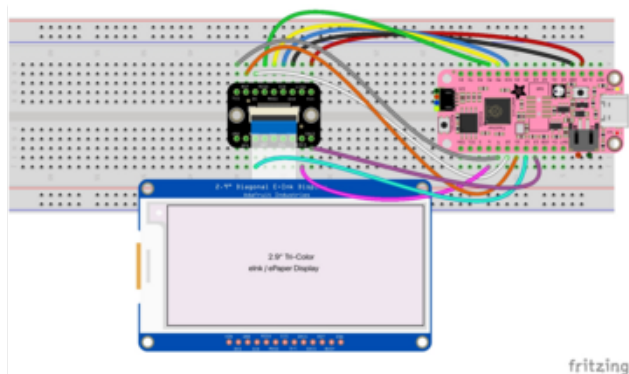
EYESPI Cable - 18 Pin 100mm long Flex PCB (FPC) A-B type

Connect this to that when a 18-pin FPC connector is needed. This 25 cm long cable is made of a flexible PCB. It's A-B style which means that pin one on one side will match...

<https://www.adafruit.com/product/5239>

The following example shows how to connect the 2.9" elnk display to a Feather RP2040 using the EYESPI breakout board.

Connect the following Feather pins to the associated EYESPI breakout pins:



- Breakout Vin to Feather Vin (red wire)
- Breakout Gnd to Feather Gnd (black wire)
- Breakout SCK to Feather SCK (blue wire)
- Breakout MOSI to Feather MOSI (yellow wire)
- Breakout MISO to Feather MISO (green wire)
- Breakout MEMCS to Feather pin 6 (pink wire)
- Breakout TCS to Feather pin 9 (grey wire)
- Breakout DC to Feather pin 10 (white wire)
- Breakout RST to Feather pin 11 (orange wire)
- Breakout BUSY to Feather pin 12 (cyan wire)
- Breakout SDCS to Feather pin 13 (purple wire)

Finally, connect your **display EYESPI connector** to the **breakout EYESPI connector** using an **EYESPI cable**. For details on using the EYESPI connector properly, visit [Plugging in an EYESPI Cable \(https://adafru.it/18eU\)](https://adafru.it/18eU).

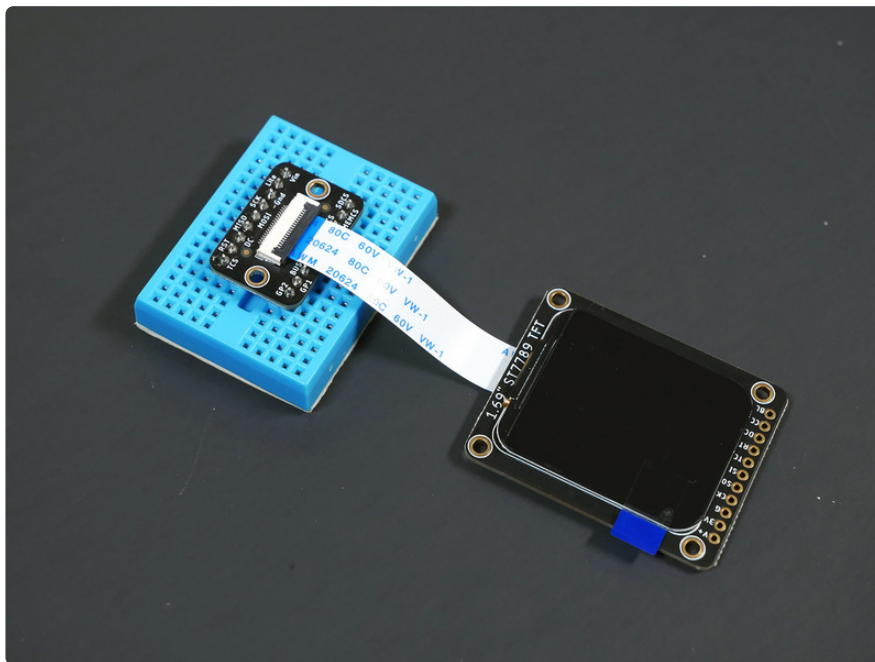
EYESPI Pins

Though there are 18 pins available on the EYESPI connector, many displays do not use all available pins. This display requires the following pins:

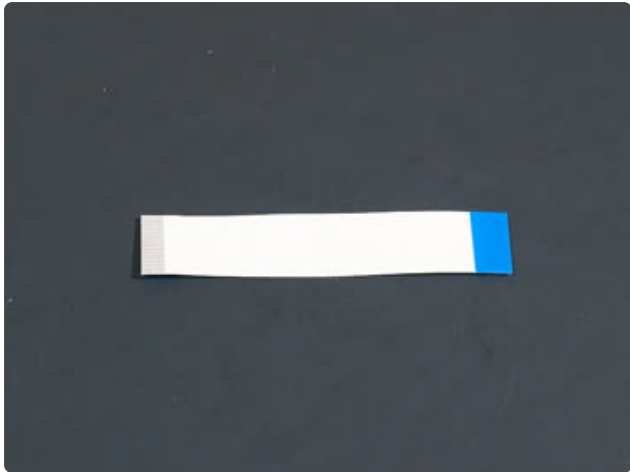
- **Vin** - This is the power pin. To power the board (and thus your display), connect to the same power as the logic level of your microcontroller, e.g. for a 3V micro like a Feather, use 3V, and for a 5V micro like an Arduino, use 5V
- **GND** - This is common ground for power and logic
- **SCK** - This is the SPI clock input pin
- **MOSI** - This is the SPI MOSI (**M**icrocontroller **O**ut / **S**erial **I**n) pin. It is used to send data from the microcontroller to the SD card and/or display

- **MISO** - This is the SPI MISO (**M**icrocontroller **I**n / **S**erial **O**ut) pin. It's used for the SD card. It isn't used for the display because it's write-only. It is 3.3V logic out (but can be read by 5V logic)
 - **TCS** - eInk Chip Select pin
 - **DC** - Data/command pin
 - **MEMCS** - **M**emory **C**hip **S**elect, required for communicating with the onboard RAM chip
 - **RST** - This is the display reset pin. Connecting to ground resets the display! It's best to have this pin controlled by the library so the display is reset cleanly, but you can also connect it to the microcontroller's Reset pin, which works for most cases. Often, there is an automatic-reset chip on the display which will reset it on power-up, making this connection unnecessary in that case
 - **BUSY** - e-Ink busy detect pin
 - **SDCS** - This is the SD card chip select pin. This pin is required for communicating with the SD card holder onboard the connected display
-

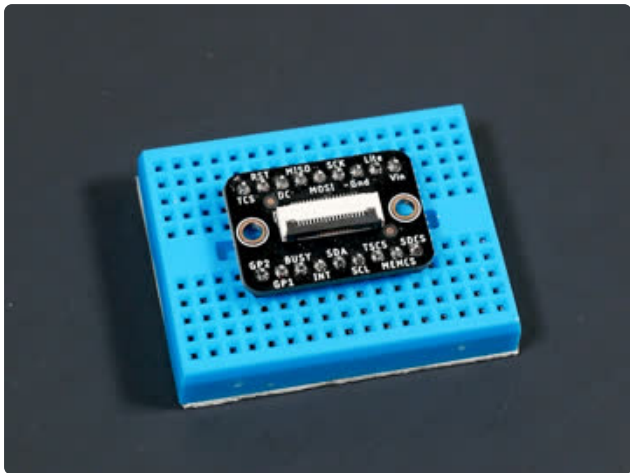
Plugging in an EYESPI Cable



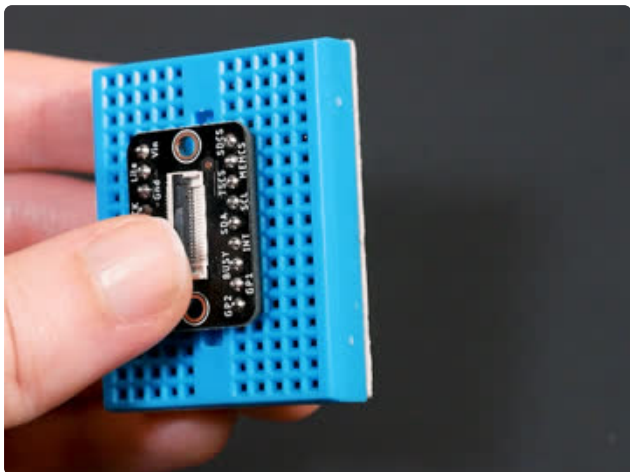
You can connect an EYESPI compatible display to the EYESPI breakout board using an EYESPI cable. An EYESPI cable is an 18 pin flexible PCB (FPC). The FPC can only be connected properly in one orientation, so be sure to follow the steps below to ensure that your display and breakout are plugged in properly.



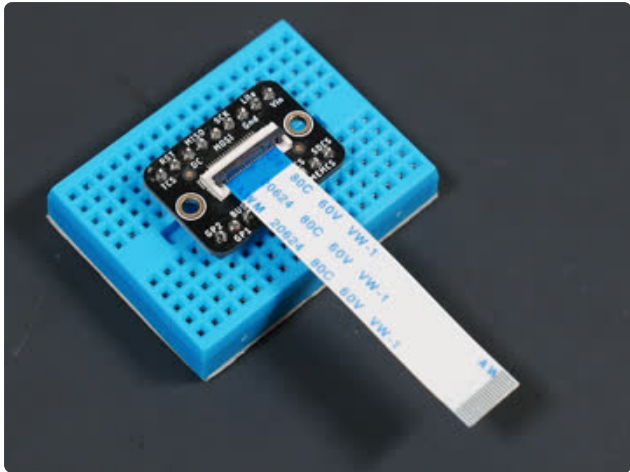
Each EYESPI cable has **blue stripes** on either end. On the other side of the cable, underneath the blue stripe, are the connector pins that make contact with the FPC connector pins on the display or breakout.



To begin inserting an EYESPI cable to an FPC connector, gently lift the FPC connector black latch up.



Then, insert the EYESPI cable into the open FPC connector by sliding the cable into the connector. You want to **see the blue stripe facing up towards you**. This inserts the cable pins into the FPC connector.

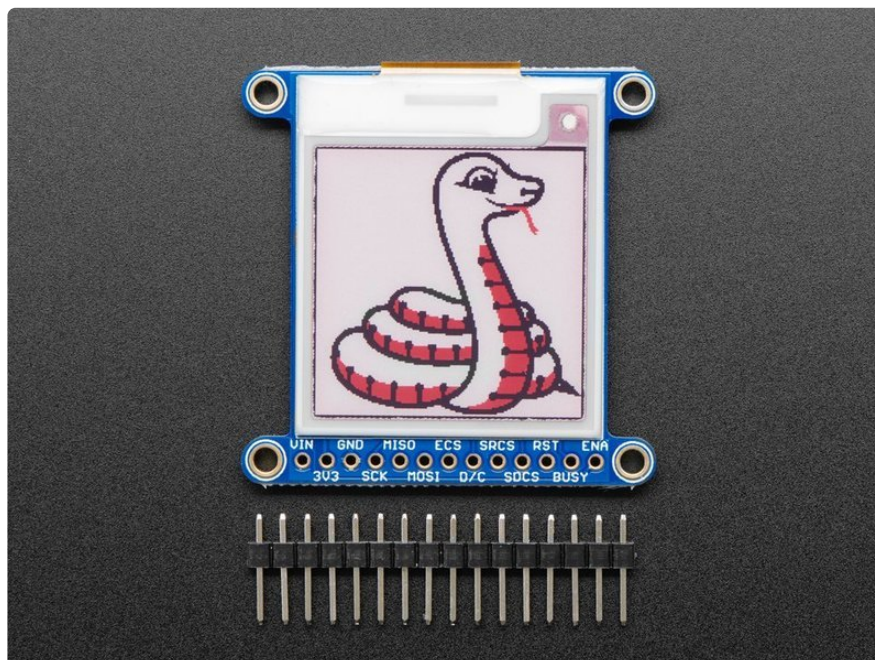


To secure the cable, lower the FPC connector latch onto the EYESPI cable.



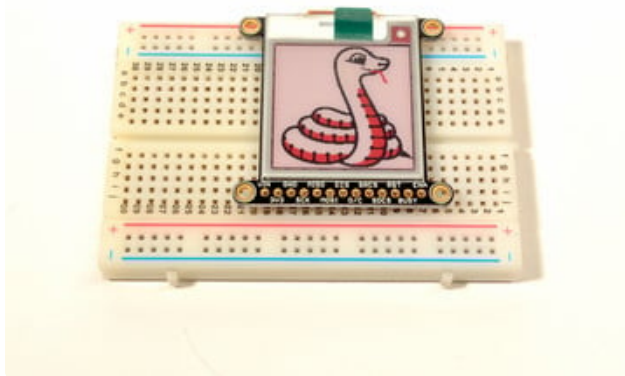
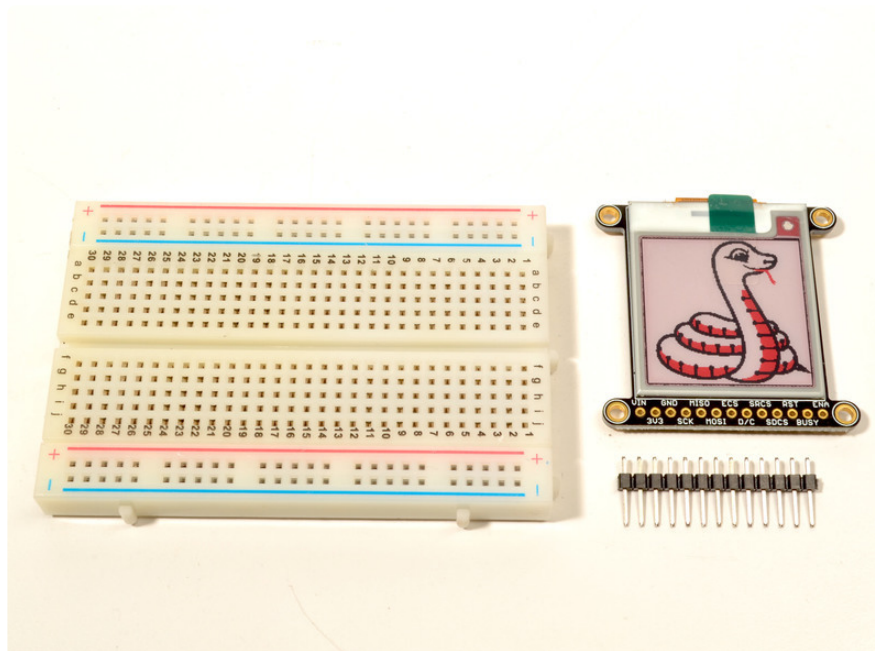
Repeat this process for the FPC connector on your display. Again, ensure that the **blue stripe** on either end of the cable is facing up.

Assembly



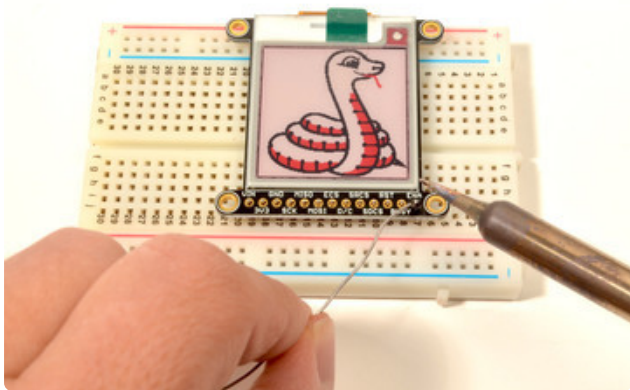
Assembly

Cut the header down to length if necessary. It will be easier to solder if you insert it into a breadboard - **long pins down**



Add the E-Ink Display

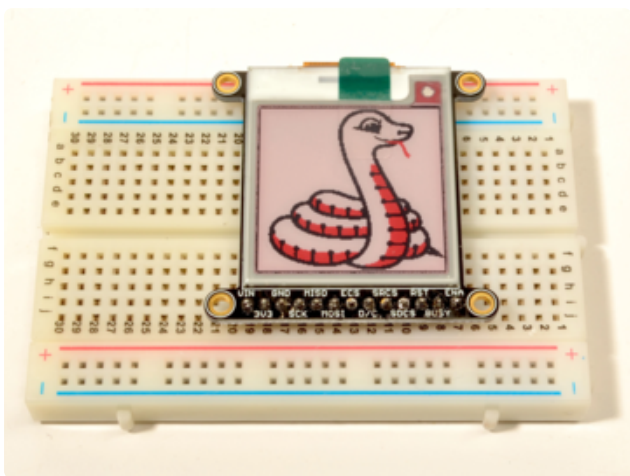
Place the board over the pins so that the short pins poke through the top of the breakout pads



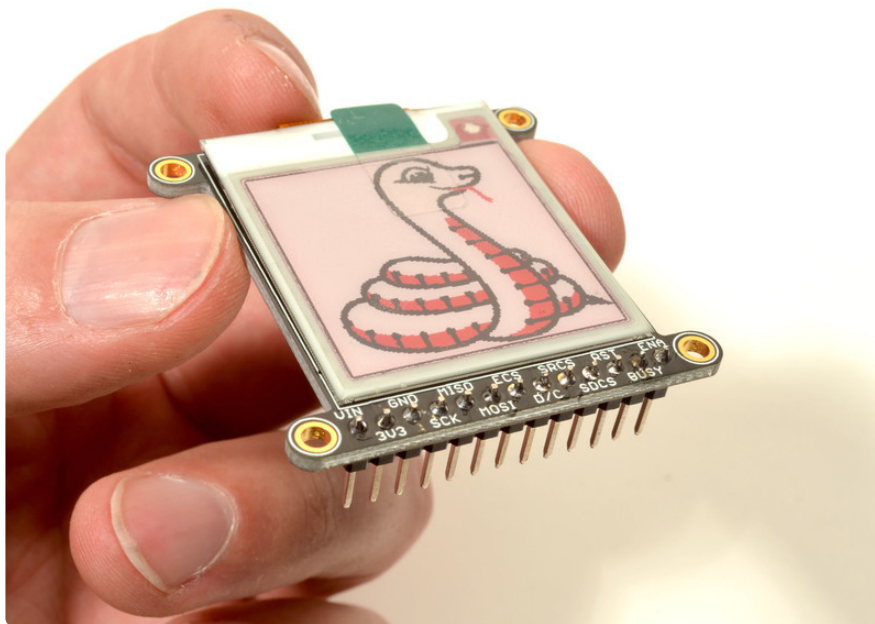
And Solder!

Be sure to solder all pins for reliable electrical contact.

(For tips on soldering, be sure to check out the [Guide to Excellent Soldering](https://adafruit.it/aTk) (<https://adafruit.it/aTk>)).



OK, you're done!



Wiring

EYESPI Connector

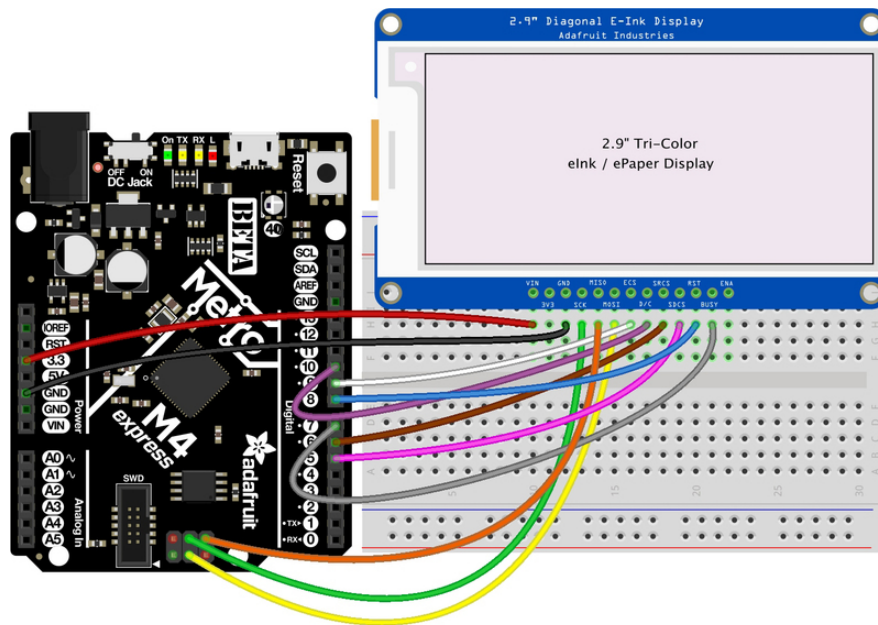
If you have one of the newer breakouts with an EYESPI connector on the back, you can use that with an 18-pin ribbon cable to connect it either directly to an adapter board such as the [EYESPI BFF for QT Py or Xiao](http://adafru.it/5772) (<http://adafru.it/5772>) or by using the [EYESPI Breakout](http://adafru.it/5613) (<http://adafru.it/5613>) and wiring that to your microcontroller. See the [EYESPI page](https://adafru.it/1a1u) (<https://adafru.it/1a1u>) for more information.

Breakout Wiring

Though it looks like a lot of connections, wiring up an eInk breakout is pretty straightforward! Below shows using hardware SPI to connect it to an Adafruit Metro M4.

- **Vin** connects to the microcontroller board's **5V** or **3.3V** power supply pin.
- **GND** connects to ground.
- **CLK** connects to SPI clock. It's easiest to connect it to **pin 3 of the ICSP header**.
- **MOSI** connects to SPI MOSI. It's easiest to connect it to **pin 4 of the ICSP header**.
- **MISO** connects to SPI MISO. It's easiest to connect it to **pin 1 of the ICSP header**.
- **ECS** connects to our e-Ink Chip Select pin. We'll be using **Digital 9**.
- **D/C** connects to our e-Ink data/command select pin. We'll be using **Digital 10**.

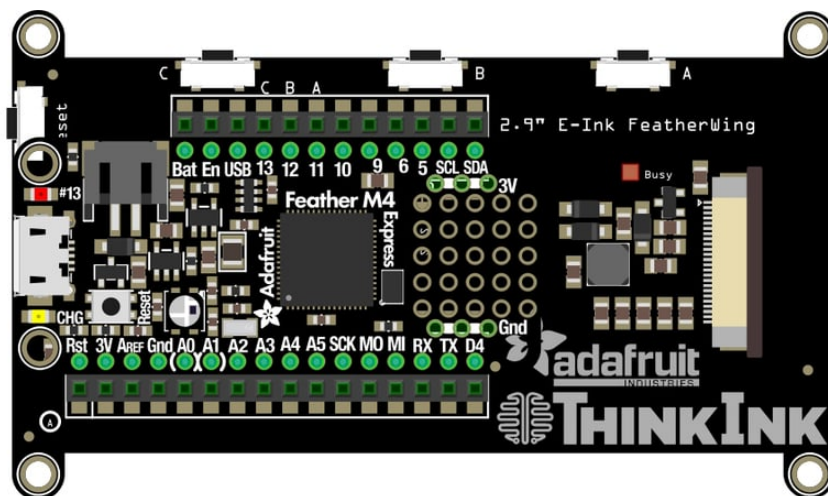
- **SRCS** connects to our SRAM Chip Select pin. We'll be using **Digital 6**.
- **RST** connects to our e-Ink reset pin. We'll be using **Digital 8**.
- **BUSY** connects to our e-Ink busy pin. We'll be using **Digital 7**.
- **SDCS** connects to our SD Card Chip Select pin. We'll be using **Digital 5**.



fritzing

FeatherWing Connection

FeatherWing usage is easy, simply plug your Feather board into the FeatherWing board.



fritzing

Python Wiring

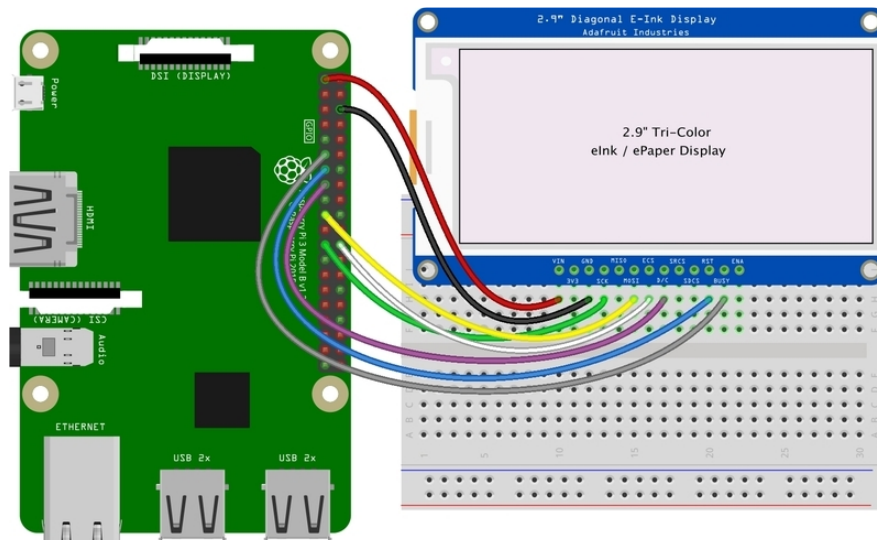


Text emphasized with a blue exclamation:

Wiring for the elnk friend breakout is the same as the Tri-Color breakout.

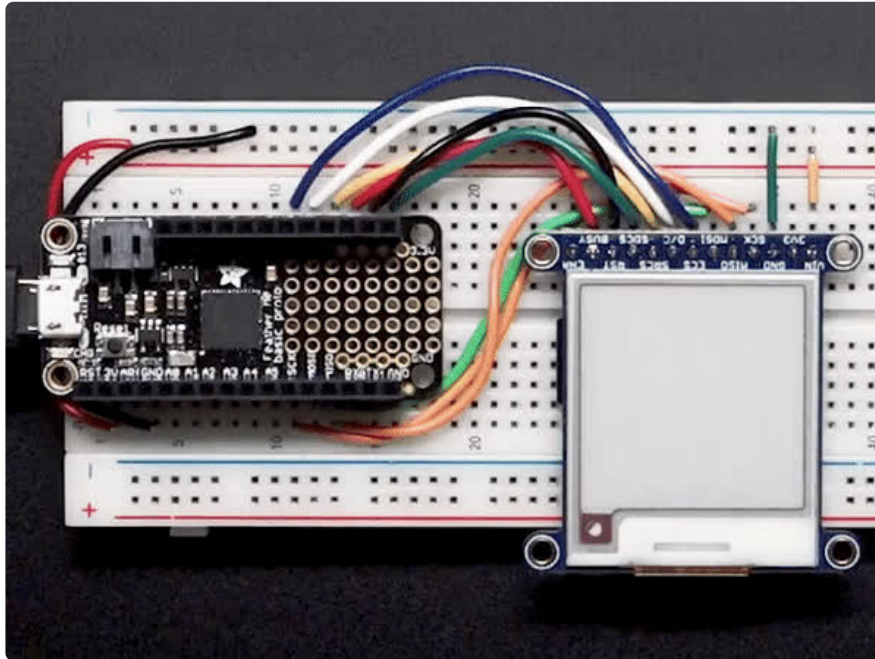
There are many Single Board Computers (SBC) so showing them all is impractical. The Raspberry Pi is the most common and it is shown below. Others are likely similar. The SPI peripheral may need to be enabled on such boards.

- Raspberry Pi 3.3 to display VIN
- Raspberry Pi GND to display GND
- Raspberry Pi SCLK to display SCK
- Raspberry Pi MOSI to display MOSI
- Raspberry Pi GPIO CE0 to display ECS
- Raspberry Pi GPIO 22 to display D/C
- Raspberry Pi GPIO 27 to display RST
- Raspberry Pi GPIO 17 to display BUSY



fritzing

Usage and Expectations



One thing to remember with these small e-ink screens is that its very slow compared to OLEDs, TFTs, or even 'memory displays'. **It will take may seconds to fully erase and replace an image**

There's also a recommended limit on refreshing - **you shouldn't refresh or change the display more than every 3 minutes (180 seconds).**

You don't have to refresh often, but with tri-color displays, the larger red ink dots will slowly rise, turning the display pinkish instead of white background. **To keep the background color clear and pale, refresh once a day**

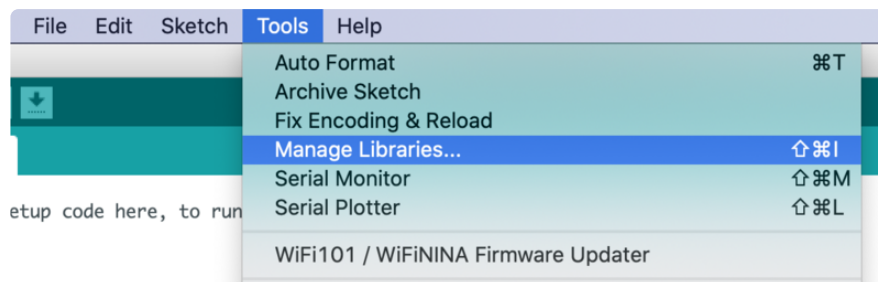


Text emphasized with Do not update more than once every 180 seconds or a red exclamation: you may permanently damage the display

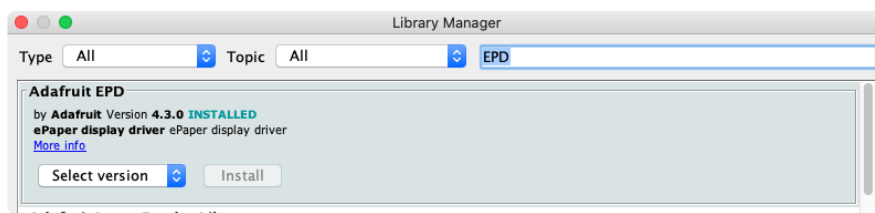
Arduino Setup

To use the display, you will need to [install the Adafruit_EPD library \(code on our github repository\) \(https://adafru.it/BRK\)](https://adafru.it/BRK). It is available from the Arduino library manager so we recommend using that.

From the IDE open up the library manager...



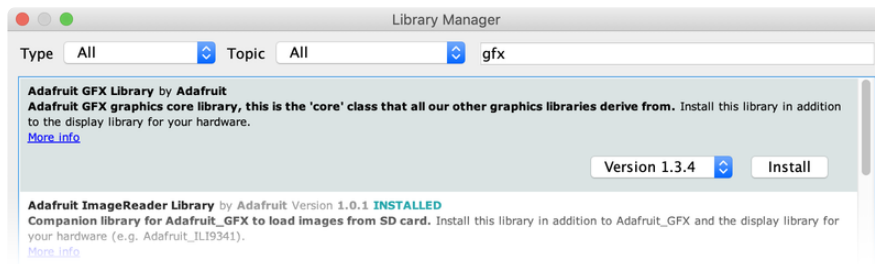
And type in **adafruit EPD** to locate the library. Click **Install**



If you would like to draw bitmaps, do the same with **adafruit ImageReader**, click **Install**



Do the same to install the latest **adafruit GFX** library, click **Install**



If using an earlier version of the Arduino IDE (pre-1.8.10), locate and install **Adafruit_BusIO** (newer versions handle this prerequisite automatically).

Arduino Usage

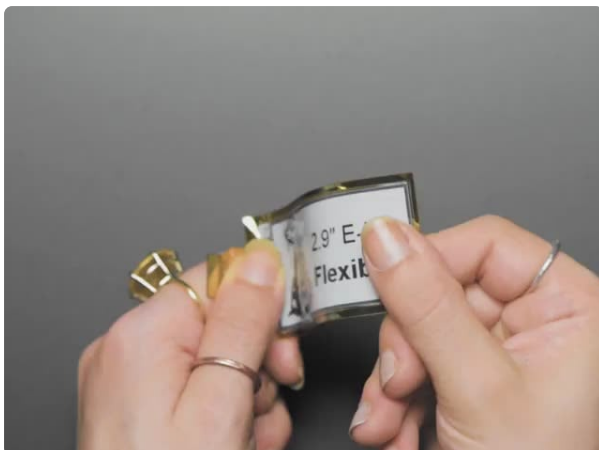
!

Text

Here is where the differences in the tri-color/monochrome and chipset/dimensions start mattering. Check carefully to make emphasized sure you are running the right example and creating the with a red matching ThinkInk type for your display or you won't see exclamation:anything happen on the EPD (or the image may be really weird looking).

2.9" Monochrome 296x128 Pixel Display

For the 296x128 Flexible Display, below is a monochrome demo.

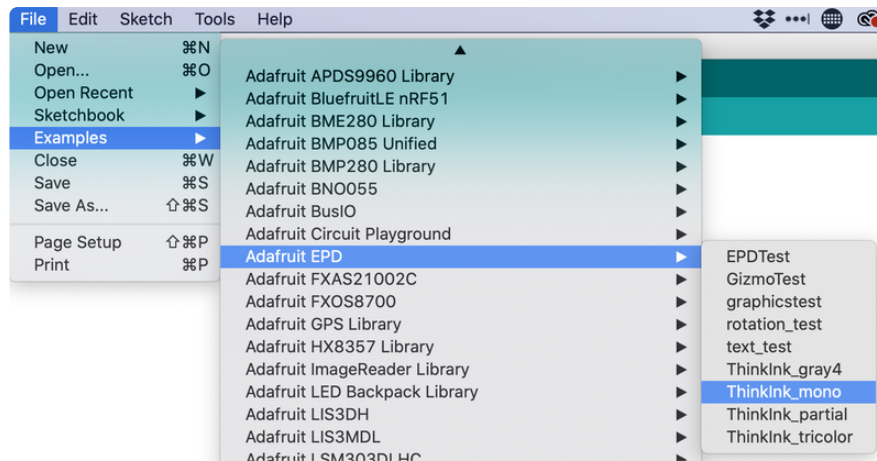


[2.9" Flexible 296x128 Monochrome eInk / ePaper Display](https://www.adafruit.com/product/4262)

Woah, the cyber-future is here! Flexible E-Ink has been demo'd at high-tech events for years but now you can actually get your paws on it. This display is true E-Ink / E-Paper,...

<https://www.adafruit.com/product/4262>

Open up **File→Examples→Adafruit_EPD→ThinkInk_mono**

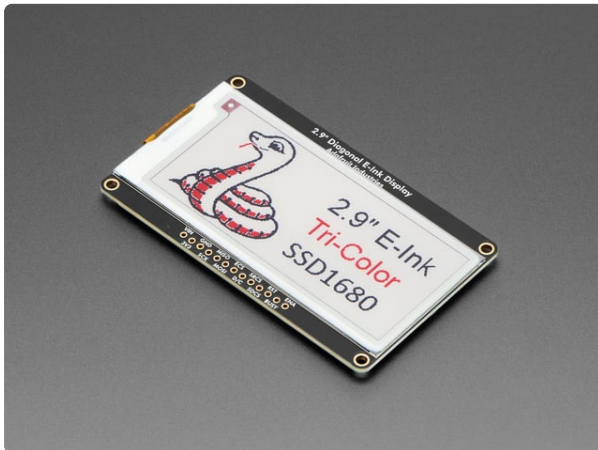


2.9" Tri-Color 296x128 Pixel Display

For the 296x128 Tri-Color display, below is a tri-color demo.



Text 2.9" Tri-Color elnk Display and FeatherWing Revision As of emphasized June 20, 2025 - The tri-color display and FeatherWing have with a been updated (old ones discontinued) to now use a spiffy yellow modern SSD1680 chip - we have support in CircuitPython/exclamation:Python/Arduino for it!



Adafruit 2.9" Red/Black/White elnk Display Breakout - THINK INK

Easy e-paper finally comes to microcontrollers, with this breakout that's designed to make it a breeze to add a tri-color elnk display. Chances are you've seen one of those...

<https://www.adafruit.com/product/1028>



Adafruit 2.9" Tri-Color eInk / ePaper Display FeatherWing

Easy e-paper comes to your Feather with this breakout that's designed to make it a breeze to add a tri-color eInk display.

Chances are you've seen one of those...

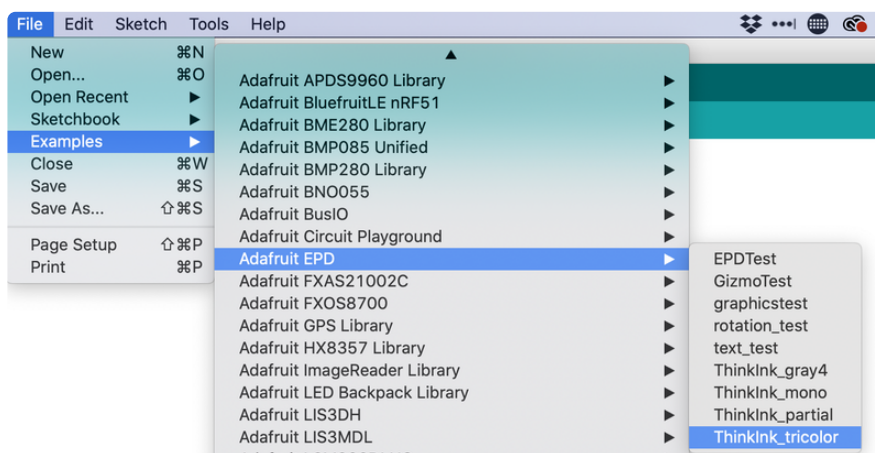
<https://www.adafruit.com/product/4778>



Text emphasized with a yellow exclamation:

The demo updates every 15 seconds which is fine for demonstrating the functionality for a short time, but we recommend not updating more often than 180 seconds if you are planning on running any code long term.

Open up **File**→**Examples**→**Adafruit_EPD**→**ThinkInk_tricolor**



2.9" Grayscale 296x128 Pixel Display

For the 296x128 Grayscale display, below is the grayscale demo.

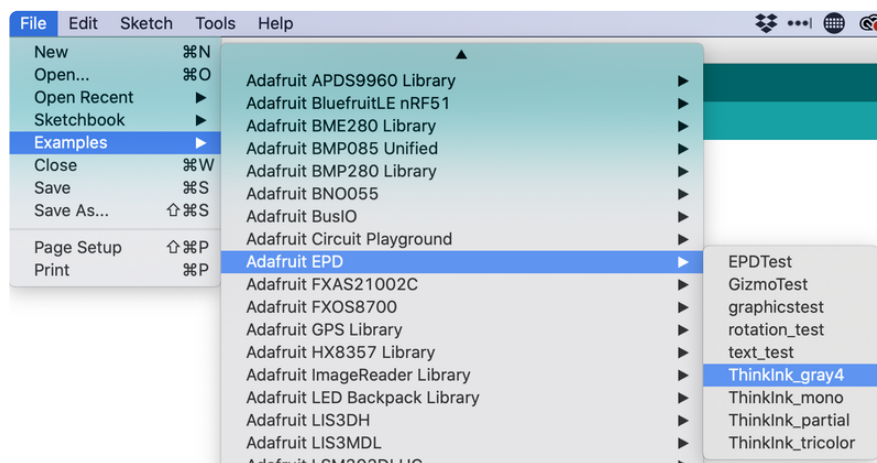


Adafruit 2.9" Grayscale eInk / ePaper Display FeatherWing

Easy e-paper comes to your Feather with this breakout that's designed to make it a breeze to add a monochrome eInk display. Chances are you've seen one of those...

<https://www.adafruit.com/product/4777>

Open up **File→Examples→Adafruit_EPD→ThinkInk_gray4**



Configure Pins



Text emphasized with a yellow exclamation:

For the FeatherWing you must be sure both EPD_RESET and EPD_BUSY are set to -1 since neither of these lines are connected or the E-Ink will not update.

No matter what display you have, you will need to verify that your pins match your wiring. At the top of the sketch find the lines that look like:

```
1 #define EPD_DC 10
2 #define EPD_CS 9
```

```

3 | #define SRAM_CS    6
4 | #define EPD_RESET  8 // can set to -1 and share with microcontroller Reset!
5 | #define EPD_BUSY   7 // can set to -1 to not use a pin (will wait a fixed delay)

```

If you are using the FeatherWing only: Change both `EPD_RESET` and `EPD_BUSY` to -1 since neither of these lines are connected on the FeatherWing.

If you wired the display differently than on the wiring page, adjust the pin numbers accordingly.

Configure Display Type & Size

Find the part of the script where you can pick which display is going to be used. The elnk displays are made up a combination of a Chipset and a Film in different sizes. Adafruit has narrowed it down to just a few choices between the size of the display, chipset, and film based on available combinations. In the sketch, it is sorted by size, so it's easy to find your display.

You will need to uncomment the appropriate initializer and and leave any other type commented:

- For the [2.9" 296x128 Monochrome Flexible Display](http://adafru.it/4262) (<http://adafru.it/4262>), you will use the `ThinkInk_290_Mono_M06` display initializer.
- For the **SSD1680** chipset version of the [2.9" 296x128 Tri-Color breakout](http://adafru.it/1028) (<http://adafru.it/1028>) or [2.9" 296x128 Tri-Color FeatherWing](http://adafru.it/4778) (<http://adafru.it/4778>), you will use the `ThinkInk_290_Tricolor_Z94` display initializer.
- For the **UC8151D** chipset version of the [2.9" 296x128 Tri-Color breakout](http://adafru.it/1028) (<http://adafru.it/1028>), you will use the `ThinkInk_290_Tricolor_Z13` display initializer.
- For the **IL0373** chipset version of the [2.9" 296x128 Tri-Color breakout](http://adafru.it/1028) (<http://adafru.it/1028>) or [2.9" 296x128 Tri-Color FeatherWing](http://adafru.it/4778) (<http://adafru.it/4778>), you will use the `ThinkInk_290_Tricolor_Z10` display initializer.
- For the [2.9" 296x128 Grayscale breakout](http://adafru.it/4086) (<http://adafru.it/4086>) , you will use the `ThinkInk_290_Grayscale4_T5` display initializer.

For example, for the mono 296x128 flexible display, uncomment this line, and comment any other line that is creating a ThinkInk display object

```

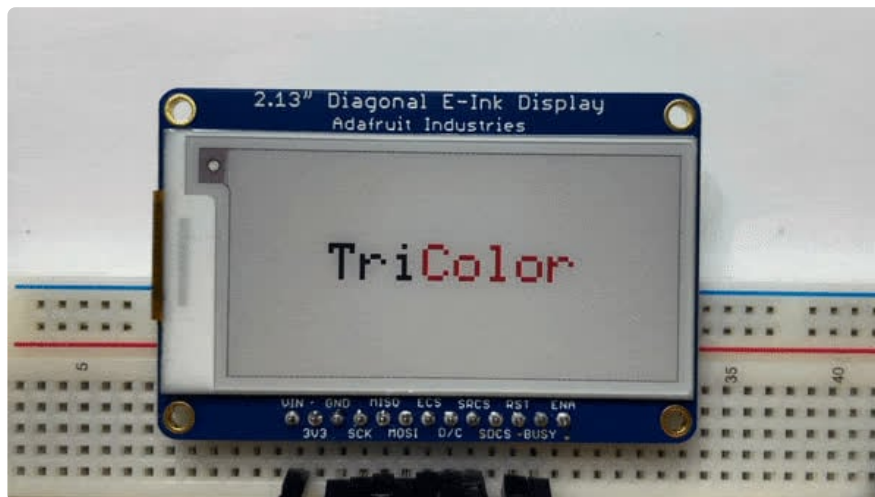
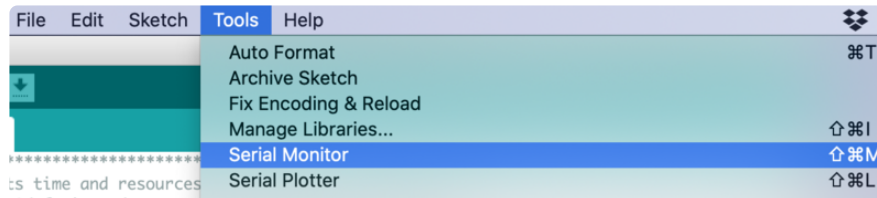
1 | // 2.9" Monochrome displays with 296x128 pixels and UC8151D chipset
2 | //ThinkInk_290_Mono_M06 display(EPD_DC, EPD_RESET, EPD_CS, SRAM_CS, EPD_BUSY);

```

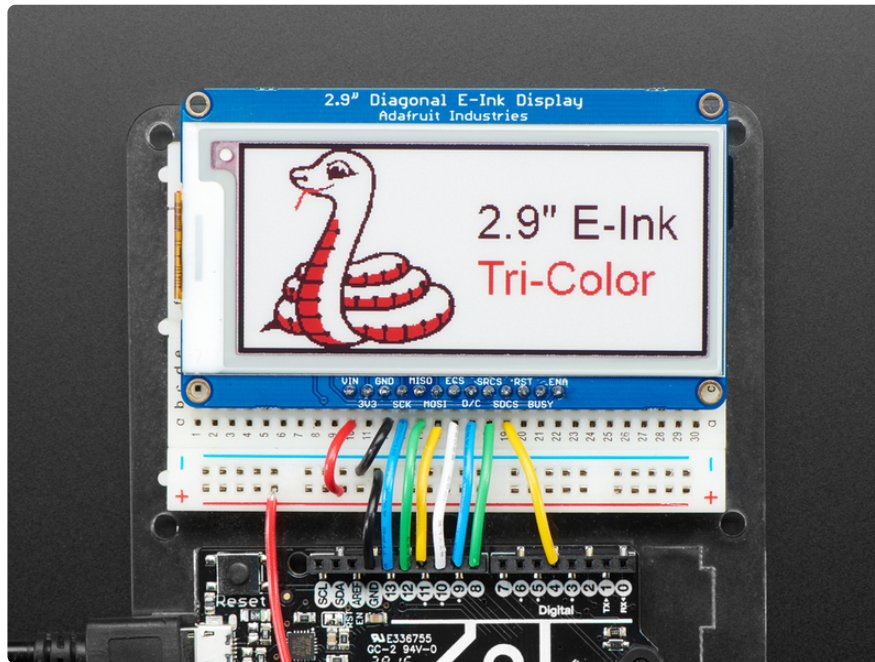

Upload Sketch

After checking the pinouts and the display type one more time, go ahead and upload the sketch to your board. Once it is done uploading, open the **Serial Monitor**.

The display should start running a series of display tests



Arduino Bitmaps



Not only can you draw shapes but you can also load images from the SD card, perfect for static images!

Tri-Color and Mono Display Demo

The Tri-color and Mono demo uses a single bitmap. The Blinka bitmaps below is used for the demonstration. Select the one that is correct for your display:



<https://adafru.it/19f2>



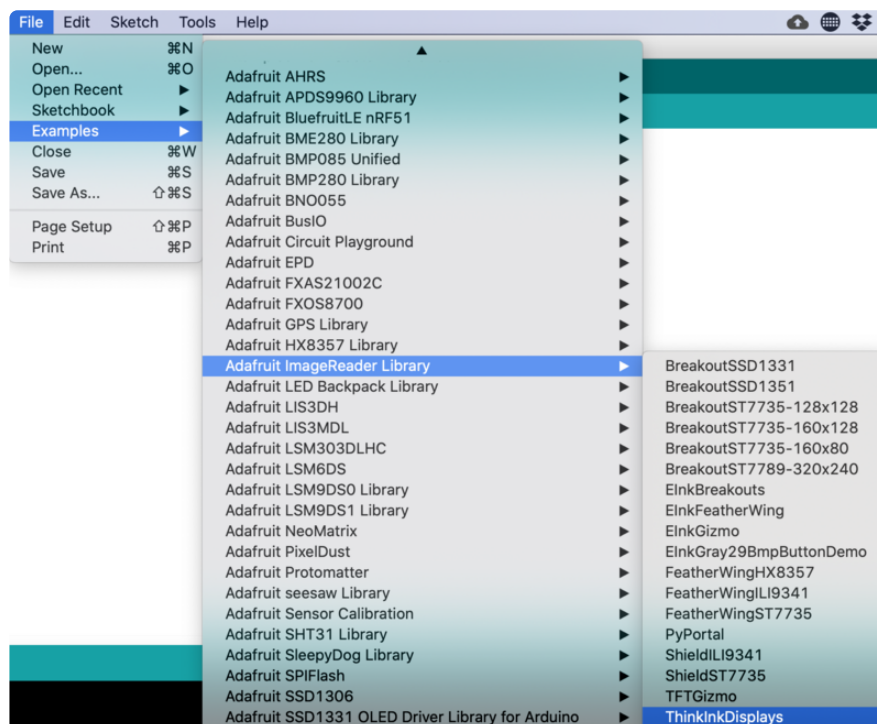
2.9" E-Ink Tri-Color

<https://adafru.it/19f3>

Download the **blinka.bmp** file and place it into the base directory of a microSD card and insert it into the microSD socket in the breakout.

Plug the microSD card into the display. You may want to try the **SD library** examples before continuing, especially one that lists all the files on the SD card

Open the **file→examples→Adafruit_ImageReader→ThinkInkDisplays** example

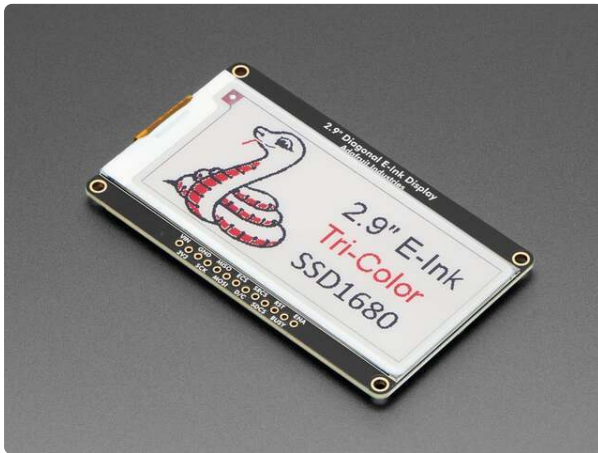


There are just a couple of things you may need to change in this file.

If you are using the FeatherWing, set **EPD_RESET** and **EPD_BUSY** to -1 otherwise it will wait indefinitely.

You'll need to change the initializer for your display. These options are heavily commented on the previous [Arduino Usage page \(https://adafru.it/1alq\)](https://adafru.it/1alq) in this guide.

Upload to your board and you should see an image of Blinky appear.

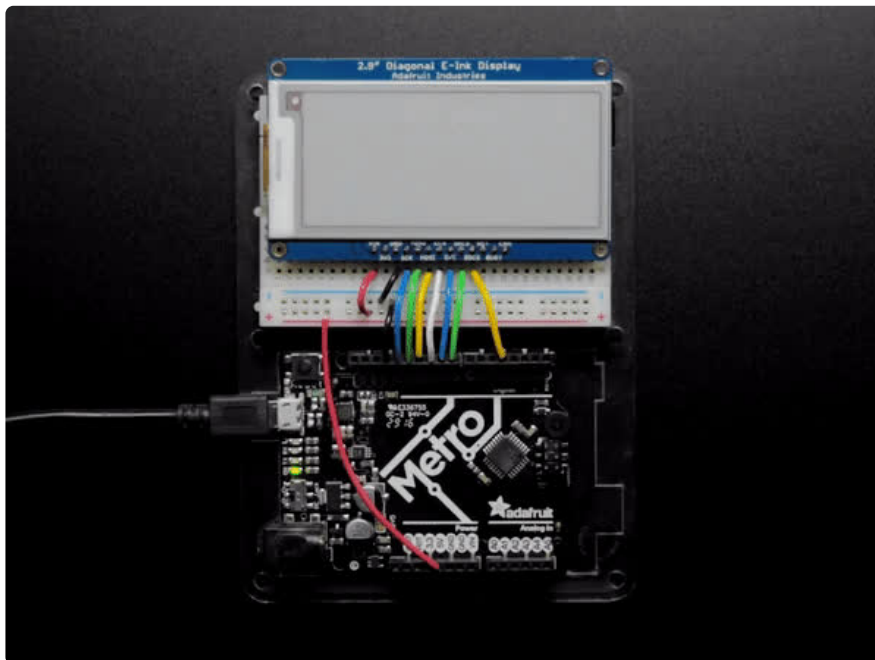


Adafruit 2.9" eInk Display Breakouts and FeatherWings

By M. LeBlanc-Williams

▮ [Arduino Usage](https://adafru.it/1alq)

<https://learn.adafruit.com/adafruit-2-9-eink-display-breakouts-and-featherwings/arduino-usage>

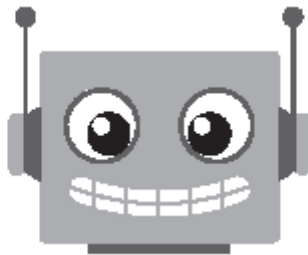


GrayScale Display Demo

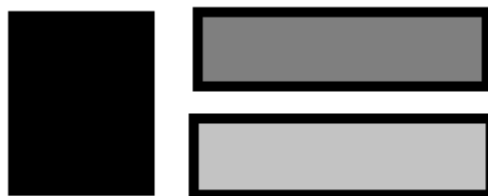
The 2.9" Grayscale display can show a max of 296x128 pixels. The grayscale demo uses three bitmaps. These bitmaps may be used for fun in the demo:



<https://adafru.it/OdE>



<https://adafru.it/OdF>

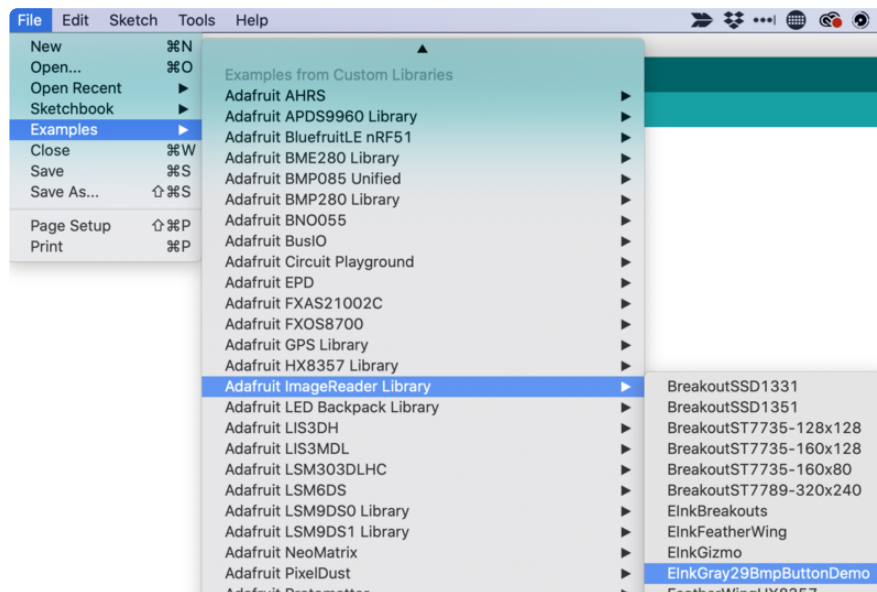


<https://adafru.it/OdG>

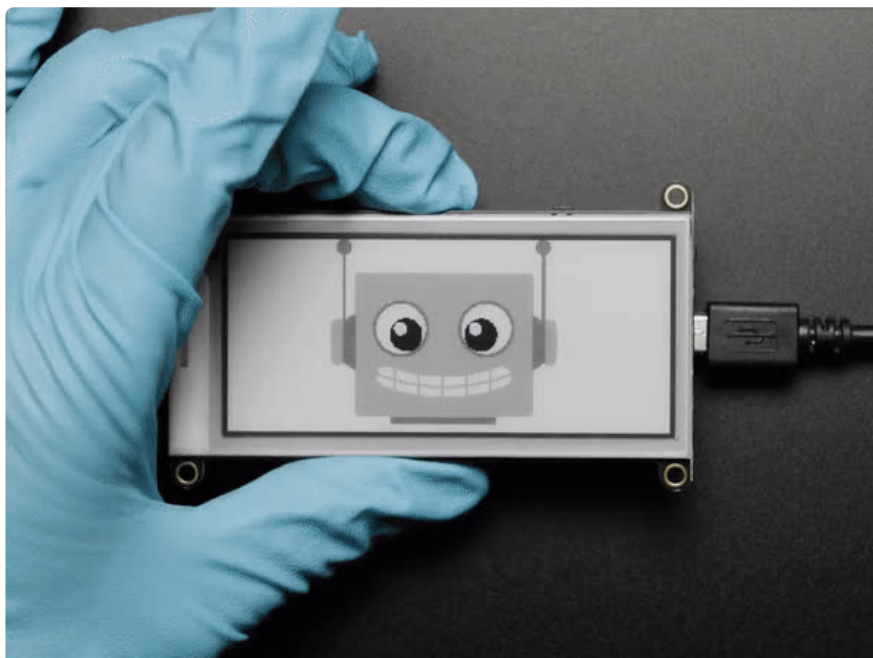
Rename the files to **panda_head.bmp**, **adabot_head.bmp**, and **29gray4.bmp** and place them into the base directory of a microSD card and insert it into the microSD socket in the breakout.

Plug the microSD card into the display. You may want to try the **SD library** examples before continuing, especially one that lists all the files on the SD card

Open the **file->examples->Adafruit_ImageReader->EInkGray29BmpButtonDemo** example



Upload to your board and press the buttons on top. You should see a different image appear for each button.



If you want to later use your own image, use an image editing tool and crop your image to no larger than 296 pixels wide and 128 pixels high. Save it as a 24-bit color

BMP file - it must be 24-bit color format to work, even if it was originally a 16-bit color image - because of the way BMPs are stored and displayed!

CircuitPython Monochrome Usage



Text Here is where the differences in the tri-color/monochrome and chipset/dimensions start mattering. Check carefully to make emphasized sure you are running the right example and creating the with a red matching library type for your display or you won't see exclamation:anything happen on the EPD (or the image may be really weird looking).

This page goes over how to use the **2.9" Monochrome eInk** displays with CircuitPython.



[2.9" Flexible 296x128 Monochrome eInk / ePaper Display](https://www.adafruit.com/product/4262)

Woah, the cyber-future is here! Flexible E-Ink has been demo'd at high-tech events for years but now you can actually get your paws on it. This display is true E-Ink / E-Paper,...

<https://www.adafruit.com/product/4262>

CircuitPython eInk displayio Library Installation

To use displayio, you will need to install the appropriate library for your display.

First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board. You will need the latest version of CircuitPython.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/zdx) (<https://adafru.it/zdx>). The introduction guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>) for both Express and non-Express boards.

You will need to copy the appropriate displayio driver from the bundle **lib** folder to a **lib** folder on your **CIRCUITPY** drive. The displayio driver contains the initialization codes specific to your display that are needed to for it to work. Since there is more than one driver, you will need to copy the correct file over. Here is a list of each of the displays and the correct driver for that display.

Text emphasized with a yellow exclamation:

To use the elnk displays with displayio, you will need to use the latest version of CircuitPython and a board that can fit **displayio**. See the Support Matrix to determine if **displayio** is available on a given board: https://circuitpython.readthedocs.io/en/latest/shared-bindings/support_matrix.html

Adafruit_CircuitPython_UC8151D

The flexible monochrome displays use the **Adafruit_CircuitPython_UC8151D** library. To easily get all the required files and libraries, you can click the **Download Project Bundle** link at the top of the example code below. Just unzip, open the folder that corresponds to the version of CircuitPython you have installed, and copy the contents to the **CIRCUITPY** drive.

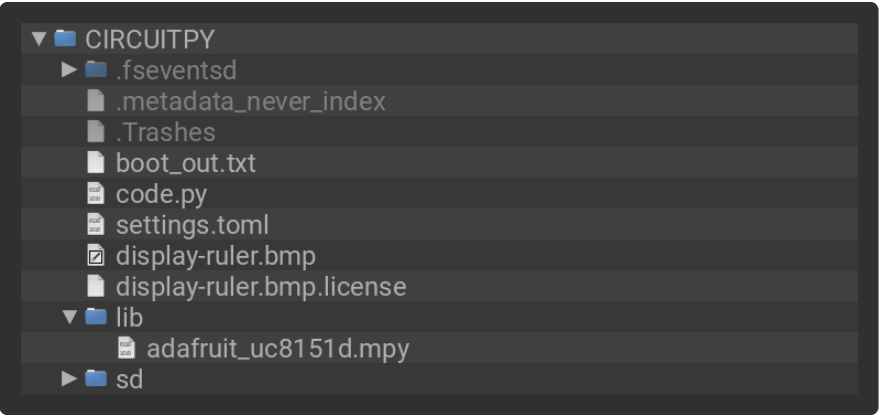


Image File

To show you how to use the elnk with displayio, you'll draw a bitmap onto it. First start by downloading **display-ruler.bmp**

<https://adafru.it/Tsa>

Copy **display-ruler.bmp** into the root directory of your **CIRCUITPY** drive.

Monochrome Display Usage

In the examples folder for your UC8151C displayio driver, there should be a test for your display which is listed here:

```
1  # SPDX-FileCopyrightText: 2017 Scott Shawcroft, written for Adafruit Industries
2  # SPDX-FileCopyrightText: Copyright (c) 2021 Melissa LeBlanc-Williams for Adafruit Industries
3  #
4  # SPDX-License-Identifier: Unlicense
5
6  """Simple test script for 2.9" 296x128 monochrome display.
7
8  Supported products:
9  * Adafruit Flexible 2.9" Monochrome
10  * https://www.adafruit.com/product/4262
11  """
12  # pylint: disable=no-member
13
14  import time
15
16  import board
17  import displayio
18  from fourwire import FourWire
19
20  import adafruit_uc8151d
21
22  displayio.release_displays()
23
24  # This pinout works on a Feather M4 and may need to be altered for other boards.
25  spi = board.SPI() # Uses SCK and MOSI
26  epd_cs = board.D9
27  epd_dc = board.D10
28  epd_reset = board.D5
29  epd_busy = None
30
31  display_bus = FourWire(spi, command=epd_dc, chip_select=epd_cs, reset=epd_reset,
32  baudrate=1000000)
33  time.sleep(1)
34
35  display = adafruit_uc8151d.UC8151D(
36      display_bus, width=296, height=128, rotation=90, busy_pin=epd_busy
37  )
38
39  g = displayio.Group()
40
41  pic = displayio.OnDiskBitmap("/display-ruler.bmp")
42  t = displayio.TileGrid(pic, pixel_shader=pic.pixel_shader)
43  g.append(t)
44
45  # Place the display group on the screen
46  display.root_group = g
47
```

```

48 # Refresh the display to have it actually show the image
49 # NOTE: Do not refresh eInk displays sooner than 180 seconds
50 display.refresh()
51 print("refreshed")
52
time.sleep(180)

```

https://github.com/adafruit/Adafruit_CircuitPython_UC8151D/blob/main/examples/uc8151d_simpletest.py

Configure and Upload

You will want to change the `epd_reset` and `epd_busy` to the correct values. If you are using the [eInk Breakout Friend](http://adafru.it/4224) (<http://adafru.it/4224>) and wired it up as shown on the Wiring page, you will want to change it to these values:

```

1 epd_reset = board.D8
2 epd_busy = board.D7

```

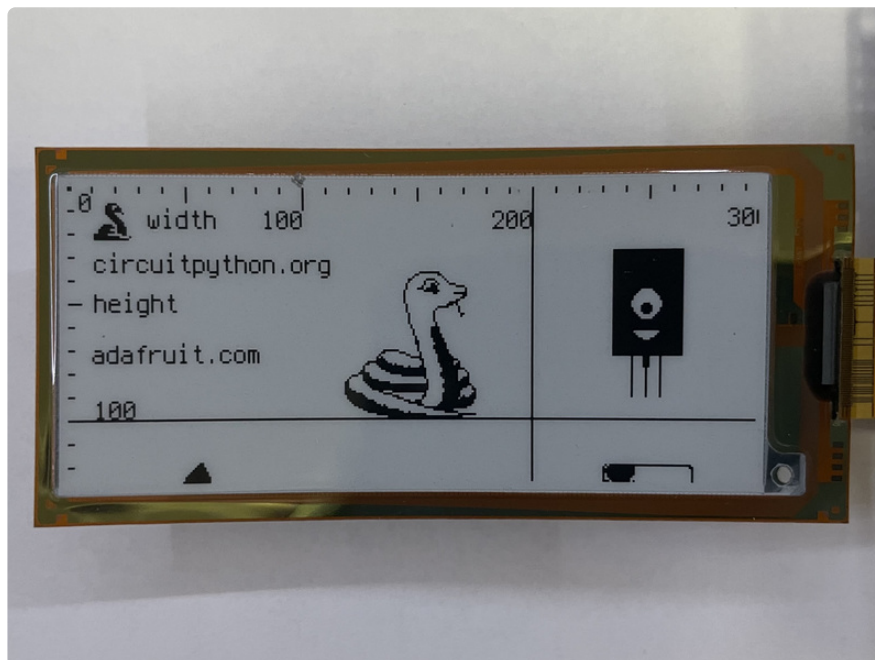
If you are using the [eInk FeatherWing Friend](http://adafru.it/4446) (<http://adafru.it/4446>), you will want to change both of these values to `None`:

```

1 epd_reset = None
2 epd_busy = None

```

Save it to your **CIRCUITPY** drive as **code.py** and it should automatically run. Your display will look something like this:



CircuitPython Grayscale Usage



Here is where the differences in the tri-color/monochrome and chipset/dimensions start mattering. Check carefully to make sure you are running the right example and creating the display with a matching library type for your display or you won't see anything happen on the EPD (or the image may be really weird looking).

This page goes over how to use the **2.9" Grayscale eInk** displays with CircuitPython.



[Adafruit 2.9" Grayscale eInk / ePaper Display FeatherWing](https://www.adafruit.com/product/4777)

Easy e-paper comes to your Feather with this breakout that's designed to make it a breeze to add a monochrome eInk display. Chances are you've seen one of those...

<https://www.adafruit.com/product/4777>

CircuitPython eInk displayio Library Installation

To use displayio, you will need to install the appropriate library for your display.

First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board. You will need the latest version of CircuitPython.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/zdx) (<https://adafru.it/zdx>). The introduction guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>) for both Express and non-Express boards.

You will need to copy the appropriate displayio driver from the bundle **lib** folder to a **lib** folder on your **CIRCUITPY** drive. The displayio driver contains the initialization codes specific to your display that are needed for it to work. Since there is more

than one driver, you will need to copy the correct file over. Here is a list of each of the displays and the correct driver for that display.

SSD1680 Updated Version

If you have the SSD1680 version of the 2.9" Grayscale FeatherWing, use this example code to get started.

```
1  # SPDX-FileCopyrightText: 2026 Mikey Sklar, written for Adafruit Industries
2  #
3  # SPDX-License-Identifier: Unlicense
4
5  """4-gray grayscale for the Adafruit 2.9" SSD1680 eInk FeatherWing (#4777), via adafruit_epd.
6
7  The framebuf path (companion to the displayio adafruit_ssd1680 driver). Reach for adafruit_epd
8  when you want external-SRAM offload or direct framebuffer access. 296x128 GDEM029T94 /
9  FPC-7519rev.b. Uses the upstream Adafruit_SSD1680_Grayscale4 class (colstart=0 hardcoded, 296
10 gate handled) — no library change; just vcom=0x24 for this panel.
11
12 FeatherWing wiring notes (differ from the #4197 EYESPI breakout):
13   CS=D9  DC=D10  SCK/MOSI on the shared bus.
14   BUSY is not connected on the Wing -> busy_pin=None (timed refresh).
15   RST is on the Feather RESET line, not a GPIO -> the panel can only be woken by a Feather
16   hardware reset, so we never deep-sleep it (power_down -> no-op). Run after a power-on/reset.
17   """
18
19 import time
20
21 import board
22 import digitalio
23
24 from adafruit_epd.epd import Adafruit_EPd
25 from adafruit_epd.ssd1680 import Adafruit_SSD1680_Grayscale4
26
27
28 class WingGrayscale4(Adafruit_SSD1680_Grayscale4):
29     # FeatherWing: RST is on the Feather RESET line, so the panel can't be GPIO-reset to wake
30     # from deep sleep -> never deep-sleep it.
31     def power_down(self):
32         pass
33
34
35 spi = board.SPI()
36 cs = digitalio.DigitalInOut(board.D9)
37 dc = digitalio.DigitalInOut(board.D10)
38 rst = digitalio.DigitalInOut(board.D11) # harmless on the Wing (RST not routed here)
39
40 display = WingGrayscale4(
41     128,
42     296,
43     spi,
44     cs_pin=cs,
45     dc_pin=dc,
46     sramcs_pin=None,
```

```

47     rst_pin=rst,
48     busy_pin=None, # BUSY not connected on the Wing
49     vcom=0x24, # FPC-7519rev.b (the class default 0x1C is for GDEY0213B74)
50 )
51 display.rotation = 3 # landscape; matches the displayio driver's 270
52 W = display.width
53 print("Init OK — drawing 4-gray info card...")
54
55 display.fill(Adafruit_EPDM.WHITE)
56 display.text("Adafruit ThinkInk", 6, 6, Adafruit_EPDM.BLACK, size=2)
57 display.text("2.9" 296x128", 6, 28, Adafruit_EPDM.BLACK, size=2)
58 display.text("4-Gray E-Ink", 6, 50, Adafruit_EPDM.DARK, size=2)
59 display.text("SSD1680 #4777", 6, 74, Adafruit_EPDM.BLACK, size=1)
60
61 # 4-level gray ramp across the bottom: black | dark | light | white
62 RAMP_TOP, RAMP_H = 100, 24
63 SEG = W // 4
64 ramp = (Adafruit_EPDM.BLACK, Adafruit_EPDM.DARK, Adafruit_EPDM.LIGHT, Adafruit_EPDM.WHITE)
65 for i, color in enumerate(ramp):
66     x = i * SEG
67     display.fill_rect(x, RAMP_TOP, SEG if i < 3 else W - x, RAMP_H, color)
68 display.rect(0, RAMP_TOP, W, RAMP_H, Adafruit_EPDM.BLACK)
69 for i in range(1, 4):
70     display.vline(i * SEG, RAMP_TOP, RAMP_H, Adafruit_EPDM.BLACK)
71
72 print("Refreshing...")
73 t0 = time.monotonic()
74 display.display()
75 print(f"Done in {time.monotonic() - t0:.1f}s")

```

https://github.com/adafruit/Adafruit_CircuitPython_EPDM/blob/main/examples/feather_epdm_grayscale_2in9_4777.py

Text
emphasized
with a
yellow
exclamation:

To use the eInk displays with displayio, you will need to use the latest version of CircuitPython and a board that can fit **displayio**. See the Support Matrix to determine if **displayio** is available on a given board: https://circuitpython.readthedocs.io/en/latest/shared-bindings/support_matrix.html

Adafruit_CircuitPython_IL0373

The GrayScale FeatherWings use the **Adafruit_CircuitPython_IL0373** library. To easily get all the required files and libraries, you can click the **Download Project Bundle** link at the top of the example code below. Just unzip, open the folder that corresponds to the version of CircuitPython you have installed, and copy the contents to the **CIRCUITPY** drive.

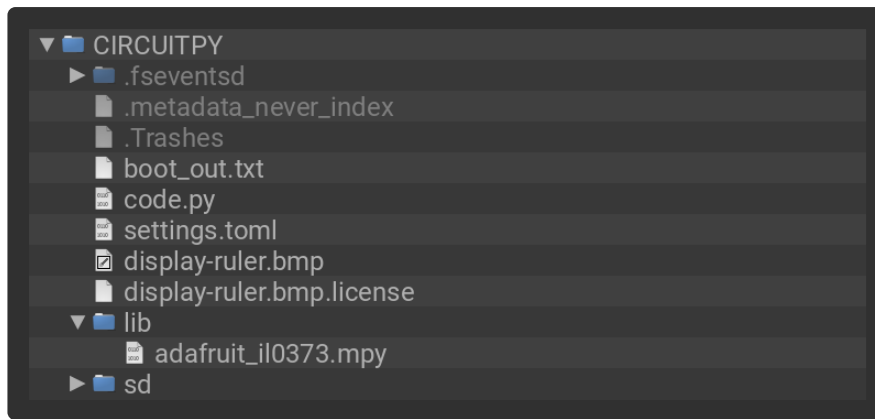


Image File

To show you how to use the `elink` with `displayio`, you'll draw a bitmap onto it. First start by downloading **display-ruler.bmp**

<https://adafru.it/Tsa>

Copy **display-ruler.bmp** into the root directory of your **CIRCUITPY** drive.

Grayscale Display Usage

In the examples folder for your IL0373 `displayio` driver, there should be a test for your display which is listed here:

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  """Simple test script for 2.9" 296x128 grayscale display.
5
6  Supported products:
7  * Adafruit 2.9" Grayscale
8  * https://www.adafruit.com/product/4777
9  """
10
11  import time
12
13  import board
14  import busio
15  import displayio
16  import fourwire
17
18  import adafruit_il0373
19
20  displayio.release_displays()
```

```

21
22 # This pinout works on a Feather M4 and may need to be altered for other boards.
23 spi = busio.SPI(board.SCK, board.MOSI) # Uses SCK and MOSI
24 epd_cs = board.D9
25 epd_dc = board.D10
26
27 display_bus = fourwire.FourWire(spi, command=epd_dc, chip_select=epd_cs, baudrate=1000000)
28 time.sleep(1)
29
30 display = adafruit_il0373.IL0373(
31     display_bus,
32     width=296,
33     height=128,
34     rotation=270,
35     black_bits_inverted=False,
36     color_bits_inverted=False,
37     grayscale=True,
38     refresh_time=1,
39 )
40
41 g = displayio.Group()
42
43 pic = displayio.OnDiskBitmap("/display-ruler.bmp")
44 t = displayio.TileGrid(pic, pixel_shader=pic.pixel_shader)
45 g.append(t)
46
47 display.root_group = g
48
49 display.refresh()
50
51 print("refreshed")
52 time.sleep(120)

```

https://github.com/adafruit/Adafruit_CircuitPython_IL0373/blob/main/examples/il0373_2.9_grayscale.py

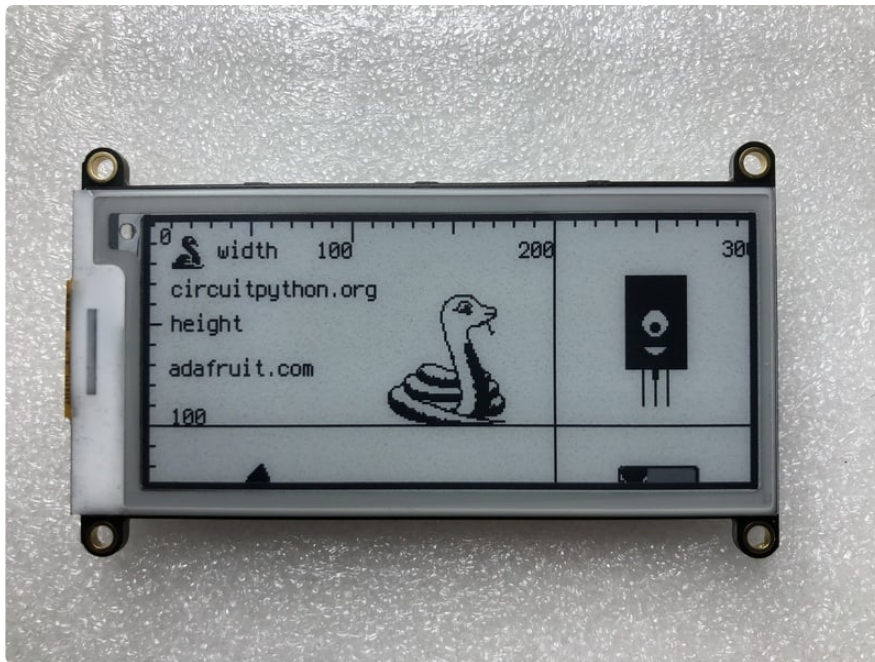
You will want to change the `epd_reset` and `epd_busy` to the correct values. If you wired it up as shown on the Wiring page, you will want to change it to these values:

```

1 epd_reset = board.D8
2 epd_busy = board.D7

```

Save it to your **CIRCUITPY** drive as **code.py** and it should automatically run. Your display will look something like this:

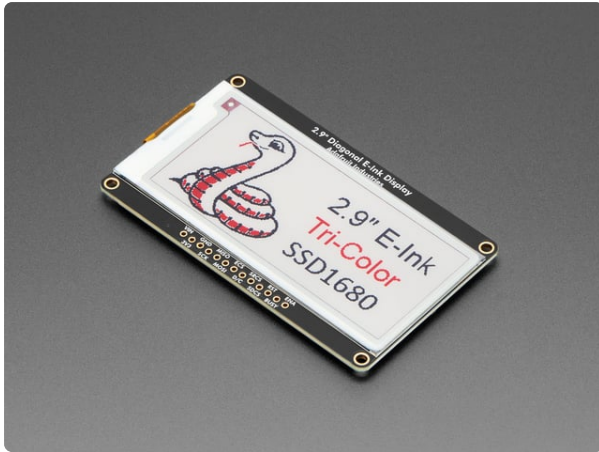


CircuitPython Tri-Color Usage

Here is where the differences in the tri-color/monochrome and chipset/dimensions start mattering. Check carefully to make emphasized sure you are running the right example and creating the with a red matching library type for your display or you won't see exclamation:anything happen on the EPD (or the image may be really weird looking).

This page goes over how to use the **2.9" Tri-Color eInk** displays with CircuitPython.

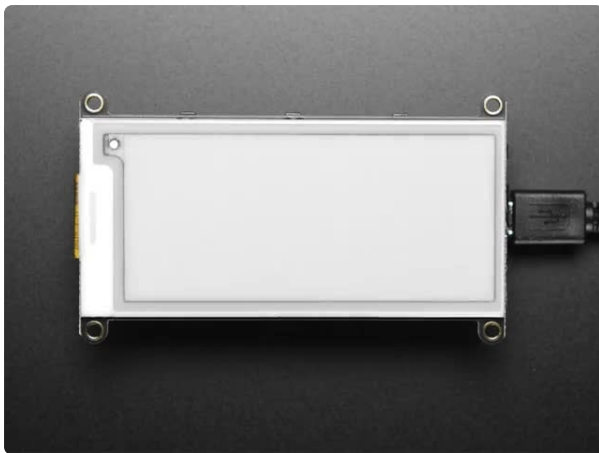
2.9" Tri-Color eInk Display and FeatherWing Revision As of emphasized June 20, 2025 - The tri-color display and FeatherWing have with a been updated (old ones discontinued) to now use a spiffy yellow modern SSD1680 chip - we have support in CircuitPython/exclamation:Python/Arduino for it!



Adafruit 2.9" Red/Black/White eInk Display Breakout - THINK INK

Easy e-paper finally comes to microcontrollers, with this breakout that's designed to make it a breeze to add a tri-color eInk display. Chances are you've seen one of those...

<https://www.adafruit.com/product/1028>



Adafruit 2.9" Tri-Color eInk / ePaper Display FeatherWing

Easy e-paper comes to your Feather with this breakout that's designed to make it a breeze to add a tri-color eInk display. Chances are you've seen one of those...

<https://www.adafruit.com/product/4778>

CircuitPython eInk displayio Library Installation

To use displayio, you will need to install the appropriate library for your display.

First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board. You will need the latest version of CircuitPython.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/zdx) (<https://adafru.it/zdx>). The introduction guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>) for both Express and non-Express boards.

You will need to copy the appropriate displayio driver from the bundle **lib** folder to a **lib** folder on your **CIRCUITPY** drive. The displayio driver contains the initialization codes specific to your display that are needed to for it to work. Since there is more than one driver, you will need to copy the correct file over. Here is a list of each of the displays and the correct driver for that display.



Text emphasized with a yellow exclamation:

To use the elnk displays with displayio, you will need to use the latest version of CircuitPython and a board that can fit **displayio**. See the Support Matrix to determine if **displayio** is available on a given board: https://circuitpython.readthedocs.io/en/latest/shared-bindings/support_matrix.html

Image File

All of the examples below use the same bitmap image. You'll need the **display-ruler.bmp** bitmap file on your CIRCUITPY drive.

<https://adafru.it/Tsa>

Copy **display-ruler.bmp** into the root directory of your **CIRCUITPY** drive.

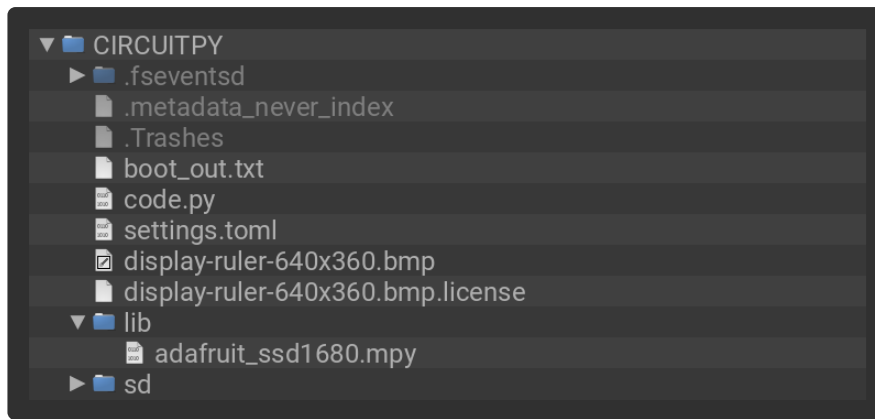
Adafruit_CircuitPython_SSD1680



Text emphasized with a yellow exclamation:

As of June 20, 2025 - The displays have been updated to use the SSD1680 chip

As of June 20, 2025, the 2.9" Tri-Color display and FeatherWing use the **Adafruit_CircuitPython_SSD1680** library. To easily get all the required files and libraries, you can click the **Download Project Bundle** link at the top of the example code below. Just unzip, open the folder that corresponds to the version of CircuitPython you have installed, and copy the contents to the **CIRCUITPY** drive.



Display Usage

```
1  # SPDX-FileCopyrightText: 2017 Scott Shawcroft, written for Adafruit Industries
2  # SPDX-FileCopyrightText: Copyright (c) 2023 Jose D. Montoya
3  #
4  # SPDX-License-Identifier: Unlicense
5
6
7  """Simple test script for Adafruit 2.9" Tri-Color eInk Display Breakout
8  Supported products:
9  * Adafruit 2.9" Tri-Color eInk Display Breakout
10  * https://www.adafruit.com/product/1028
11
12  """
13
14  import time
15
16  import board
17  import displayio
18  from fourwire import FourWire
19
20  import adafruit_ssd1680
21
22  displayio.release_displays()
23
24  # This pinout works on a Metro M4 and may need to be altered for other boards.
25  spi = board.SPI() # Uses SCK and MOSI
26  epd_cs = board.D9
27  epd_dc = board.D10
28  epd_reset = board.D5
29  epd_busy = board.D6
30
31  display_bus = FourWire(spi, command=epd_dc, chip_select=epd_cs, baudrate=1000000)
32  time.sleep(1)
33
34  display = adafruit_ssd1680.SSD1680(
35      display_bus,
36      width=296,
37      height=128,
38      highlight_color=0xFF0000,
39      rotation=270,
40  )
```

```

41
42 g = displayio.Group()
43
44
45 pic = displayio.OnDiskBitmap("/display-ruler-640x360.bmp")
46
47 t = displayio.TileGrid(pic, pixel_shader=pic.pixel_shader)
48
49 g.append(t)
50
51 display.root_group = g
52
53 display.refresh()
54
55 print("refreshed")
56
57 time.sleep(120)

```

https://github.com/adafruit/Adafruit_CircuitPython_SSD1680/blob/main/examples/ssd1680_2.9_tricolor_breakout.py

For the FeatherWing, you will want to change the `epd_reset` and `epd_busy` values to `None` :

```

1 epd_reset = None
2 epd_busy = None

```

Save it to your **CIRCUITPY** drive as **code.py** and it should automatically run. Your display will look like this:

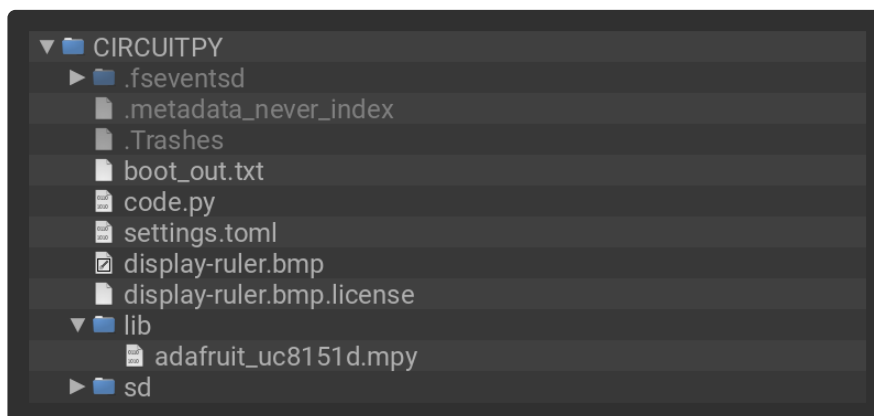


Adafruit_CircuitPython_UC8151D



Text emphasized with Between November 16, 2022 and June 20, 2025 the a yellow exclamation: 2.9" Tri-Color elnk display used the UC8151D chipset.

If you purchased the 2.9" Tri-Color elnk display between of November 16, 2022 and June 20, 2025, the display has the UC8151D chipset and uses the **Adafruit_CircuitPython_UC8151D** library. To easily get all the required files and libraries, you can click the **Download Project Bundle** link at the top of the example code below. Just unzip, open the folder that corresponds to the version of CircuitPython you have installed, and copy the contents to the **CIRCUITPY** drive.



UC8151D Display Usage

In the examples folder for your UC8151D displayio driver, there should be a test for your display which is listed here:

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  """Simple test script for Adafruit 2.9" 296x128 tri-color display
5  Supported products:
6  * Adafruit 2.9" Tri-Color Display Breakout
7  * https://www.adafruit.com/product/1028
8  """
9
10 import time
11
```

```

12 import board
13 import displayio
14 from fourwire import FourWire
15
16 import adafruit_uc8151d
17
18 # Used to ensure the display is free in CircuitPython
19 displayio.release_displays()
20
21 # Define the pins needed for display use
22 # This pinout is for a Feather M4 and may be different for other boards
23 spi = board.SPI() # Uses SCK and MOSI
24 epd_cs = board.D9
25 epd_dc = board.D10
26 epd_reset = board.D5
27 epd_busy = board.D6
28
29 # Create the displayio connection to the display pins
30 display_bus = FourWire(spi, command=epd_dc, chip_select=epd_cs, reset=epd_reset,
31 baudrate=1000000)
32 time.sleep(1) # Wait a bit
33
34 # Create the display object - the third color is red (0xff0000)
35 display = adafruit_uc8151d.UC8151D(
36     display_bus,
37     width=296,
38     height=128,
39     rotation=270,
40     busy_pin=epd_busy,
41     highlight_color=0xFF0000,
42 )
43
44 # Create a display group for our screen objects
45 g = displayio.Group()
46
47 # Display a ruler graphic from the root directory of the CIRCUITPY drive
48 pic = displayio.OnDiskBitmap("/display-ruler.bmp")
49 t = displayio.TileGrid(pic, pixel_shader=pic.pixel_shader)
50 g.append(t)
51
52 # Place the display group on the screen
53 display.root_group = g
54
55 # Refresh the display to have it actually show the image
56 # NOTE: Do not refresh elnk displays sooner than 180 seconds
57 display.refresh()
58 print("refreshed")
59
time.sleep(180)

```

https://github.com/adafruit/Adafruit_CircuitPython_UC8151D/blob/main/examples/uc8151d_2.9_color.py

You will want to change the `epd_reset` and `epd_busy` to the correct values. If you wired it up as shown on the Wiring page, you will want to change it to these values:


```
1 epd_reset = board.D8
2 epd_busy = board.D7
```

Save it to your **CIRCUITPY** drive as **code.py** and it should automatically run. Your display will look something like this:



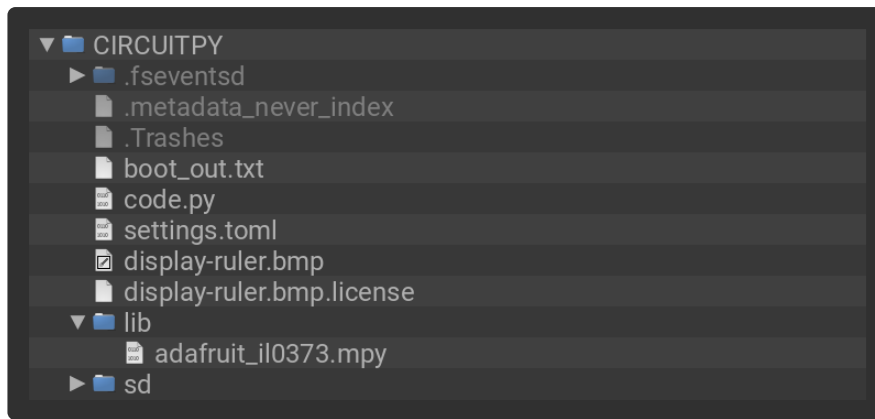
Adafruit_CircuitPython_IL0373



Text emphasized with a yellow exclamation:

Before November 16, 2022, the 2.9" Tri-Color elnk display used the IL0373 chipset. Before June 24, 2025, the 2.9" Tri-Color elnk FeatherWing used the IL0373 chipset.

If you purchased the 2.9" Tri-Color elnk display before November 16, 2022, or the FeatherWing before June 20, 2025, they used the IL0373 chipset with the **Adafruit_CircuitPython_IL0373** library. To easily get all the required files and libraries, you can click the **Download Project Bundle** link at the top of the example code below. Just unzip, open the folder that corresponds to the version of CircuitPython you have installed, and copy the contents to the **CIRCUITPY** drive.



IL0373 Display Usage

In the examples folder for your IL0373 displayio driver, there should be a test for your display which is listed here:

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  """Simple test script for Adafruit 2.9" 296x128 tri-color display
5  Supported products:
6  * Adafruit 2.9" Tri-Color Display Breakout
7  * https://www.adafruit.com/product/1028
8  """
9
10 import time
11
12 import board
13 import displayio
14 import fourwire
15
16 import adafruit_il0373
17
18 # Used to ensure the display is free in CircuitPython
19 displayio.release_displays()
20
21 # Define the pins needed for display use
22 # This pinout is for a Feather M4 and may be different for other boards
23 spi = board.SPI() # Uses SCK and MOSI
24 epd_cs = board.D9
25 epd_dc = board.D10
26 epd_reset = board.D5
27 epd_busy = board.D6
28
29 # Create the displayio connection to the display pins
30 display_bus = fourwire.FourWire(
31     spi, command=epd_dc, chip_select=epd_cs, reset=epd_reset, baudrate=1000000
32 )
33 time.sleep(1) # Wait a bit
34
35 # Create the display object - the third color is red (0xff0000)
36 display = adafruit_il0373.IL0373(
```

```

37     display_bus,
38     width=296,
39     height=128,
40     rotation=270,
41     busy_pin=epd_busy,
42     highlight_color=0xFF0000,
43 )
44
45 # Create a display group for our screen objects
46 g = displayio.Group()
47
48 # Display a ruler graphic from the root directory of the CIRCUITPY drive
49 pic = displayio.OnDiskBitmap("/display-ruler.bmp")
50 t = displayio.TileGrid(pic, pixel_shader=pic.pixel_shader)
51 g.append(t)
52
53 display.root_group = g
54
55 display.refresh()
56
57 print("refreshed")
58 time.sleep(180)

```

https://github.com/adafruit/Adafruit_CircuitPython_IL0373/blob/main/examples/il0373_2.9_color.py

For the FeatherWing, you will want to change the `epd_reset` and `epd_busy` values to `None`:

```

1 epd_reset = None
2 epd_busy = None

```

Save it to your **CIRCUITPY** drive as **code.py** and it should automatically run. Your display will look like this:



Python Setup

It's easy to use elnk breakouts with Python and the [Adafruit CircuitPython EPD](https://adafru.it/BTd) (<https://adafru.it/BTd>) library. This library allows you to easily write Python code to control the display.

Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafru.it/BSN) (<https://adafru.it/BSN>).



Text emphasized with a blue exclamation:

Note this is not a kernel driver that will let you have the console appear on the elnk. However, this is handy when you want to use the elnk display purely from 'user Python' code!



Text emphasized with a yellow exclamation:

You can only use this technique with Linux/computer devices that have hardware SPI support, and not all single board computers have an SPI device, so check before continuing

Setup Virtual Environment

If you are installing on the Bookworm version of Raspberry Pi OS or later, you will need to install your python modules in a virtual environment. You can find more information in the [Python Virtual Environment Usage on Raspberry Pi \(https://adafru.it/19a5\)](https://adafru.it/19a5) guide. To Install and activate the virtual environment, use the following commands:

```
sudo apt install python3-venv
python -m venv env --system-site-packages
```

To activate the virtual environment:

```
source env/bin/activate
```

Install Adafruit_Blinka

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling SPI on your platform and verifying you are running Python 3. [Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready \(https://adafru.it/BSN\)](https://adafru.it/BSN)!

Python Installation of EPD Library

Once that's done, from your command line run the following command:

```
pip3 install adafruit-circuitpython-epd
```

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

If that complains about pip3 not being installed, then run this first to install it:

```
sudo apt-get install python3-pip
```

Download font5x8.bin

This library also requires a font file to run! You can download it below. Before continuing, make sure the folder you are running scripts from contains the **font5x8.bin** file.

<https://adafru.it/Xbr>

Alternatively, you can use **wget** to directly download the file to your pi:

```
wget https://github.com/adafruit/Adafruit_CircuitPython_framebuf/raw/main/examples/font5x8.bin
```

DejaVu TTF Font

Raspberry Pi usually comes with the DejaVu font already installed, but in case it didn't, you can run the following to install it:

```
sudo apt-get install fonts-dejavu
```

This package was previously calls **ttf-dejavu**, so if you are running an older version of Raspberry Pi OS, it may be called that.

Pillow Library

Some of the examples also use PIL, the Python Imaging Library, to allow graphics and using text with custom fonts. There are several system libraries that PIL relies on, so installing via a package manager is the easiest way to bring in everything:

```
sudo apt-get install python3-pil
```

Chip Enable Lines

Follow [these instructions](https://adafru.it/19fg) (<https://adafru.it/19fg>) for dealing with SPI chip enable line issues.

That's it. You should be ready to go.

Python Usage



Text emphasized with a yellow exclamation: Note this is not a kernel driver that will let you have the console appear on the elnk. However, this is handy when you want to use the elnk display purely from 'user Python' code!



Text emphasized with a yellow exclamation: You can only use this technique with Linux/computer devices that have hardware SPI support, and not all single board computers have an SPI device, so check before continuing

To demonstrate the usage of the display, we'll initialize it and draw some lines from the Python REPL.

Run the following code to import the necessary modules and set up the pin assignments. You set the SRAM CS pin to **None** because the Raspberry Pi has lots of RAM, so you don't really need it.

```
1 import digitalio
2 import busio
3 import board
4 from adafruit_epd.epd import Adafruit_EPd
5
6 spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
7 ecs = digitalio.DigitalInOut(board.CE0)
8 dc = digitalio.DigitalInOut(board.D22)
9 rst = digitalio.DigitalInOut(board.D27)
10 busy = digitalio.DigitalInOut(board.D17)
11 srcs = None
```



Text emphasized with a blue exclamation: Depending on the exact E-Ink display you're using, the driver and object initialization will differ a bit because Python must be told what chip driver to use and the size of the display.

Run the following code to initialize the 2.9" 296x128 Flexible Monochrome display:



2.9" Flexible 296x128 Monochrome eInk / ePaper Display

Woah, the cyber-future is here! Flexible E-Ink has been demo'd at high-tech events for years but now you can actually get your paws on it. This display is true E-Ink / E-Paper,...

<https://www.adafruit.com/product/4262>

```
1 from adafruit_epd.uc8151d import Adafruit_UC8151D
2 display = Adafruit_UC8151D(128, 296, spi, cs_pin=ecs, dc_pin=dc, sramcs_pin=srcs, rst_pin=rst,
3   busy_pin=busy)
4 display.set_black_buffer(1, True)
   display.set_color_buffer(1, True)
```

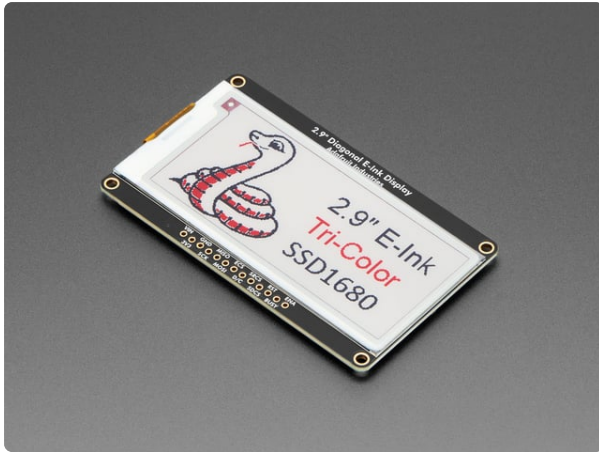
If you have one of the older flexible displays, you can use the following code to initialize it:

```
1 from adafruit_epd.il0373 import Adafruit_IL0373
2 display = Adafruit_IL0373(128, 296, spi, cs_pin=ecs, dc_pin=dc, sramcs_pin=srcs, rst_pin=rst,
3   busy_pin=busy)
4 display.set_black_buffer(1, False)
   display.set_color_buffer(1, False)
```

For the 2.9" 296x128 Tri-Color display:



Text 2.9" Tri-Color eInk Display and FeatherWing Revision As of emphasized June 20, 2025 - The tri-color display and FeatherWing have with a been updated (old ones discontinued) to now use a spiffy yellow modern SSD1680 chip - we have support in CircuitPython/ exclamation:Python/Arduino for it!



Adafruit 2.9" Red/Black/White eInk Display Breakout - THINK INK

Easy e-paper finally comes to microcontrollers, with this breakout that's designed to make it a breeze to add a tri-color eInk display. Chances are you've seen one of those...

<https://www.adafruit.com/product/1028>

IL0373 Chipset:

```
1 from adafruit_epd.il0373 import Adafruit_IL0373
2 display = Adafruit_IL0373(128, 296, spi, cs_pin=ecs, dc_pin=dc, sramcs_pin=srcs, rst_pin=rst,
   busy_pin=busy)
```

SSD1680 Chipset:

```
1 display = Adafruit_SSD1680(128, 296, spi, cs_pin=ecs, dc_pin=dc, sramcs_pin=srcs, rst_pin=rst,
   busy_pin=busy)
```

Monochrome Example

Now to clear the screen buffer and draw some shapes. Once done drawing, the code must tell the screen to update using the `display()` method.

```
1 display.rotation = 3
2 display.fill(Adafruit_EPD.WHITE)
3
4 display.fill_rect(20, 20, 50, 60, Adafruit_EPD.BLACK)
5 display.hline(80, 30, 60, Adafruit_EPD.BLACK)
6 display.vline(80, 30, 60, Adafruit_EPD.BLACK)
7
8 display.display()
```

Tri-Color Example

The Tri-Color example is almost the same as the monochrome example, except another color is added in. Once done drawing, the code needs to tell the screen to update using the `display()` method.

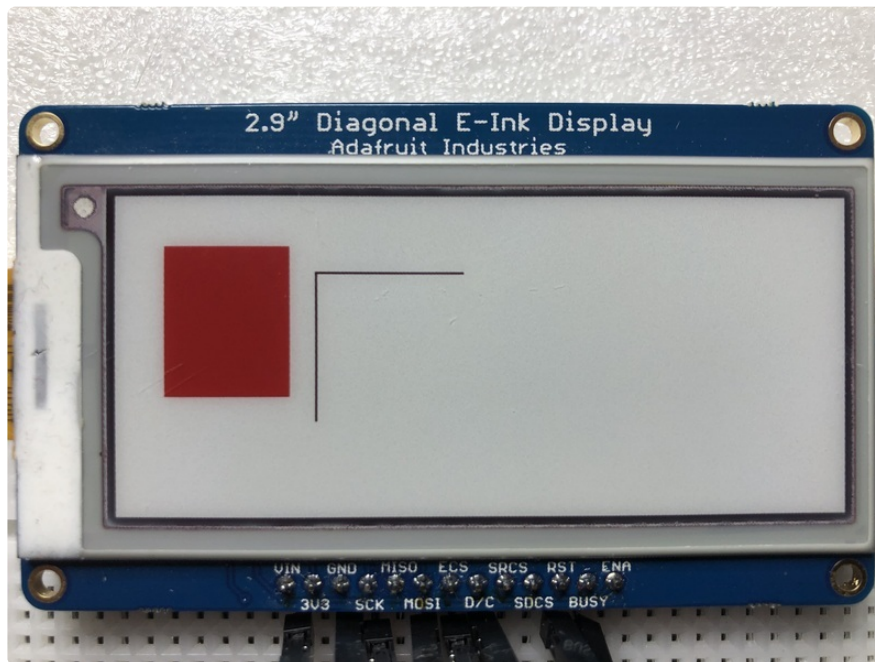
```
1 display.rotation = 3
2 display.fill(Adafruit_EPD.WHITE)
3
```

```

4  display.fill_rect(20, 20, 50, 60, Adafruit_EPD.RED)
5  display.hline(80, 30, 60, Adafruit_EPD.BLACK)
6  display.vline(80, 30, 60, Adafruit_EPD.BLACK)
7
8  display.display()

```

Your display will look something like this:



That's all there is to drawing simple shapes with eInk displays and CircuitPython!

Full Example Code

Here is the full example code.



Text emphasized
with a blue
exclamation:

To run the code sample below, you will need to change
the pins the same way as you did in the Tri-color Bitmap
Example.

```

1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  import board

```

```

5 import busio
6 import digitalio
7
8 from adafruit_epd.ek79686 import Adafruit_EK79686
9 from adafruit_epd.epd import Adafruit_EPd
10 from adafruit_epd.il0373 import Adafruit_IL0373, Adafruit_IL0373_213_Flex_Mono
11 from adafruit_epd.il0398 import Adafruit_IL0398
12 from adafruit_epd.il91874 import Adafruit_IL91874
13 from adafruit_epd.jd79661 import Adafruit_JD79661
14 from adafruit_epd.ssd1608 import Adafruit_SSD1608
15 from adafruit_epd.ssd1675 import Adafruit_SSD1675
16 from adafruit_epd.ssd1680 import Adafruit_SSD1680
17 from adafruit_epd.ssd1680b import Adafruit_SSD1680B
18 from adafruit_epd.ssd1681 import Adafruit_SSD1681
19 from adafruit_epd.ssd1683 import Adafruit_SSD1683
20 from adafruit_epd.uc8151d import Adafruit_UC8151D
21 from adafruit_epd.uc8179 import Adafruit_UC8179
22
23 # create the spi device and pins we will need
24 spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
25 ecs = digitalio.DigitalInOut(board.D12)
26 dc = digitalio.DigitalInOut(board.D11)
27 srcs = digitalio.DigitalInOut(board.D10) # can be None to use internal memory
28 rst = digitalio.DigitalInOut(board.D9) # can be None to not use this pin
29 busy = digitalio.DigitalInOut(board.D5) # can be None to not use this pin
30
31 # give them all to our drivers
32 print("Creating display")
33 # display = Adafruit_JD79661(122, 150, # 2.13" Quad-color display
34 # display = Adafruit_SSD1608(200, 200, # 1.54" HD mono display
35 # display = Adafruit_SSD1675(122, 250, # 2.13" HD mono display
36 # display = Adafruit_SSD1680(122, 250, # 2.13" HD Tri-color display
37 # display = Adafruit_SSD1680B(122, 250 # 2.13" HD (Tri-color or mono) with GDEY0213B74
38 # display = Adafruit_SSD1681(200, 200, # 1.54" HD Tri-color display
39 # display = Adafruit_SSD1681(200, 200, # 1.54" HD Tri-color display
40 # display = Adafruit_IL91874(176, 264, # 2.7" Tri-color display
41 # display = Adafruit_EK79686(176, 264, # 2.7" Tri-color display
42 # display = Adafruit_IL0373(152, 152, # 1.54" Tri-color display
43 # display = Adafruit_UC8151D(128, 296, # 2.9" mono flexible display
44 # display = Adafruit_UC8179(648, 480, # 5.83" mono 648x480 display
45 # display = Adafruit_UC8179(800, 480, # 7.5" mono 800x480 display
46 # display = Adafruit_IL0373(128, 296, # 2.9" Tri-color display IL0373
47 # display = Adafruit_IL0373_213_Flex_Mono(104, 212, # 2.13" mono flex display
48 # display = Adafruit_SSD1680(128, 296, # 2.9" Tri-color display SSD1680
49 # display = Adafruit_SSD1683(400, 300, # 4.2" 300x400 Tri-Color display
50 # display = Adafruit_IL0398(400, 300, # 4.2" Tri-color display
51 display = Adafruit_IL0373(
52     104,
53     212, # 2.13" Tri-color display
54     spi,
55     cs_pin=ecs,
56     dc_pin=dc,
57     sramcs_pin=srcs,
58     rst_pin=rst,
59     busy_pin=busy,
60 )

```

```

61  """ display = Adafruit_UC8179(800, 480,      # 7.5" tricolor 800x480 display
62      spi,
63      cs_pin=ecs,
64      dc_pin=dc,
65      sramcs_pin=srcs,
66      rst_pin=rst,
67      busy_pin=busy,
68      tri_color = True
69  )"""
70
71  # IF YOU HAVE A 2.13" FLEXIBLE DISPLAY OR!
72  # UC8179 5.83" or 7.5" monochrome displays
73  # uncomment these lines!
74  # display.set_black_buffer(1, False)
75  # display.set_color_buffer(1, False)
76
77  # IF YOU HAVE A 2.9" FLEXIBLE DISPLAY uncomment these lines!
78  # display.set_black_buffer(1, True)
79  # display.set_color_buffer(1, True)
80
81  display.rotation = 1
82  if type(display) == Adafruit_JD79661:
83      WHITE = Adafruit_JD79661.WHITE
84      BLACK = Adafruit_JD79661.BLACK
85      RED = Adafruit_JD79661.RED
86      YELLOW = Adafruit_JD79661.YELLOW
87  else:
88      WHITE = Adafruit_EPD.WHITE
89      BLACK = Adafruit_EPD.BLACK
90      RED = Adafruit_EPD.RED
91
92  # clear the buffer
93  print("Clear buffer")
94  display.fill(WHITE)
95  display.pixel(10, 100, BLACK)
96
97  print("Draw Rectangles")
98  display.fill_rect(5, 5, 10, 10, RED)
99  display.rect(0, 0, 20, 30, BLACK)
100
101  print("Draw lines")
102  if type(display) == Adafruit_JD79661:
103      display.line(0, 0, display.width - 1, display.height - 1, YELLOW)
104      display.line(0, display.height - 1, display.width - 1, 0, YELLOW)
105  else:
106      display.line(0, 0, display.width - 1, display.height - 1, BLACK)
107      display.line(0, display.height - 1, display.width - 1, 0, RED)
108
109  print("Draw text")
110  display.text("hello world", 25, 10, BLACK)
111  display.display()

```

https://github.com/adafruit/Adafruit_CircuitPython_EPD/blob/main/examples/epd_simpletest.py

Bitmap Example

Here's a complete example of how to display a bitmap image on your display. **Note that any .bmp image you want to display must be exactly the size of your display.** The image below will be used on the 2.9" display. Click the button below to download the image and save it as **blinka.bmp** on your **Raspberry Pi**. The code uses a Tri-Color bitmap, but it should still work on a monochrome display.

<https://adafru.it/19f8>

Save the following code to your Raspberry Pi as **epd_bitmap.py**.

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  import board
5  import busio
6  import digitalio
7
8  from adafruit_epd.ek79686 import Adafruit_EK79686
9  from adafruit_epd.epd import Adafruit_EPd
10 from adafruit_epd.il0373 import Adafruit_IL0373
11 from adafruit_epd.il0398 import Adafruit_IL0398
12 from adafruit_epd.il91874 import Adafruit_IL91874
13 from adafruit_epd.ssd1608 import Adafruit_SSD1608
14 from adafruit_epd.ssd1675 import Adafruit_SSD1675
15 from adafruit_epd.ssd1680 import Adafruit_SSD1680
16 from adafruit_epd.ssd1680_legacy import Adafruit_SSD1680_Legacy
17 from adafruit_epd.ssd1680b import Adafruit_SSD1680B
18 from adafruit_epd.ssd1681 import Adafruit_SSD1681
19 from adafruit_epd.ssd1683 import Adafruit_SSD1683
20 from adafruit_epd.uc8151d import Adafruit_UC8151D
21 from adafruit_epd.uc8179 import Adafruit_UC8179
22
23 # create the spi device and pins we will need
24 spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
25 ecs = digitalio.DigitalInOut(board.D10)
26 dc = digitalio.DigitalInOut(board.D9)
27 srcs = digitalio.DigitalInOut(board.D7) # can be None to use internal memory
28 rst = digitalio.DigitalInOut(board.D11) # can be None to not use this pin
29 busy = digitalio.DigitalInOut(board.D12) # can be None to not use this pin
30
31 # give them all to our driver
32 print("Creating display")
33 # display = Adafruit_SSD1608(200, 200, # 1.54" HD mono display
34 # display = Adafruit_SSD1675(122, 250, # 2.13" HD mono display
35 # display = Adafruit_SSD1680(122, 250, # 2.13" HD Tri-color display
36 # display = Adafruit_SSD1680B(122, 250 # Newer 2.13" HD (Tri-color or mono) with GDEY0213B74
```

```

37 # display = Adafruit_SSD1680_Legacy(122, 250, # pre-2024 SSD1680 Bonnet
38 # display = Adafruit_SSD1681(200, 200,      # 1.54" HD Tri-color display
39 # display = Adafruit_IL91874(176, 264,      # 2.7" Tri-color display
40 # display = Adafruit_EK79686(176, 264,      # 2.7" Tri-color display
41 # display = Adafruit_IL0373(152, 152,      # 1.54" Tri-color display
42 # display = Adafruit_UC8151D(128, 296,      # 2.9" mono flexible display
43 # display = Adafruit_UC8179(648, 480,      # 5.83" mono 648x480 display
44 # display = Adafruit_UC8179(800, 480,      # 7.5" mono 800x480 display
45 # display = Adafruit_IL0373(128, 296,      # 2.9" Tri-color display IL0373
46 # display = Adafruit_SSD1680(128, 296,      # 2.9" Tri-color display SSD1680
47 # display = Adafruit_SSD1683(400, 300,      # 4.2" 300x400 Tri-Color display
48 # display = Adafruit_IL0398(400, 300,      # 4.2" Tri-color display
49 display = Adafruit_IL0373(
50     104,
51     212, # 2.13" Tri-color display
52     spi,
53     cs_pin=ecs,
54     dc_pin=dc,
55     sramcs_pin=srcs,
56     rst_pin=rst,
57     busy_pin=busy,
58 )
59 """ display = Adafruit_UC8179(800, 480,      # 7.5" tricolor 800x480 display
60     spi,
61     cs_pin=ecs,
62     dc_pin=dc,
63     sramcs_pin=srcs,
64     rst_pin=rst,
65     busy_pin=busy,
66     tri_color = True
67 )"""
68
69 # IF YOU HAVE A 2.13" FLEXIBLE DISPLAY OR!
70 # UC8179 5.83" or 7.5" displays
71 # uncomment these lines!
72 # display.set_black_buffer(1, False)
73 # display.set_color_buffer(1, False)
74
75 # IF YOU HAVE A 2.9" FLEXIBLE DISPLAY uncomment these lines!
76 # display.set_black_buffer(1, True)
77 # display.set_color_buffer(1, True)
78
79 display.rotation = 0
80
81 FILENAME = "blinka.bmp"
82
83
84 def read_le(s):
85     # as of this writing, int.from_bytes does not have LE support, DIY!
86     result = 0
87     shift = 0
88     for byte in bytearray(s):
89         result += byte << shift
90         shift += 8
91     return result
92

```

```

93
94 class BMPError(Exception):
95     pass
96
97
98 def display_bitmap(epd, filename):
99     try:
100         f = open(filename, "rb")
101     except OSError:
102         print("Couldn't open file")
103         return
104
105     print("File opened")
106     try:
107         if f.read(2) != b"BM": # check signature
108             raise BMPError("Not BitMap file")
109
110         bmpFileSize = read_le(f.read(4))
111         f.read(4) # Read & ignore creator bytes
112
113         bmpImageoffset = read_le(f.read(4)) # Start of image data
114         headerSize = read_le(f.read(4))
115         bmpWidth = read_le(f.read(4))
116         bmpHeight = read_le(f.read(4))
117         flip = True
118
119         print(f"Size: {bmpFileSize}\nImage offset: {bmpImageoffset}\nHeader size: {headerSize}")
120         print(f"Width: {bmpWidth}\nHeight: {bmpHeight}")
121
122         if read_le(f.read(2)) != 1:
123             raise BMPError("Not singleplane")
124         bmpDepth = read_le(f.read(2)) # bits per pixel
125         print(f"Bit depth: {bmpDepth}")
126         if bmpDepth != 24:
127             raise BMPError("Not 24-bit")
128         if read_le(f.read(2)) != 0:
129             raise BMPError("Compressed file")
130
131         print("Image OK! Drawing...")
132
133         rowSize = (bmpWidth * 3 + 3) & ~3 # 32-bit line boundary
134
135         for row in range(bmpHeight): # For each scanline...
136             if flip: # Bitmap is stored bottom-to-top order (normal BMP)
137                 pos = bmpImageoffset + (bmpHeight - 1 - row) * rowSize
138             else: # Bitmap is stored top-to-bottom
139                 pos = bmpImageoffset + row * rowSize
140
141             # print ("seek to %d" % pos)
142             f.seek(pos)
143             rowdata = f.read(3 * bmpWidth)
144             for col in range(bmpWidth):
145                 b, g, r = rowdata[3 * col : 3 * col + 3] # BMP files store RGB in BGR
146                 if r < 0x80 and g < 0x80 and b < 0x80:
147                     epd.pixel(col, row, Adafruit_EPD.BLACK)
148                 elif r >= 0x80 and g >= 0x80 and b >= 0x80:

```

```

149         pass # epd.pixel(row, col, Adafruit_EPD.WHITE)
150     elif r >= 0x80:
151         epd.pixel(col, row, Adafruit_EPD.RED)
152 except OSError:
153     print("Couldn't read file")
154 except BMPEError as e:
155     print("Failed to parse BMP: " + e.args[0])
156 finally:
157     f.close()
158 print("Finished drawing")
159
160
161 # clear the buffer
162 display.fill(Adafruit_EPD.WHITE)
163 display_bitmap(display, FILENAME)
164 display.display()

```

https://github.com/adafruit/Adafruit_CircuitPython_EPD/blob/main/examples/epd_bitmap.py

Before running it, you will need to change a few pin definitions. Find the section of code that looks like this:

```

1  ecs = digitalio.DigitalInOut(board.D10)
2  dc = digitalio.DigitalInOut(board.D9)
3  srcs = digitalio.DigitalInOut(board.D7) # can be None to use internal memory
4  rst = digitalio.DigitalInOut(board.D11) # can be None to not use this pin
5  busy = digitalio.DigitalInOut(board.D12) # can be None to not use this pin

```

Change the pins to the following to match the wiring on the Raspberry Pi:

```

1  ecs = digitalio.DigitalInOut(board.CE0)
2  dc = digitalio.DigitalInOut(board.D22)
3  srcs = None
4  rst = digitalio.DigitalInOut(board.D27)
5  busy = digitalio.DigitalInOut(board.D17)

```

Next, find the section that looks like this:

```

1  # display = Adafruit_SSD1608(200, 200, # 1.54" HD mono display
2  # display = Adafruit_SSD1675(122, 250, # 2.13" HD mono display
3  # display = Adafruit_SSD1680(122, 250, # 2.13" HD Tri-color display
4  # display = Adafruit_SSD1681(200, 200, # 1.54" HD Tri-color display
5  # display = Adafruit_IL91874(176, 264, # 2.7" Tri-color display
6  # display = Adafruit_IL0373(152, 152, # 1.54" Tri-color display
7  # display = Adafruit_UC815D(128, 296, # 2.9" mono flexible display
8  # display = Adafruit_IL0373(128, 296, # 2.9" Tri-color display
9  # display = Adafruit_IL0398(400, 300, # 4.2" Tri-color display
10 display = Adafruit_IL0373(
11     104,
12     212, # 2.13" Tri-color display
13     spi,
14     cs_pin=ecs,

```

```

15     dc_pin=dc,
16     sramcs_pin=srscs,
17     rst_pin=rst,
18     busy_pin=busy,
19 )

```

Comment out these lines:

```

1     display = Adafruit_IL0373(
2         104,
3         212, # 2.13" Tri-color display

```

and **uncomment** the line that corresponds with your display.

Next to tell the display the rotation setting desired. This can be a value between **0-3**. For the 2.13" displays, a value of **3** seems to work well.

```

1     display.rotation = 3

```

Now go to the command prompt on your Raspberry Pi and run the script with the following command:

```
python3 epd_bitmap.py
```

After a few seconds, your display should show an image like this:

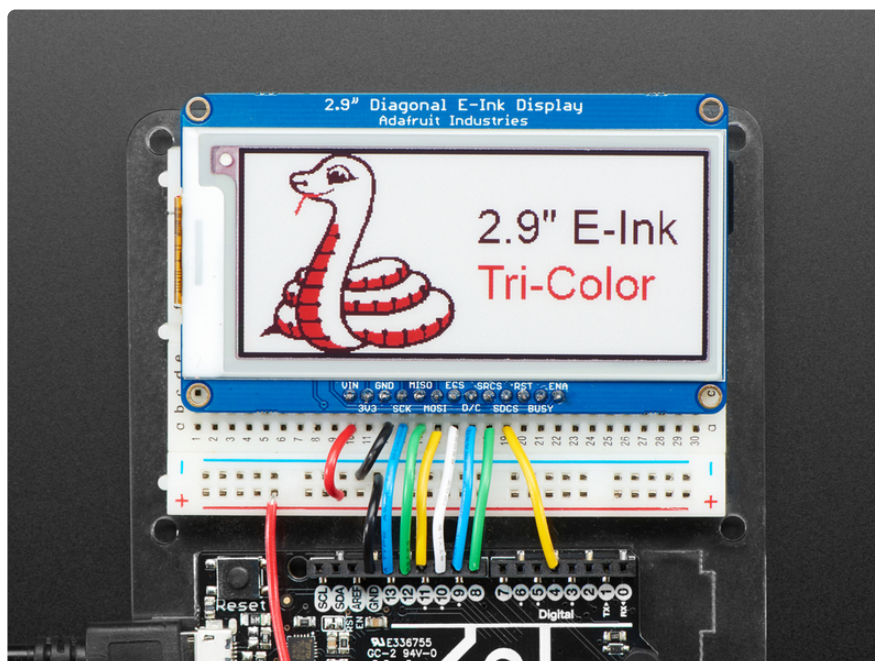
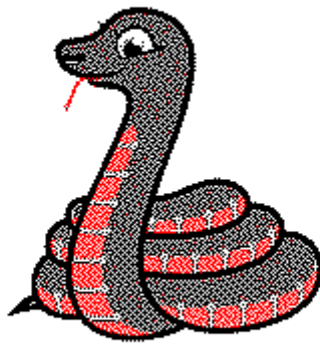


Image Drawing with Pillow

This example will use Pillow to resize and crop the image automatically and draw it on the the ePaper Display. Pillow is really powerful and with it you can open and render additional file formats such as PNG or JPG. Let's start with downloading a PNG of blinka that has been adjusted down to 3 colors so it prints nicely on an ePaper Display. This uses a PNG format file because it is a lossless format and won't introduce unexpected colors on the display.



Make sure you save it as **blinka.png** and place it in the same folder as your script. Here's the code to load onto the Raspberry Pi. Go ahead and copy it onto your Raspberry Pi and save it as **epd_pillow_image.py**.

```
1  # SPDX-FileCopyrightText: 2019 Melissa LeBlanc-Williams for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  """
5  Image resizing and drawing using the Pillow Library. For the image, check out the
6  associated Adafruit Learn guide at:
7  https://learn.adafruit.com/adafruit-eink-display-breakouts/python-code
8
9  """
10
11  import board
12  import busio
13  import digitalio
14  from PIL import Image
15
16  from adafruit_epd.ek79686 import Adafruit_EK79686
17  from adafruit_epd.il0373 import Adafruit_IL0373
18  from adafruit_epd.il0398 import Adafruit_IL0398
```



```

19 from adafruit_epd.il91874 import Adafruit_IL91874
20 from adafruit_epd.jd79661 import Adafruit_JD79661
21 from adafruit_epd.ssd1608 import Adafruit_SSD1608
22 from adafruit_epd.ssd1675 import Adafruit_SSD1675
23 from adafruit_epd.ssd1680 import Adafruit_SSD1680
24 from adafruit_epd.ssd1680_legacy import Adafruit_SSD1680_Legacy
25 from adafruit_epd.ssd1680b import Adafruit_SSD1680B
26 from adafruit_epd.ssd1681 import Adafruit_SSD1681
27 from adafruit_epd.ssd1683 import Adafruit_SSD1683
28 from adafruit_epd.uc8151d import Adafruit_UC8151D
29 from adafruit_epd.uc8179 import Adafruit_UC8179
30
31 # create the spi device and pins we will need
32 spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
33 ecs = digitalio.DigitalInOut(board.CE0)
34 dc = digitalio.DigitalInOut(board.D22)
35 srcs = None
36 rst = digitalio.DigitalInOut(board.D27)
37 busy = digitalio.DigitalInOut(board.D17)
38
39 # give them all to our driver
40 # display = Adafruit_JD79661(122, 150,      # 2.13" Quad-color display
41 # display = Adafruit_SSD1608(200, 200,      # 1.54" HD mono display
42 # display = Adafruit_SSD1675(122, 250,      # 2.13" HD mono display
43 # display = Adafruit_SSD1680(122, 250,      # 2.13" HD Tri-color or mono display
44 # display = Adafruit_SSD1680B(122, 250,      # Newer 2.13" HD (Tri-color or mono) with GDEY0213B74
45 # display = Adafruit_SSD1680_Legacy(122, 250, # pre-2024 SSD1680 Bonnet
46 # display = Adafruit_SSD1681(200, 200,      # 1.54" HD Tri-color display
47 # display = Adafruit_IL91874(176, 264,      # 2.7" Tri-color display
48 # display = Adafruit_EK79686(176, 264,      # 2.7" Tri-color display
49 # display = Adafruit_IL0373(152, 152,      # 1.54" Tri-color display
50 # display = Adafruit_UC8151D(128, 296,      # 2.9" mono flexible display
51 # display = Adafruit_UC8179(648, 480,      # 5.83" mono 648x480 display
52 # display = Adafruit_UC8179(800, 480,      # 7.5" mono 800x480 display
53 # display = Adafruit_IL0373(128, 296,      # 2.9" Tri-color display IL0373
54 # display = Adafruit_SSD1680(128, 296,      # 2.9" Tri-color display SSD1680
55 # display = Adafruit_SSD1683(400, 300,      # 4.2" 300x400 Tri-Color display
56 # display = Adafruit_IL0398(400, 300,      # 4.2" Tri-color display
57 display = Adafruit_IL0373(
58     104,
59     212, # 2.13" Tri-color display
60     spi,
61     cs_pin=ecs,
62     dc_pin=dc,
63     sramcs_pin=srcs,
64     rst_pin=rst,
65     busy_pin=busy,
66 )
67 """ display = Adafruit_UC8179(800, 480,      # 7.5" tricolor 800x480 display
68     spi,
69     cs_pin=ecs,
70     dc_pin=dc,
71     sramcs_pin=srcs,
72     rst_pin=rst,
73     busy_pin=busy,
74     tri_color = True

```

```

75 )"""
76
77 # IF YOU HAVE A 2.13" FLEXIBLE DISPLAY OR!
78 # UC8179 5.83" or 7.5" displays
79 # uncomment these lines!
80 # display.set_black_buffer(1, False)
81 # display.set_color_buffer(1, False)
82
83 # IF YOU HAVE A 2.9" FLEXIBLE DISPLAY uncomment these lines!
84 # display.set_black_buffer(1, True)
85 # display.set_color_buffer(1, True)
86
87 display.rotation = 1
88
89 image = Image.open("blinka.png")
90
91 # Scale the image to the smaller screen dimension
92 image_ratio = image.width / image.height
93 screen_ratio = display.width / display.height
94 if screen_ratio < image_ratio:
95     scaled_width = image.width * display.height // image.height
96     scaled_height = display.height
97 else:
98     scaled_width = display.width
99     scaled_height = image.height * display.width // image.width
100 image = image.resize((scaled_width, scaled_height), Image.BICUBIC)
101
102 # Crop and center the image
103 x = scaled_width // 2 - display.width // 2
104 y = scaled_height // 2 - display.height // 2
105 image = image.crop((x, y, x + display.width, y + display.height)).convert("RGB")
106
107 # Convert to Monochrome and Add dithering
108 # image = image.convert("1").convert("L")
109
110 if type(display) == Adafruit_JD79661:
111     # Create a palette with the 4 colors: Black, White, Red, Yellow
112     # The palette needs 768 values (256 colors × 3 channels)
113     palette = []
114
115     # We'll map the 256 palette indices to our 4 colors
116     # 0-63: Black, 64-127: Red, 128-191: Yellow, 192-255: White
117     for i in range(256):
118         if i < 64:
119             palette.extend([0, 0, 0]) # Black
120         elif i < 128:
121             palette.extend([255, 0, 0]) # Red
122         elif i < 192:
123             palette.extend([255, 255, 0]) # Yellow
124         else:
125             palette.extend([255, 255, 255]) # White
126
127     # Create a palette image
128     palette_img = Image.new("P", (1, 1))
129     palette_img.putpalette(palette)
130

```

```

131     # Optional: Enhance colors before dithering for better results
132     # from PIL import ImageEnhance
133     # enhancer = ImageEnhance.Color(image)
134     # image = enhancer.enhance(1.5) # Increase color saturation
135
136     # Quantize the image using Floyd-Steinberg dithering
137     image = image.quantize(palette=palette_img, dither=Image.FLOYDSTEINBERG)
138
139     # Convert back to RGB for the display driver
140     image = image.convert("RGB")
141
142     # Display image.
143     display.image(image)
144     display.display()

```

https://github.com/adafruit/Adafruit_CircuitPython_EPD/blob/main/examples/epd_pillow_image.py

The code starts with library imports including a couple of Pillow modules and the ePaper display drivers.

```

1  import digitalio
2  import busio
3  import board
4  from PIL import Image, ImageDraw
5  from adafruit_epd.il0373 import Adafruit_IL0373
6  from adafruit_epd.il91874 import Adafruit_IL91874
7  from adafruit_epd.il0398 import Adafruit_IL0398
8  from adafruit_epd.ssd1608 import Adafruit_SSD1608
9  from adafruit_epd.ssd1675 import Adafruit_SSD1675
10 from adafruit_epd.ssd1680 import Adafruit_SSD1680
11 from adafruit_epd.ssd1681 import Adafruit_SSD1681
12 from adafruit_epd.uc8151d import Adafruit_UC8151D

```

That is followed by initializing the SPI bus and defining a few pins. The choices allow you to use the same code with the EPD bonnets, if you chose to do so.

```

1  spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
2  ecs = digitalio.DigitalInOut(board.CE0)
3  dc = digitalio.DigitalInOut(board.D22)
4  srcs = None
5  rst = digitalio.DigitalInOut(board.D27)
6  busy = digitalio.DigitalInOut(board.D17)

```

These examples work on as many displays as possible with very few changes. Go ahead and comment out the following lines:

```

display = Adafruit_IL0373(
    104,
    212, # 2.13" Tri-color display

```

and uncomment the line appropriate for your display.

```
1  #display = Adafruit_SSD1608(200, 200,    # 1.54" HD mono display
2  #display = Adafruit_SSD1675(122, 250,    # 2.13" HD mono display
3  #display = Adafruit_SSD1680(122, 250,    # 2.13" HD Tri-color display
4  #display = Adafruit_SSD1681(200, 200,    # 1.54" HD Tri-color display
5  #display = Adafruit_IL91874(176, 264,    # 2.7" Tri-color display
6  #display = Adafruit_IL0373(152, 152,    # 1.54" Tri-color display
7  #display = Adafruit_IL0373(128, 296,    # 2.9" Tri-color display
8  #display = Adafruit_IL0398(400, 300,    # 4.2" Tri-color display
9  display = Adafruit_IL0373(
10     104,
11     212, # 2.13" Tri-color display
12     spi,
13     cs_pin=ecs,
14     dc_pin=dc,
15     sramcs_pin=srscs,
16     rst_pin=rst,
17     busy_pin=busy
18 )
```

Next change the rotation setting to **3**.

```
1  display.rotation = 3
```

Next is to open the Blinka image, which is named **blinka.png**, and it is assumed the file is in the same directory that you are running the script from. Feel free to change it if it doesn't match your configuration.

```
1  image = Image.open("blinka.png")
```

Here's where it starts to get interesting. It is desirable to scale the image so that it matches either the width or height of the display, depending on which is smaller, so that there may be some of the image to chop off when it is cropped. Start by calculating the width to height ratio of both the display and the image. If the height is the closer of the dimensions, you want to match the image height to the display height and let it be a bit wider than the display. Otherwise, you want to do the opposite.

Once you've figured out how to scale it, pass in the new dimensions and using a **Bicubic** rescaling method, the code reassigns the newly rescaled image back to **image**. Pillow has quite a few different methods to choose from, but Bicubic does a great job and is reasonably fast.

Nearest actually gives a little better result with the Tri-color elnks, but loses detail with displaying a color image on the monochrome display, so this code uses the best balance.

```
1 image_ratio = image.width / image.height
2 screen_ratio = display.width / display.height
3 if screen_ratio < image_ratio:
4     scaled_width = image.width * display.height // image.height
5     scaled_height = display.height
6 else:
7     scaled_width = display.width
8     scaled_height = image.height * display.width // image.width
9 image = image.resize((scaled_width, scaled_height), Image.BICUBIC)
```

Next to figure the starting x and y points of the image to begin cropping so that the image ends up centered. Do that by using a standard centering function, which is basically requesting the difference of the center of the display and the center of the image. Just like with scaling, replace the `image` variable with the newly cropped image.

```
1 x = scaled_width // 2 - display.width // 2
2 y = scaled_height // 2 - display.height // 2
3 image = image.crop((x, y, x + display.width, y + display.height)).convert("RGB")
```

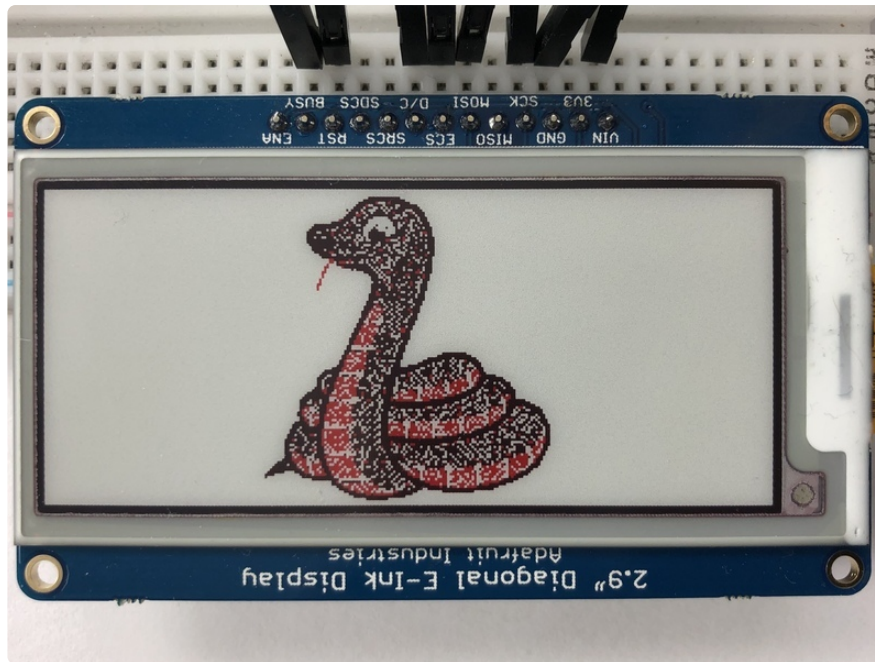
Finally, take the `image`, draw it to the frame buffer and `display` it. At this point, the image should have the exact same dimensions at the display and fill it completely.

```
1 display.image(image)
2 display.display()
```

Now go to the command prompt on your Raspberry Pi and run the script with the following command:

```
python3 epd_pillow_image.py
```

After a few seconds, your display should show this image:



Drawing Shapes and Text with Pillow

The next example takes a look at drawing shapes and text. This is very similar to the displayio example, but it uses Pillow instead. Go ahead and copy it onto your Raspberry Pi and save it as **epd_pillow_demo.py**. Here's the code for that.

```

1  # SPDX-FileCopyrightText: 2019 Melissa LeBlanc-Williams for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  """
5  ePaper Display Shapes and Text demo using the Pillow Library.
6
7  """
8
9  import board
10 import busio
11 import digitalio
12 from PIL import Image, ImageDraw, ImageFont
13
14 from adafruit_epd.ek79686 import Adafruit_EK79686
15 from adafruit_epd.il0373 import Adafruit_IL0373
16 from adafruit_epd.il0398 import Adafruit_IL0398
17 from adafruit_epd.il91874 import Adafruit_IL91874
18 from adafruit_epd.jd79661 import Adafruit_JD79661
19 from adafruit_epd.ssd1608 import Adafruit_SSD1608
20 from adafruit_epd.ssd1675 import Adafruit_SSD1675
21 from adafruit_epd.ssd1680 import Adafruit_SSD1680
22 from adafruit_epd.ssd1680_legacy import Adafruit_SSD1680_Legacy
23 from adafruit_epd.ssd1680b import Adafruit_SSD1680B
24 from adafruit_epd.ssd1681 import Adafruit_SSD1681
25 from adafruit_epd.ssd1683 import Adafruit_SSD1683
26 from adafruit_epd.uc8151d import Adafruit_UC8151D
27 from adafruit_epd.uc8179 import Adafruit_UC8179

```



```

28
29 # First define some color constants
30 WHITE = (0xFF, 0xFF, 0xFF)
31 BLACK = (0x00, 0x00, 0x00)
32 RED = (0xFF, 0x00, 0x00)
33
34 # Next define some constants to allow easy resizing of shapes and colors
35 BORDER = 20
36 FONTSIZE = 24
37 BACKGROUND_COLOR = BLACK
38 FOREGROUND_COLOR = WHITE
39 TEXT_COLOR = RED
40
41 # create the spi device and pins we will need
42 spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
43 ecs = digitalio.DigitalInOut(board.CE0)
44 dc = digitalio.DigitalInOut(board.D22)
45 srcs = None
46 rst = digitalio.DigitalInOut(board.D27)
47 busy = digitalio.DigitalInOut(board.D17)
48
49 # give them all to our driver
50 # display = Adafruit_JD79661(122, 150, # 2.13" Quad-color display
51 # display = Adafruit_SSD1608(200, 200, # 1.54" HD mono display
52 # display = Adafruit_SSD1675(122, 250, # 2.13" HD mono display
53 # display = Adafruit_SSD1680(122, 250, # 2.13" HD Tri-color or mono display
54 # display = Adafruit_SSD1680B(122, 250, # Newer 2.13" HD (Tri-color or mono) with GDEY0213B74
55 # display = Adafruit_SSD1680_Legacy(122, 250, # pre-2024 SSD1680 Bonnet
56 # display = Adafruit_SSD1681(200, 200, # 1.54" HD Tri-color display
57 # display = Adafruit_IL91874(176, 264, # 2.7" Tri-color display
58 # display = Adafruit_EK79686(176, 264, # 2.7" Tri-color display
59 # display = Adafruit_IL0373(152, 152, # 1.54" Tri-color display
60 # display = Adafruit_UC8151D(128, 296, # 2.9" mono flexible display
61 # display = Adafruit_UC8179(648, 480, # 5.83" mono 648x480 display
62 # display = Adafruit_UC8179(800, 480, # 7.5" mono 800x480 display
63 # display = Adafruit_IL0373(128, 296, # 2.9" Tri-color display IL0373
64 # display = Adafruit_SSD1680(128, 296, # 2.9" Tri-color display SSD1680
65 # display = Adafruit_SSD1683(400, 300, # 4.2" 300x400 Tri-Color display
66 # display = Adafruit_IL0398(400, 300, # 4.2" Tri-color display
67 display = Adafruit_IL0373(
68     104,
69     212, # 2.13" Tri-color display
70     spi,
71     cs_pin=ecs,
72     dc_pin=dc,
73     sramcs_pin=srcs,
74     rst_pin=rst,
75     busy_pin=busy,
76 )
77 "" display = Adafruit_UC8179(800, 480, # 7.5" tricolor 800x480 display
78     spi,
79     cs_pin=ecs,
80     dc_pin=dc,
81     sramcs_pin=srcs,
82     rst_pin=rst,
83     busy_pin=busy,

```

```

84     tri_color = True
85 )"""
86
87 # IF YOU HAVE A 2.13" FLEXIBLE DISPLAY OR!
88 # UC8179 5.83" or 7.5" displays
89 # uncomment these lines!
90 # display.set_black_buffer(1, False)
91 # display.set_color_buffer(1, False)
92
93 # IF YOU HAVE A 2.9" FLEXIBLE DISPLAY uncomment these lines!
94 # display.set_black_buffer(1, True)
95 # display.set_color_buffer(1, True)
96
97 display.rotation = 1
98
99 image = Image.new("RGB", (display.width, display.height))
100
101 # Get drawing object to draw on image.
102 draw = ImageDraw.Draw(image)
103
104 # Draw a filled box as the background
105 draw.rectangle((0, 0, display.width - 1, display.height - 1), fill=BACKGROUND_COLOR)
106
107 # Draw a smaller inner foreground rectangle
108 draw.rectangle(
109     (BORDER, BORDER, display.width - BORDER - 1, display.height - BORDER - 1),
110     fill=FOREGROUND_COLOR,
111 )
112
113 # Load a TTF Font
114 font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", FONTSIZE)
115
116 # Draw Some Text
117 text = "Hello World!"
118 (font_width, font_height) = font.getsize(text)
119 draw.text(
120     (display.width // 2 - font_width // 2, display.height // 2 - font_height // 2),
121     text,
122     font=font,
123     fill=TEXT_COLOR,
124 )
125
126 # Display image.
127 display.image(image)
128 display.display()

```

https://github.com/adafruit/Adafruit_CircuitPython_EPD/blob/main/examples/epd_pillow_demo.py

Just like in the last example, use the imports, but this time include the `ImageDraw` and `ImageFont` Pillow modules to allow text rendering.

```

1 import digitalio
2 import busio
3 import board
4 from PIL import Image, ImageDraw, ImageFont

```

```

5  from adafruit_epd.il0373 import Adafruit_IL0373
6  from adafruit_epd.il91874 import Adafruit_IL91874
7  from adafruit_epd.il0398 import Adafruit_IL0398
8  from adafruit_epd.ssd1608 import Adafruit_SSD1608
9  from adafruit_epd.ssd1675 import Adafruit_SSD1675
10 from adafruit_epd.ssd1680 import Adafruit_SSD1680
11 from adafruit_epd.ssd1681 import Adafruit_SSD1681
12 from adafruit_epd.uc8151d import Adafruit_UC8151D

```

Next to define some colors that can be used with Pillow.

```

1  WHITE = (0xFF, 0xFF, 0xFF)
2  BLACK = (0x00, 0x00, 0x00)
3  RED = (0xFF, 0x00, 0x00)

```

After that, create some parameters that are easy to change. If you had a smaller display for instance, you could reduce the **FONTSIZE** and **BORDER** parameters. The **BORDER** will be the size in pixels of the green border between the edge of the display and the inner purple rectangle. The **FONTSIZE** will be the size of the font in points so that it can adjust easily for different displays. You could play around with the colors as well. One thing to note is that on monochrome displays, **RED** will show up as **BLACK**.

For the 2.9" display, a **BORDER** value of **20** and a **FONTSIZE** value of **24** looks good.

```

1  BORDER = 10
2  FONTSIZE = 20
3  BACKGROUND_COLOR = BLACK
4  FOREGROUND_COLOR = WHITE
5  TEXT_COLOR = RED

```

After that, the initializer and rotation sections are exactly the same as in the previous example. **Go ahead and adjust your initializer as explained in the previous example.** After that, create an **image** with the dimensions and use that to create a **draw** object. The **draw** object will have all of the drawing functions.

```

1  image = Image.new('RGB', (display.width, display.height))
2
3  draw = ImageDraw.Draw(image)

```

Next clear whatever is on the screen by drawing a rectangle using the **BACKGROUND_COLOR** that takes up the full screen.

```

1  draw.rectangle((0, 0, display.width, display.height), fill=BACKGROUND_COLOR)

```

Next to draw an inner rectangle using the `BACKGROUND_COLOR`. Use the `BORDER` parameter to calculate the size and position to draw the rectangle.

```
1 draw.rectangle((BORDER, BORDER, display.width - BORDER - 1, display.height - BORDER - 1),
2               fill=BACKGROUND_COLOR)
```

Next to load a TTF font. The `DejaVuSans.ttf` font should come preloaded on your Pi in the location in the code. This will also make use of the `FONTSIZE` parameter discussed earlier.

```
1 font = ImageFont.truetype('/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf', FONTSIZE)
```

Now to draw the text Hello World onto the center of the display. You may recognize the centering calculation was the same one used to center crop the image in the previous example. In this example though, the font size values are obtained using the `getsize()` function of the font object.

```
1 text = "Hello World!"
2 (font_width, font_height) = font.getsize(text)
3 draw.text((display.width//2 - font_width//2, display.height//2 - font_height//2),
4           text, font=font, fill=TEXT_COLOR)
```

Finally, just like before, display the image.

```
1 display.image(image)
2 display.display()
```

Now go to the command prompt on your Raspberry Pi and run the script with the following command:

```
python3 epd_pillow_demo.py
```

After a few seconds, your display should show this image:



WipperSnapper Wiring & Test

What is WipperSnapper

WipperSnapper is a firmware designed to turn any WiFi-capable board into an Internet-of-Things device without programming a single line of code. WipperSnapper connects to [Adafruit IO \(https://adafru.it/fsU\)](https://adafru.it/fsU), a web platform designed ([by Adafruit! \(https://adafru.it/Bo5\)](https://adafru.it/Bo5)) to display, respond, and interact with your project's data.

Simply load the WipperSnapper firmware onto your board, add credentials, and plug it into power. Your board will automatically register itself with your Adafruit IO account.

From there, you can add components to your board such as buttons, switches, potentiometers, sensors, and more! Components are dynamically added to hardware, so you can immediately start interacting, logging, and streaming the data your projects produce without writing code.

If you've never used WipperSnapper, click below to read through the quick start guide before continuing.

<https://adafru.it/Vfd>

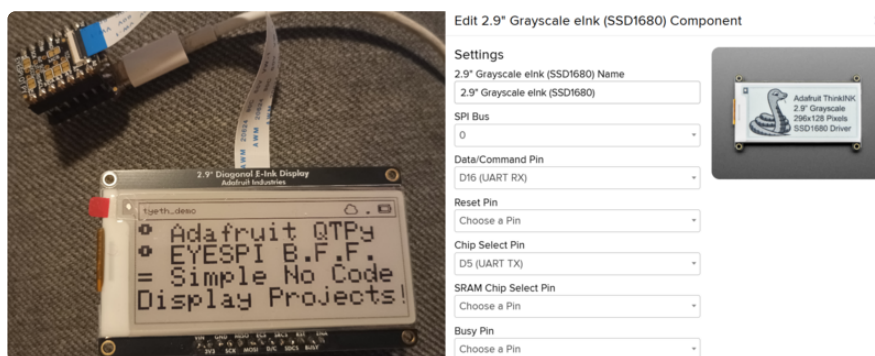
Wiring

EYESPI Connector

If you have one of the newer breakouts with an EYESPI connector on the back, you can use that with an 18-pin ribbon cable to connect it either directly to an adapter board such as the [EYESPI BFF for QT Py or Xiao](http://adafru.it/5772) (<http://adafru.it/5772>) or by using the [EYESPI Breakout](http://adafru.it/5613) (<http://adafru.it/5613>) and wiring that to your microcontroller.

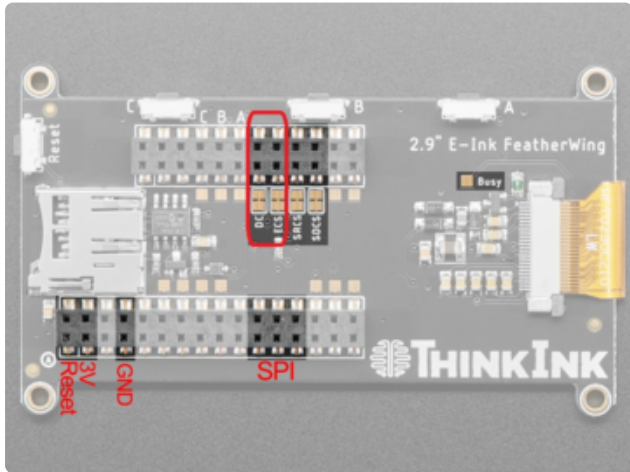
See the [EYESPI pages](https://adafru.it/1a1u) (<https://adafru.it/1a1u>) for more information, then read the Breakout Wiring information below, and adjust that to your setup when it comes to configuring the pins in the New Component dialog.

If using the QTPy BFF, then it's as simple as selecting the **RX** pin for CS (chip select), and **TX** pin for D/C.



FeatherWing Connection

FeatherWing usage is easy, simply plug your Feather board into the FeatherWing board.



To match up the pin numbers for your feather, visit the Pinout page on your board's Product Learn Guide.

You can quickly find it via the Docs link on the Wippersnapper devices page after setting up your board.

The E-Ink Featherwing uses the same pin layout as shown in the breakout wiring below, the two required pins on the shorter side (ECS+DC) are 4 unconnected pins away from the non-USB end, and the other side has a fixed layout for SPI (MOSI/ MISO/SCK) on Feathers.

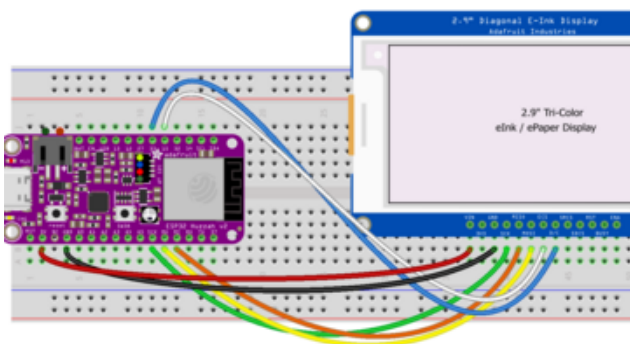
If you wish to use the 3 A/B/C buttons, then confirm the pins for those on your board (marked C, B, A to the left of D/C on the featherwing image).

Breakout Wiring

If you need help with soldering, then see the [Assembly page \(https://adafru.it/1ayb\)](https://adafru.it/1ayb), and this guide on [how to solder header pins \(https://adafru.it/RdJ\)](https://adafru.it/RdJ).

First, wire up the E-Ink display to your board as described below. Alternatively you may be able to use an EYESPI connector cable with a suitable socket (like on the [EYE SPI BFF \(http://adafru.it/5772\)](http://adafru.it/5772) for QT Py & XIAO boards), or directly board-to-board if using the featherwing version. Later identify the pins by checking board+display pinout learn guide pages.

Here is an example of the 2.13" E-Ink wired to an [Adafruit ESP32 Feather V2 \(http://adafru.it/5400\)](http://adafru.it/5400) on a breadboard [with male Jumper wires \(2.54mm Dupont connectors\) \(https://adafru.it/ECb\)](https://adafru.it/ECb).



VIN connects to the Feather **3V** pin
GND connects to Feather ground
SCK connects to SPI clock. On the Feather ESP32v2 that's **SCK**.
MISO connects to SPI MISO. On the Feather ESP32v2 that's **MI**
MOSI connects to SPI MOSI. On the Feather ESP32v2 that's **MO**
ECS connects to the E-Ink Chip Select pin. Use **Digital 15***, the **white** wire.
 (*or you can change this to any pin)
D/C connects to SPI data/command select pin. Use **Digital 33***, the **blue** wire.
 The **SRCS** (SRAM CS pin for display), **Busy** pin, and **Reset** pin are optional in this setup, but can provide improved performance and stability.

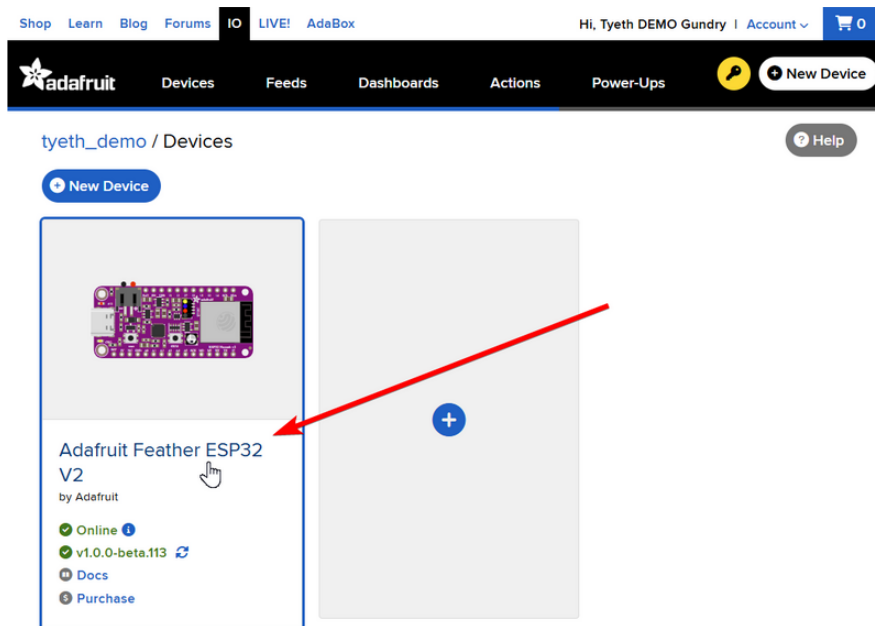
Usage

Connect your board to Adafruit IO Wippersnapper and [navigate to the WipperSnapper board list \(https://adafru.it/TAu\)](https://adafru.it/TAu).

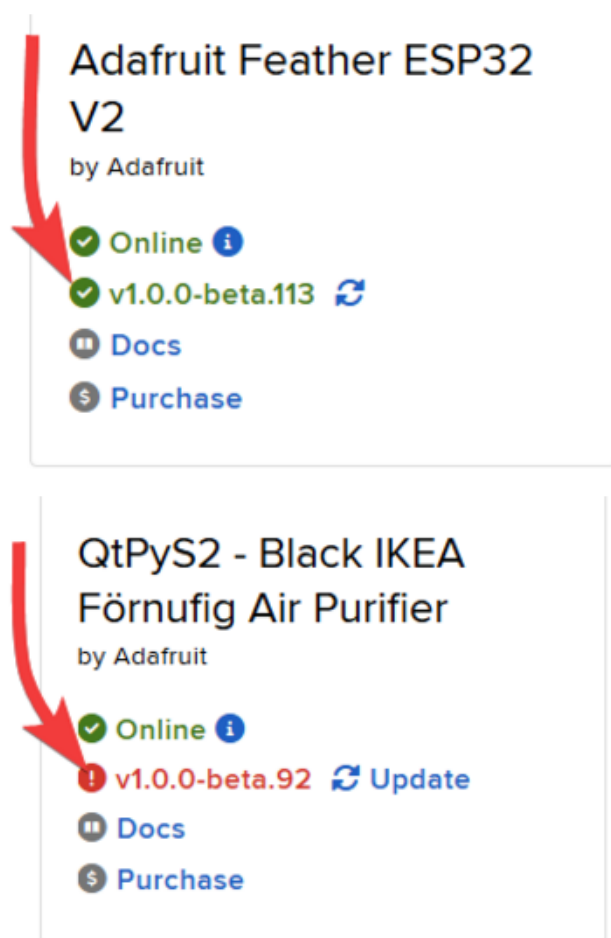
On that page, **select the WipperSnapper board you're using** to be brought to the board's interface page.



Text This guide page assumes you have **already installed** emphasized **WipperSnapper on your board**, and it is showing as "Online" with a on the Devices page. If not, then follow the quick start guide yellow linked above, or visit the [New Device page](#) and follow the on-exclamation:screen instructions.



If you do not see your board listed here - you need [to connect your board to Adafruit IO](https://adafru.it/Vfd) (<https://adafru.it/Vfd>) first.



On the device page, quickly check that you're running the latest version of the WipperSnapper firmware.

The device tile on the left indicates the version number of the firmware running on the connected board.

If the firmware version is green with a checkmark - continue with this guide. If the firmware version is red with an exclamation mark "!" - [update to the latest WipperSnapper firmware](https://adafru.it/Vfd) (<https://adafru.it/Vfd>) on your board before continuing.

Click the **New Component** button or the **+** button to bring up the component picker.



Adafruit IO supports a large amount of components, try searching for **Display** to see all the supported displays.



Filtering and searching for components

Since WipperSnapper supports such a large number of components, you can use keyword filtering. Try searching for various keywords, like:

- component names: **aht20** , **servo** , **buzzer** , **button** , **neopixel** , etc
- sensor types: **light** , **temperature** , **pressure** , **humidity** , etc
- interface: **i2c** , **uart** , **ds18x20** , **pin** , **display** , etc (also I2C addresses e.g. **0x44**)
- vendor: **Adafruit** , **ASAIR** , **Infineon** , **Bosch** , **Honeywell** , **Sensirion** , etc
- display type / driver / family: **st7789** , **ili0373** , **sh1107** , **seg** , **lcd**

There are also product and documentation links for every component. Follow the links beneath the component descriptions to be taken to the appropriate product page, Learn Guide, or datasheet.

New Component

Which component would you like to set up?

SSD1680

Displaying 1 matching Components.



To quickly find your display, type **SSD1680** into the search bar, then select the **2.9" Grayscale elnk (SSD1680)** component.

On the component configuration page, the display's bus number and/or address should be listed along with the components settings.

Select all the pin options that match your wiring setup (leaving any drop-downs for any unattached pins unpopulated).

Finally choose any preferences like font size or alignment, then click **Create**.

Create 2.9" Grayscale elnk (SSD1680) Component



Settings

2.9" Grayscale elnk (SSD1680) Name

2.9" Grayscale elnk (SSD1680)

SPI Bus

0

Data/Command Pin

D33

Reset Pin

Choose a Pin

Chip Select Pin

D15

SRAM Chip Select Pin

Choose a Pin

Busy Pin

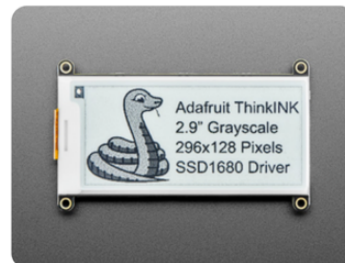
Choose a Pin

EPD Mode

Monochrome

Text Size

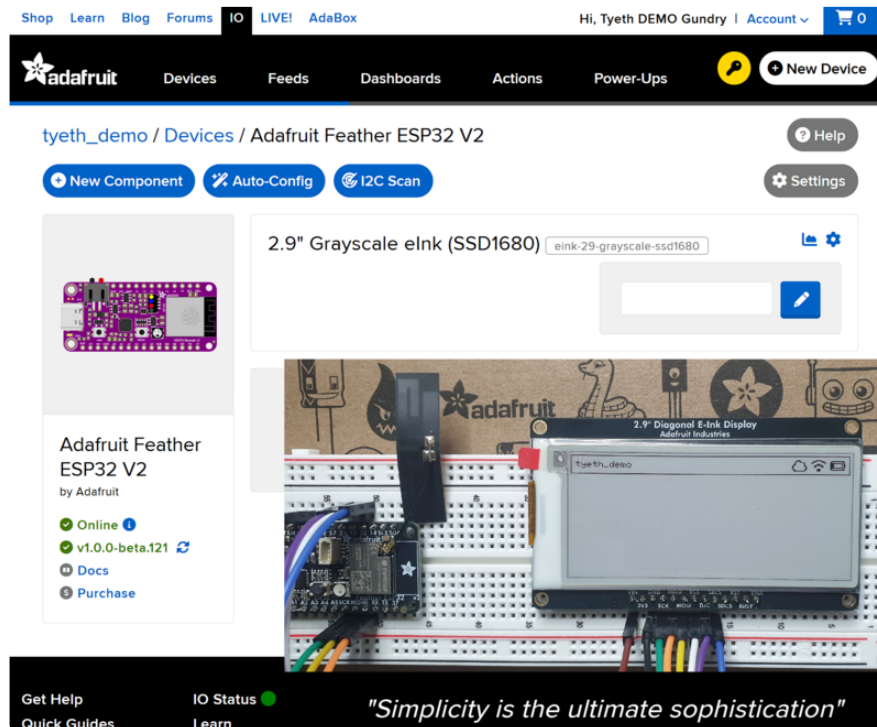
24px



[← Back to Component Type](#)

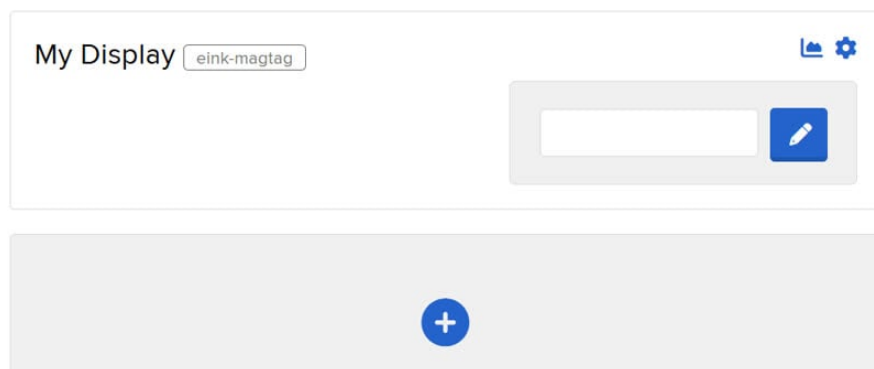
[Create Component](#)

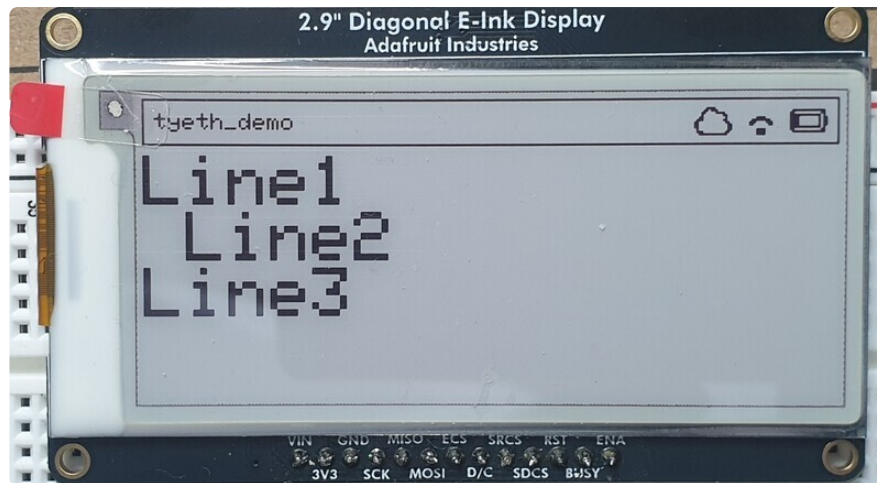
After adding the component, your device page should show the newly created display component row, and the display attached to your board should refresh / turn on when the component is added.



If appropriate, your display may also show a status bar with connection info (username, connection status, WiFi strength, and battery level).

To send a message to the display, click on the Pen button at the end of the display component row, then fill in the dialog and click **Send**:





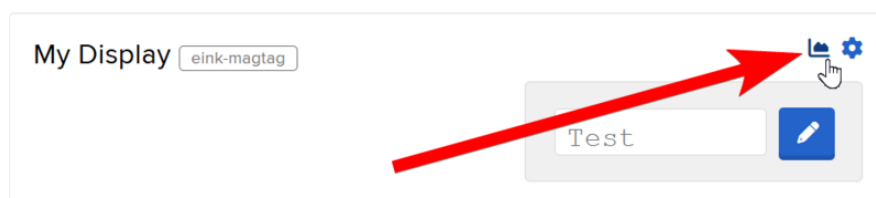
Text emphasized with a blue exclamation:

To send a forced line break, use the backslash symbol (\) followed by the letter n for new line: \n e.g. `Line1 \n Line2\nLine3` The displays do accept true line breaks too (hitting the Enter/Return key in the text input box above, or sending \n or \r\n [via JSON](#)).

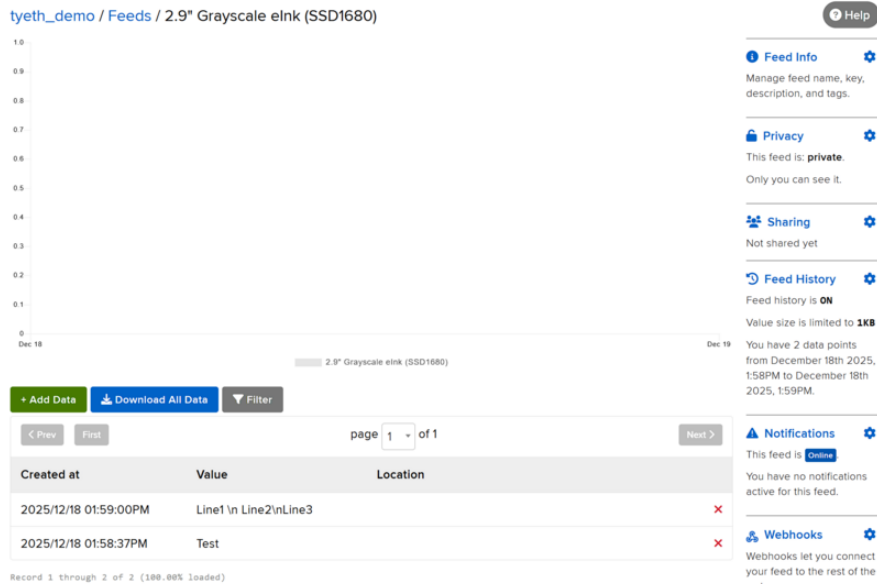
The display component is connected to a feed, which means that you can send data to the feed and have it immediately appear on screen.

This can be really useful when combined with automated [Actions](https://adafru.it/1axy) (<https://adafru.it/1axy>).

It also means there is a history of messages. To view the feed data that has been previously sent to the display, click on the graph icon at the end of the component row.



Here you can see the feed history and edit things about the feed such as the name, privacy, webhooks associated with the feed and more. If you want to learn more about how feeds work, [check out this page](https://adafru.it/10aZ) (<https://adafru.it/10aZ>).



Lastly, take note of the Settings Gear (⚙️) at the end of the component row, where you can alter the components configuration.

This will show you the same dialog as when you added the display, along with any advanced options. It's also where you can remove components.

Continue on with this guide, or maybe look at setting up an [Action \(https://adafru.it/YAT\)](https://adafru.it/YAT) to write to the display.

Downloads

Text emphasized with a blue exclamation:

2.9" Tri-Color eInk Display Revision History As of June 20, 2025 - The display has been updated (old ones discontinued) to use a spiffy modern SSD1680 chip - we have support in CircuitPython/Python/Arduino for it! As of May 22th, 2024 – We've updated this PCB with a new EYESPI connector along with the Adafruit Penguin to make a lovely and legible silkscreen. As of November 16, 2022 - this revision has been updated to have a 2.9" Tri-Color eInk display only with a UC8151D chipset.

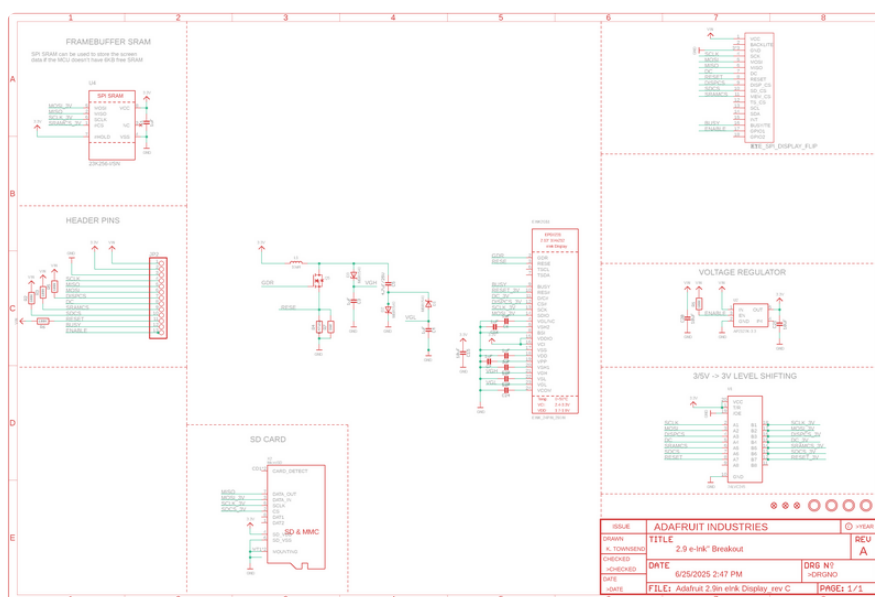
2.9" Tri-Color eInk FeatherWing Revision History As of June 24th, 2025 – The display has been updated (old one discontinued) it now uses a spiffy modern SSD1680 chip - we emphasized have support in CircuitPython/Python/Arduino for it! As of Sep 8, 2023 – we've updated this PCB with Adafruit Pinguin to exclamation:make a lovely and legible silkscreen - you may get the new PCB or the older version with vector fonts - both are identical other than the fancy silkscreen.

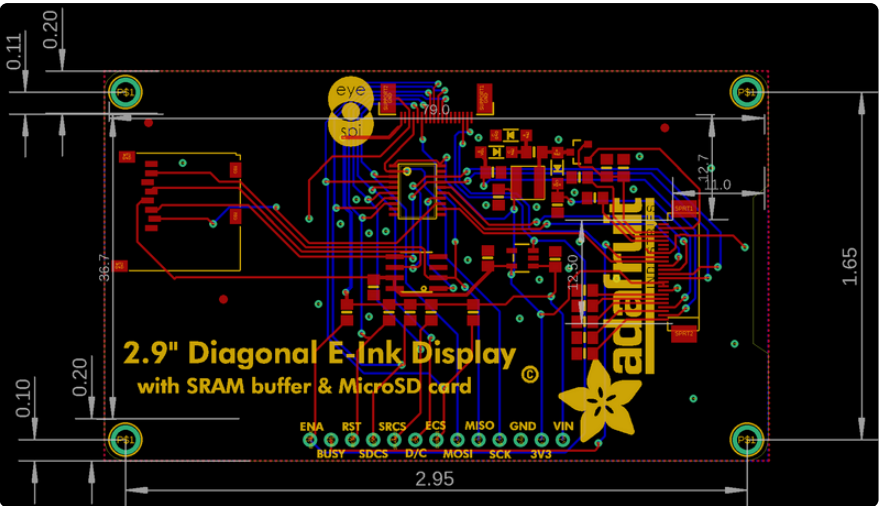
Files

- [Fritzing object in Adafruit Fritzing Library \(https://adafru.it/aP3\)](https://adafru.it/aP3)
- [IL0376F E-Ink interface chip datasheet \(https://adafru.it/BRW\)](https://adafru.it/BRW)
- [UC8151D Datasheet \(https://adafru.it/1alu\)](https://adafru.it/1alu)
- [SSD1680 Datasheet \(https://adafru.it/1alt\)](https://adafru.it/1alt)
- [PCB Files on GitHub \(https://adafru.it/BRX\)](https://adafru.it/BRX)

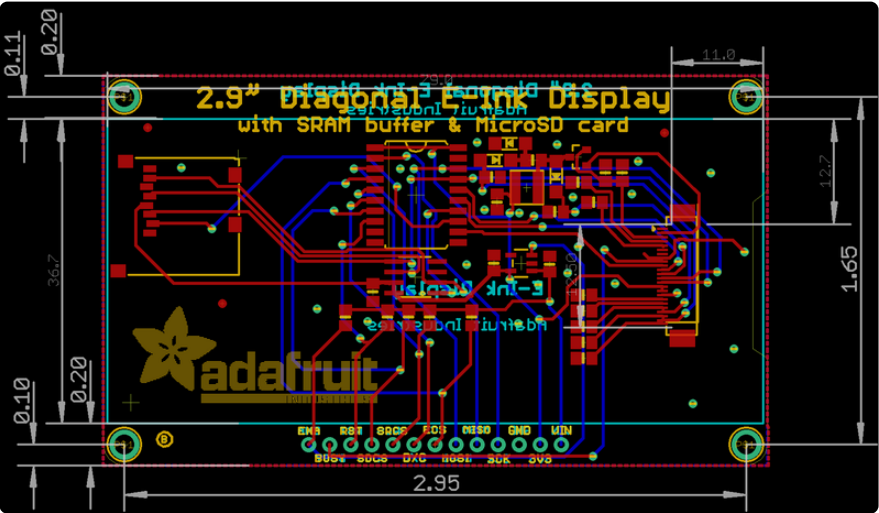
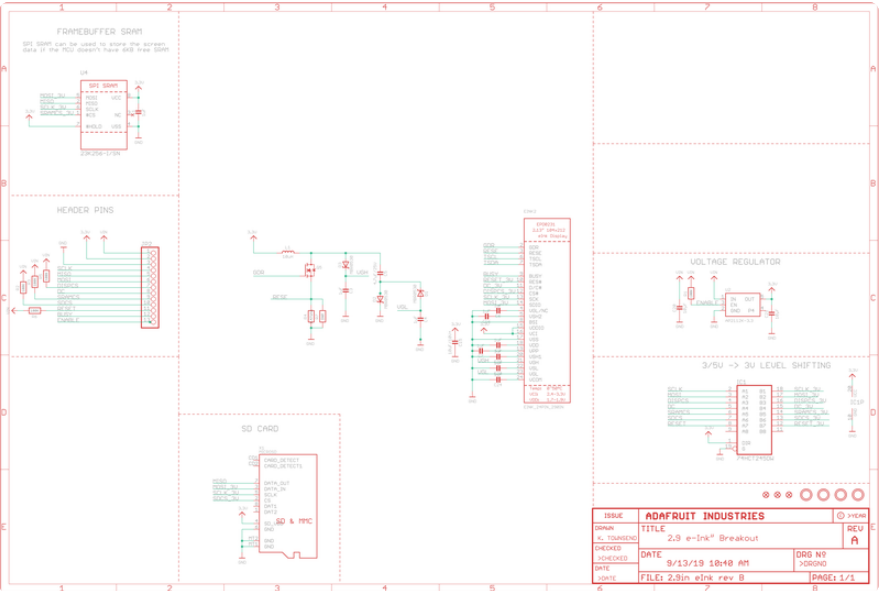
Schematic & Fabrication Prints

Rev C

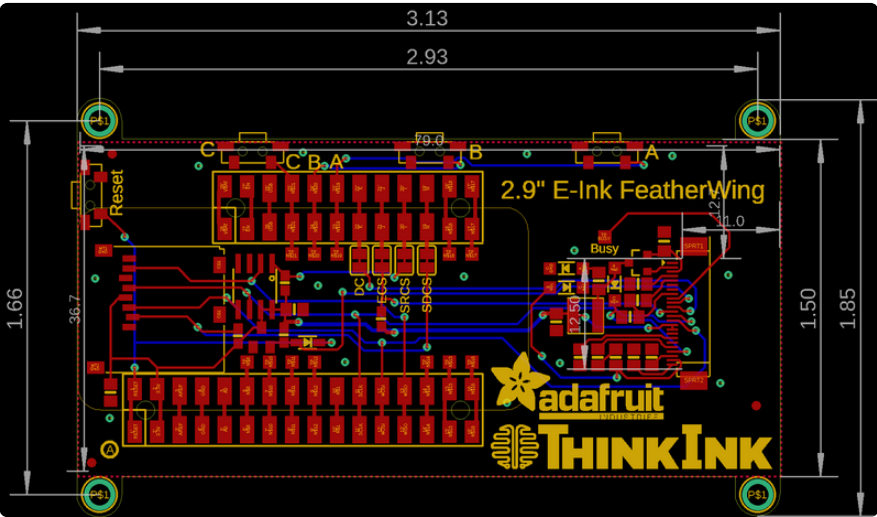
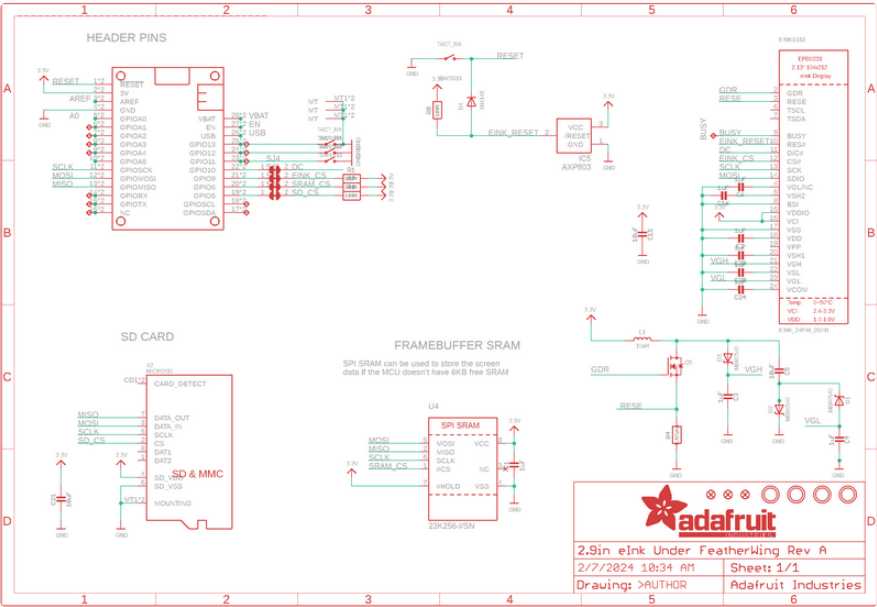




Rev B



2.9" FeatherWing



eInk Friends

